

Machine Learning

Note di corso, Progetti e Casi Studio



Università
di Catania

Università degli Studi di Catania
Dipartimento di Matematica e Informatica
Corso di Laurea Triennale in Informatica (L-31)

Autori

Emanuele Galiano
Damiano Trovato

Revisione

coming soon

Anno accademico

Anno Accademico 2025/2026

Versione: 18 febbraio 2026

Licenza

Questo documento, intitolato *Machine Learning – Note di corso, Progetti e Casi Studio*, è distribuito sotto licenza **Creative Commons Attribution–ShareAlike 4.0 International (CC BY-SA 4.0)**.

La presente licenza è stata scelta con l’obiettivo di favorire la diffusione, la condivisione e il riutilizzo del materiale didattico, garantendo al contempo il riconoscimento degli autori originali e la preservazione della natura open delle opere derivate.

Autori: Emanuele Galiano

Damiano Trovato

Affiliazione: Università degli Studi di Catania – Dipartimento di Matematica e Informatica

Anno accademico: Anno Accademico 2025/2026

Diritti concessi

In conformità con i termini della licenza Creative Commons BY-SA 4.0, è consentito a chiunque di:

- copiare e ridistribuire il materiale in qualsiasi mezzo o formato;
- adattare, modificare e trasformare il materiale;
- utilizzare il materiale anche per scopi commerciali.

Tali diritti sono concessi a titolo gratuito e non possono essere revocati, purché siano rispettate le condizioni indicate nella sezione seguente.

Condizioni

L’utilizzo del materiale è subordinato al rispetto delle seguenti condizioni:

- **Attribuzione (BY):** deve essere fornita un’adeguata attribuzione dell’opera, citando **titolo**, **autori** e, ove possibile, la **fonte**. L’attribuzione deve essere effettuata in modo ragionevole e non tale da suggerire che gli autori originali approvino l’uso o le modifiche apportate.

- **Condividi allo stesso modo (SA):** nel caso in cui il materiale venga modificato, trasformato o utilizzato per creare opere derivate, tali opere devono essere distribuite sotto la *stessa licenza* Creative Commons Attribution–ShareAlike 4.0 International.

Non è consentito applicare termini legali o misure tecnologiche che limitino giuridicamente altri utenti dall'esercitare i diritti concessi dalla licenza.

Assenza di garanzia

Il materiale è fornito “*così com'è*”, senza garanzie di alcun tipo, esplicite o implicite. In particolare, gli autori non garantiscono l'accuratezza, la completezza o l'assenza di errori nel contenuto del documento e declinano ogni responsabilità per eventuali danni derivanti dall'uso del materiale.

Testo completo della licenza

Il testo legale completo della licenza Creative Commons Attribution–ShareAlike 4.0 International è disponibile al seguente indirizzo:

<https://creativecommons.org/licenses/by-sa/4.0/>

Copyright © 2026
Emanuele Galiano
Damiano Trovato

Indice

Licenza	iii
Diritti concessi	iii
Condizioni	iii
Assenza di garanzia	iv
Testo completo della licenza	iv
 I Concetti Preliminari	 1
 II Teoria, Algoritmi e Modelli di Machine Learning	 3
 1 Introduzione al Machine Learning	 5
1.1 Prime definizioni di Machine Learning	5
1.1.1 Arthur Samuel - 1959	5
1.1.2 Perché abbiamo bisogno del Machine Learning	5
1.1.3 Esempio delle email spam e non-spam	6
1.1.4 Tom Mitchell - 1998	6
1.2 Definizioni fondamentali	6
1.2.1 Definizione di Task	6
1.2.2 Definizione di Esperienza	6
1.2.3 Definizione di Performance	7
1.2.4 Esempio completo	7
1.3 Task, esempi ed etichette	8
1.4 Features	8
1.4.1 Estrazione	8
1.4.2 Definizione formale	9
1.5 Esempio delle Email Spam e Non Spam	9
1.6 Tipologie di Task	9
1.6.1 Classificazione	10
1.6.2 Regressione	11
1.7 Tipi di Machine Learning	11
1.7.1 Supervised Learning e Unsupervised Learning	11
1.7.2 Reinforcement Learning	12
1.8 Misura di Performance (P)	13
1.8.1 Esempio	13

1.9	Elementi fondamentali del Machine Learning	14
1.9.1	Experience (E)	14
1.9.2	Dataset (D)	14
1.9.3	Design Matrix	14
1.9.4	Esempio	14
1.9.5	Learning	15
1.9.6	Rischio atteso e rischio empirico	16
1.9.7	Il paradigma Training/Testing	16
1.9.8	Normalizzazione	17
1.10	Iperparametri	18
1.10.1	k-fold Cross Validation	18
1.11	Overfitting e underfitting	18
1.11.1	Bias e Variance	19
2	I primi modelli di Machine Learning	21
2.1	Neurone computazionale - modello di McCulloch-Pitt	21
2.1.1	Limitazioni del modello di McCulloch-Pitt	22
2.2	Percettrone - Rosenblatt	22
2.2.1	Processo generale di training del percettrone	23
2.2.2	Implementazione del training	24
2.2.3	Versione di Adaline	24
2.3	Modelli lineari	25
2.3.1	Problema dello XOR	25
2.3.2	Rete di percettroni	26
3	Regressione	29
3.1	Ingredienti del task	29
3.1.1	Tipi di regressione	29
3.2	Regressione lineare semplice	30
3.2.1	Funzione di loss	30
3.3	Regressione lineare multipla	30
3.4	Feature scaling	30
3.5	Algoritmo di discesa del gradiente	31
3.5.1	Idea dell'algoritmo	31
3.5.2	Algoritmo generale	31
3.6	Regressione polinomiale e feature mapping	32
3.7	Regolarizzazione	32
3.8	Valutazione	33
3.8.1	REC curve	33
4	Classificazione	35
4.1	Introduzione alla classificazione	35
4.1.1	Classificazione: binaria e di classe	35
4.2	Classificazione binaria	35
4.2.1	Perché non usare la regressione lineare per la classificazione?	36

4.3	Regressione logistica - Sigmoidale	37
4.3.1	Vantaggi relativi all'uso della sigmoide	38
4.4	Funzione di Loss	38
4.4.1	Derivata parziale della funzione di costo	39
4.4.2	Entropia dell'informazione	39
4.4.3	Binary Cross Entropy Loss	40
4.5	Overfitting nella classificazione	41
4.6	Reject Region, regione d'incertezza	41
4.7	Classificazione multiclasse, approccio naive OVA e OVO	42
4.7.1	Metodo One vs All	42
4.7.2	Metodo One vs One	43
4.7.3	Confronto tra <i>One vs All</i> e <i>One vs One</i>	43
4.8	Stochastic Gradient Descent	44
4.9	Softmax - Classificazione Multiclasse	44
4.9.1	Definizione	44
4.9.2	Funzione Softmax	45
4.9.3	Funzione di Costo	45
4.9.4	Obiettivo di Learning	45
4.9.5	Proprietà del Softmax	46
4.9.6	Rete neurale Softmax	46
4.9.7	Funzione di Loss	47
5	Design, valutazione e scelta dei parametri di un algoritmo di ML	49
5.1	Momento e gradiente	49
5.2	Design di Algoritmi	49
5.2.1	Strategie di miglioramento	49
5.3	Valutazione	50
5.3.1	Valutazione incrociata	50
5.3.2	Bias e Varianza	52
5.3.3	Parametri di regolarizzazione	54
5.3.4	Altre curve di valutazione	54
6	Introduzione alle reti neurali	57
6.1	Definizione di rete neurale	57
6.2	Le funzioni di attivazione	59
6.3	Il teorema di approssimazione universale per le reti neurali	60
6.4	Training e inferenza di una rete neurale	61
6.4.1	Funzione di costo	61
6.4.2	Chain Rule	62
6.4.3	Inizializzazione dei parametri	62
6.4.4	Reti neurali per regressione	62
6.5	Overfitting e Underfitting nelle reti neurali	62
6.6	Caso studio: MLP	63
6.6.1	Definizione del modello	63
6.6.2	Funzione di costo	63

6.6.3	Backpropagation	64
6.6.4	Aggiornamento dei parametri	64
7	Decision Tree	67
7.1	Alberi di classificazione	67
7.1.1	Divisione ricorsiva	67
7.1.2	Migliore divisione dei nodi	68
7.1.3	Algoritmo di costruzione dell'albero	69
7.2	Alberi di regressione	70
7.2.1	Costruzione dell'albero di regressione	70
7.2.2	Divisione di un nodo	71
7.3	Pruning: riduzione dell'albero	71
7.3.1	Pre-pruning	71
7.3.2	Post-pruning	72
7.4	Regole di learning	72
7.4.1	Rule induction	73
7.4.2	Copertura delle regole	73
7.4.3	Algoritmo RIPPER	73
7.5	Alberi decisionali multivariati	74
7.5.1	Alberi decisionali multivariati lineari	74
7.5.2	Interpretazione geometrica	75
7.5.3	Alberi decisionali multivariati non lineari	75
8	Random Forests, Ferns, Meta-Algoritmi	77
8.1	Meta algoritmi nel machine learning	77
8.2	Decision stump - Ceppo di decisione	77
8.3	Decision Trees	78
8.4	Random Ferns	78
8.4.1	Feature binarie	78
8.4.2	Formulazione probabilistica	79
8.4.3	Approccio Semi-Naive Bayes	79
8.4.4	Stima delle probabilità	79
8.5	Random Tree	80
8.5.1	Randomized Learning	80
8.5.2	Guadagno di informazione	81
8.5.3	Ruolo della casualità	81
8.5.4	Randomized Learning	81
8.6	Random forest	81
9	SVM: Support Vector Machines	83
9.1	Iperpiani di separazione	83
9.1.1	Funzione di decisione lineare	83
9.1.2	Interpretazione geometrica	84
9.2	Primal SVM	85
9.2.1	Concetto di margine	85

9.2.2	Derivazione tradizionale	87
9.3	Soft Margin SVM	87
9.3.1	Soft Margin SVM e Loss function	88
9.4	Dual SVM	89
9.5	Kernel Trick	90
9.5.1	Trasformare i dati	90
9.5.2	Il <i>Kernel Trick</i>	90
10	Ensemble Learning	93
10.1	Tipi di Ensemble Learning	93
10.1.1	Averaging	93
10.1.2	Weighted Averaging	94
10.1.3	Gating	94
10.1.4	Stacking	94
10.2	Tecniche di Manipolazione dei Dati	95
10.2.1	Bootstrap replication	96
10.2.2	Bagging	96
10.2.3	Boosting	96
10.3	AdaBoost	96
10.3.1	Algoritmo	96
10.3.2	Decision Boundary	98
III	Esempi di Progetti	101
A	Classificatore di Risonanze magnetiche	103
A.1	Problema	104
A.2	Dataset	104
A.3	Metodi	105
A.3.1	ResNet-18	105
A.3.2	RadNet-50	107
A.3.3	Custom-CNN	109
A.4	Valutazione	113
A.5	Demo	114
A.5.1	Obiettivo della demo	114
A.5.2	Tecnologie utilizzate	114
A.5.3	Funzionalità principali	114
A.5.4	Modalità di utilizzo	114
A.6	Codice	115
A.7	Conclusioni	116
IV	Materiale Aggiuntivo	117
B	Approfondimenti teorici	119
B.1	Problema della separabilità non lineare nel perceptrone	119

B.2	Generalizzazione funzione di costo del softmax	120
B.3	Dimostrazione del teorema del margine a 1	121
B.4	Dual SVM	122
B.4.1	Convex duality con Moltiplicatori di Lagrange	123
C	Seminario 1: <i>Data and AI Law</i>	125
C.1	L'importanza dei dati - segreti industriali	125
C.1.1	Data Is The New Oil	125
C.1.2	<i>Mission Genesis</i> - 25 Novembre 2025	125
C.1.3	Norme: Data Protection e Copyright	126
C.2	Data Protection	126
C.2.1	GDPR - La rivoluzione	126
C.2.2	Sui Cookies	127
C.3	Copyright e diritto d'autore	127
C.3.1	Diritto d'autore	127
C.3.2	Alla tutela del Software	127
C.4	AI e Copyright	128
C.4.1	Posizioni dell'US Copyright Office	128
C.4.2	In Italia	128
C.4.3	<i>Un primo passo</i> - In Sud Africa	129
C.4.4	AI e soggettività giuridica	129
C.4.5	Caso OpenAI	129
C.5	AI e opere derivate	129
C.6	AIG e addestramento	129
C.6.1	Caso New York Times e OpenAI	129
C.6.2	FIEG contro Google AI	130
C.7	AI Act	130
C.8	AI Risk Pyramid	130
C.9	L'Italia	130
C.9.1	<i>"-umano"</i> - Aggiunta all'articolo 1	131
C.9.2	Reati relativi all'uso dei dati	131
C.10	Una soluzione ai problemi	131
D	Seminario 2: <i>Computer Vision and Medical Imaging</i>	133
D.1	Intelligenza Artificiale	133
D.1.1	Machine Learning	133
D.1.2	Machine learning e Deep Learning	133
D.2	Caso studio 1: radiomica	133
D.2.1	Pipeline radiomica	134
D.2.2	Work 1: feature extraction	135
D.2.3	Work 2: Dynamic Feature Selection (DFS)	135
D.2.4	Work 3: Federated Feature Selection	135
D.3	Caso studio 2: immagini di cellule del sangue nei vetrini	136
D.4	Machine Learning is not dead.	136

Parte I

Concetti Preliminari

Parte II

Teoria, Algoritmi e Modelli di Machine Learning

Capitolo 1

Introduzione al Machine Learning

1.1 Prime definizioni di Machine Learning

1.1.1 Arthur Samuel - 1959

Nel 1959, Arthur Samuel fornisce una **definizione di machine learning**: il machine learning è il campo di studi che abilita i computer ad **imparare, senza essere esplicitamente programmati**.

Il paradigma alla base è fortemente diverso da quello tipico: se un algoritmo classico è **programmato ad hoc** per risolvere un task e si comporta in maniera prettamente **deterministica**, un algoritmo di machine learning può **imparare a risolvere problemi** associati a vasti insiemi di dati (classificare elementi, riconoscerli, trovare correlazioni). Questo permette di spostare il focus: non più sullo sviluppo di tutti gli step necessari affinché un algoritmo risolva **un problema specifico**, ma sulla creazione di un modello che riesca ad apprendere come risolvere **un insieme di problemi simili**.

1.1.2 Perché abbiamo bisogno del Machine Learning

L'approccio utilizzato finora per risolvere i problemi è quello di:

- Trovare una logica per risolvere il problema.
- Scrivere un programma.
- Suddividerlo in pezzi più piccoli (funzioni).
- Automatizzare l'approccio.

Questo funziona bene per problemi di natura fortemente univoca, che sappiamo come risolvere, ad esempio:

- Computare l'area di un poligono.
- Risolvere equazioni differenziali.

Nel caso del poligono, supponendo di voler calcolare l'area di un rombo, i dati presi in input sarebbero dati dalla coppia (x_1, x_2) , contenente le lunghezze della diagonale principale e secondaria. Questi dati, passati ad un algoritmo, permettono di calcolarne l'area $\frac{x_1 x_2}{2}$ e generarne un output:

Dati $\rightarrow$ Programma che risolve un task $\rightarrow$ Output.

Alcuni problemi tuttavia presentano un alto grado di **incertezza**, che li rende più difficili da affrontare. Non poter fare assunzioni sui dati in input, e non conoscere tutti i possibili task, rende impossibile l'utilizzo di algoritmi standard per compiti del tipo:

- Classificazione di email spam e non spam.
- Object detection.

Il machine learning rappresenta la soluzione ideale a problemi di questo tipo, proponendo una nuova pipeline:

Dati + Output atteso $\rightarrow$ Machine Learning $\rightarrow$ Soluzioni su nuovi dati.

1.1.3 Esempio delle email spam e non-spam

Vogliamo creare un algoritmo di machine learning in grado di determinare se una mail è spam o meno. Il nostro obiettivo è quindi classificare ciascuna di queste come **spam**, o **ham**¹.

- Compra prodotto a 10\$! Offerta imperdibile! $\rightarrow$ Spam
- Ciao Giovanni, come stai? $\rightarrow$ Ham

1.1.4 Tom Mitchell - 1998

Un algoritmo **apprende dall'esperienza** E rispetto a una certa classe di **task** T e a una misura di **performance** P . Se la sua **performance** nel compito T , misurata tramite P , migliora con l'**esperienza** E , allora quel modello ha appreso con successo.

1.2 Definizioni fondamentali

1.2.1 Definizione di Task

Il **task** rappresenta il problema che deve essere risolto. Nell'esempio di determinare se una mail è spam o meno, il task è quello di **predire** l'etichetta ($Y = \text{"spam"}$ oppure $Y = \text{"ham"}$), ed è strettamente legato al modello, che rappresentiamo come funzione parametrizzata, indicata con h_{θ} .

1.2.2 Definizione di Esperienza

L'esperienza si definisce come l'insieme di informazioni a cui il modello è esposto durante il processo di apprendimento e a partire dalle quali aggiorna (ossia modifica) i propri parametri interni.

¹Email legittima, non spam.

Supervised learning. Nel **supervised learning**, l'esperienza è un dataset di esempi osservati, cioè un insieme di *realizzazioni* delle variabili aleatorie (X, Y) . Denotiamo il dataset con:

$$D = \{(x^{(i)}, y^{(i)})\}_{i=1}^m,$$

dove $x^{(i)}$ e $y^{(i)}$ sono rispettivamente input e output osservati per l'esempio i -esimo. In genere si assume che gli esempi siano estratti in modo i.i.d. da una distribuzione incognita $\mathcal{D}$.

Reinforcement learning. Nel **reinforcement learning**, l'esperienza assume una forma più generale: essa coincide con qualunque interazione che il modello ha con l'ambiente. Ad esempio, se il modello sta cercando di uscire da un labirinto, la sua esperienza al tempo t può essere rappresentata dalla tupla:

$$E_t = (S_t, A_t, R_t, S_{t+1}),$$

dove S_t corrisponde allo stato attuale in cui si trova il modello, A_t all'azione intrapresa dato lo stato corrente, R_t alla ricompensa ricevuta a seguito dell'azione e S_{t+1} allo stato successivo.

In sintesi: l'esperienza è l'informazione (storico dei dati oppure interazioni con l'ambiente) a cui il modello è esposto e da cui impara, cioè tramite cui modifica i suoi parametri interni.

1.2.3 Definizione di Performance

La misura di performance P valuta quanto bene il modello risolve un task T . In alcuni casi P è una metrica da **massimizzare** (per esempio l'accuracy, con valori in $[0, 1]$), mentre in altri casi è più naturale definire una **loss** ℓ da **minimizzare** (per esempio il mean squared error, con valori in $[0, +\infty)$).

Supponiamo che il nostro algoritmo abbia previsto un insieme di etichette per un dato numero di email che indichiamo con:

$$\{\hat{y}_i\}.$$

Dove il simbolo “hat” indica che il dato non è stato osservato ma **previsto**. L'insieme delle etichette corrette è invece dato da

$$\{y_i\}.$$

Per valutare la qualità del metodo, possiamo confrontare i due insiemi con una misura di performance:

$$P(\{y_i\}, \{\hat{y}_i\}).$$

Se P è normalizzata (come l'accuracy), possiamo interpretare $1 - P$ come errore (*error rate*). In generale, per definire l'errore si usa una loss ℓ .

1.2.4 Esempio completo

Siano:

- $x^{(1)}$: Il testo dell'email 1: “Compra prodotto a 10\$! Offerta imperdibile!”
- $x^{(2)}$: Il testo dell'email 2: “Ciao Giovanni, come stai?”
- $y^{(1)}$: L'etichetta **spam**

- $y^{(2)}$: L'etichetta **ham**
- h_θ : Il modello

Allora

$$h_\theta(x^{(1)}) = \hat{y}^{(1)} \quad \text{e} \quad h_\theta(x^{(2)}) = \hat{y}^{(2)}.$$

1.3 Task, esempi ed etichette

Un esempio (o osservazione) è una raccolta di valori misurati. Un esempio è rappresentato da un vettore:

$$x \in \mathbb{R}^d,$$

scritto anche come:

$$x = (x_1, x_2, \dots, x_d).$$

I valori del vettore x sono detti **features**, in quanto rappresentano **attributi misurabili** (o una rappresentazione numerica) dell'input. Se la dimensionalità di x è d , diremo che l'esempio ha d features. Nella maggior parte dei casi, ogni esempio x è abbinato a un output desiderato y , detto **etichetta**. Un task è quindi un modo di elaborare un input x per ottenere un output y .

Torniamo al nostro esempio: determinare se un'e-mail è spam o ham. In questo caso, l'input è l'e-mail; le features possono essere caratteristiche dell'e-mail, come il numero di errori ortografici o la presenza di alcune parole chiave; l'output atteso è l'etichetta (spam o ham).

1.4 Features

1.4.1 Estrazione

Per gestire le e-mail, dobbiamo prima trasformarle in un'entità **quantificabile**. Questo di solito viene fatto **identificando alcune caratteristiche** dei dati che sono **rilevanti per il compito dato** (numero di errori ortografici o presenza di alcune parole chiave). In pratica, stiamo cercando una funzione f che trasformi l'entità dalla sua forma originale a una forma di destinazione, adatta a risolvere un compito specifico:

$$x \rightsquigarrow f(x) \rightsquigarrow \bar{x}.$$

Dove x è il dato grezzo di input (ad esempio, il messaggio di posta elettronica completo), f è la funzione di trasformazione e $\bar{x}$ ² è l'output della trasformazione, che sarà l'input dell'algoritmo di apprendimento automatico.

La funzione f è chiamata *rappresentazione*. L'output della trasformazione è anch'esso chiamato rappresentazione. Poiché rappresentando i dati otteniamo un vettore di features, il processo di rappresentazione dei dati è talvolta chiamato **feature extraction**. Non esistono "rappresentazioni universali", ma solo rappresentazioni utili a un determinato compito.

Le rappresentazioni sono di due tipi:

- Create a mano.
- Apprese.

²Su questa notazione: da ora in poi, quando ci riferiremo all'output della trasformazione, non useremo più $\bar{x}$, ma direttamente x , dando per scontato il passaggio di rappresentazione $f(x)$.

L'estrazione delle features **mette in luce caratteristiche salienti** trascurandone altre.

1.4.2 Definizione formale

Generalmente, l'output di una funzione di rappresentazione è nella forma:

$$x = (x_1, x_2, \dots, x_d), \quad x \in \mathbb{R}^d.$$

Questo è composto da un insieme di features. **Una feature è la specifica di un attributo misurabile** dell'input: una quantità che rappresenta **aspetti dei dati** utili per risolvere il problema considerato. Ad esempio, il colore può essere un attributo; "il colore è blu" è una feature estratta da un esempio.

Le features possono essere di due tipi principali:

Categoriche: un numero finito di valori discreti. Questi possono essere:

- **Nominali:** non esiste **alcun ordinamento** tra i valori (es. cognomi, colori).
- **Ordinali:** esiste un **ordinamento rilevante** (es. basso, medio, alto).

Continue: comunemente un **sottoinsieme di numeri reali**, dove i valori sono ordinati e confrontabili. I numeri interi sono solitamente trattati come continui nei problemi pratici.

1.5 Esempio delle Email Spam e Non Spam

Consideriamo il nostro esempio in cui vogliamo distinguere le e-mail spam da quelle non spam. L'input del processo sono i messaggi di posta elettronica, quindi dobbiamo trasformarli in vettori di features:

$$x = (x_1, x_2, \dots, x_d),$$

con un processo di *feature extraction*.

Naturalmente, ci aspettiamo che le features estratte siano **utili per risolvere il nostro compito** di determinare se un'e-mail è spam o ham. Possiamo notare che le e-mail di spam spesso includono errori ortografici e parole come "Acquista", "occasione" e "10\$". Quindi, potremmo decidere di rappresentare ogni messaggio di posta elettronica con due numeri (cioè $d = 2$):

- Il conteggio degli errori ortografici.
- Il numero di volte in cui alcune parole o pattern specifici appaiono nel testo.

Una volta che i messaggi di input sono stati convertiti in **vettori di features**, possono essere visti come **vettori nello spazio** $\mathbb{R}^2$.

1.6 Tipologie di Task

Le attività possono essere di diversi tipi. Di seguito, discuteremo due compiti principali:

- **Classificazione.**
- **Regressione.**

LAVORI

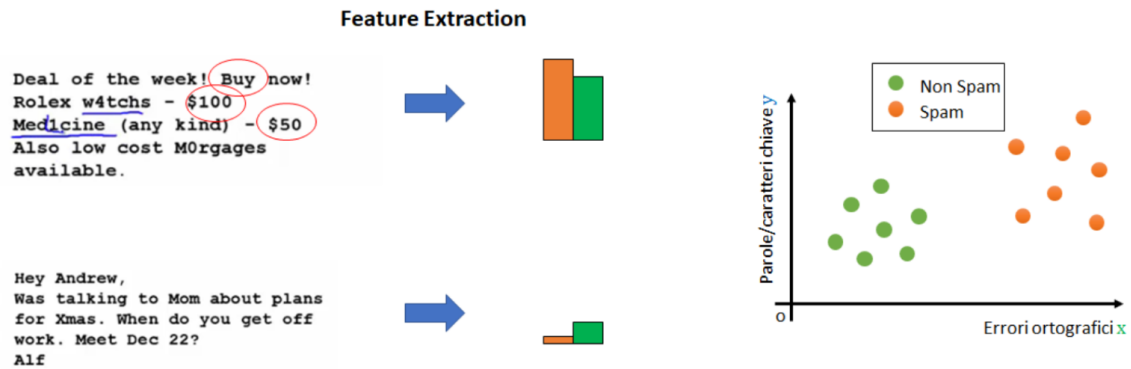


Figura 1.1: Estrazione delle feature: le e-mail vengono trasformate in vettori 2D, dove $x_1 =$ errori ortografici e $x_2 =$ pattern ripetibili, e proiettate nello spazio delle feature, dove la separazione tra spam (arancione) e non spam (verde) risulta evidente.

Assumeremo che ogni algoritmo di apprendimento automatico prenda come input esempi che sono già stati rappresentati con una funzione di rappresentazione adeguata.

1.6.1 Classificazione

In questo tipo di attività, alla macchina viene chiesto di specificare a quale di un insieme predefinito di categorie K appartiene l'input.

Esempi di questo compito sono:

- Classificare i post di Facebook come riguardanti la politica o qualcos'altro (politica vs non politica).
- Rilevamento delle e-mail di spam (spam vs e-mail legittime).
- Riconoscimento dell'oggetto raffigurato in un'immagine tra 1000 oggetti diversi.

L'algoritmo di apprendimento è solitamente fornito con un insieme di esempi:

$$\{x^{(1)}, x^{(2)}, \dots, x^{(m)}\} \quad \text{dove } x^{(i)} \in \mathbb{R}^d \forall i,$$

e un insieme di etichette corrispondenti:

$$\{y^{(1)}, y^{(2)}, \dots, y^{(m)}\} \quad \text{dove } y^{(i)} \in \{1, \dots, K\} \forall i,$$

che specificano a quale delle categorie K appartiene ogni esempio. Ad esempio, se $y^{(i)} = 3$, allora $x^{(i)}$ appartiene alla classe "3".

Nel caso della classificazione binaria (ad esempio, spam vs non spam), $y^{(i)} \in \{0, 1\}$. Per risolvere questo compito, l'algoritmo di apprendimento automatico assume la forma di una funzione:

$$h_\theta : \mathbb{R}^d \rightarrow \{1, \dots, K\},$$

tale che:

$$y^{(i)} = h_\theta(x^{(i)}).$$

Esempio:

- **Classification Task:** data un'e-mail, classificarla come spam o non spam.
- **Input:** esempi d -dimensionali $x = (x_1, \dots, x_d)$ contenenti le caratteristiche dell'e-mail.
- **Output:** etichette $y \in \{0, 1\}$ che indicano se l'e-mail è legittima o spam.

Alcuni algoritmi di classificazione **non prevedono un output discreto**, ma un vettore di **probabilità**, contenente la probabilità relativa a ciascuna delle etichette possibili. In questo caso, puntiamo ad avere la probabilità massima nello slot del vettore relativo all'etichetta vera.

1.6.2 Regressione

In questo tipo di compito, al programma del computer viene chiesto di **prevedere un valore numerico dato un input**, ad esempio:

- Prevedere il prezzo delle case date alcune caratteristiche come la città, l'età, la zona, ecc.
- Prevedere il valore futuro delle azioni di una società dai valori di altre società o da altre statistiche sul mercato.
- Contare il numero di auto presenti in un'immagine.

Analogamente alla classificazione, l'algoritmo viene fornito con esempi di training $x \in \mathbb{R}^d$ e con gli output desiderati $y \in \mathbb{R}$. L'algoritmo di apprendimento automatico assume la forma di una funzione

$$h_\theta : \mathbb{R}^d \rightarrow \mathbb{R}$$

tale che $y^{(i)} = h_\theta(x^{(i)})$.

Esempio:

- **Regression task:** predire il prezzo di una casa in base ai suoi metri quadrati.
- **Input:** dimensione della casa x (valore scalare).
- **Output:** prezzo y .

Sono algoritmi ottimi per trovare relazioni tra i dati ed effettuare predizioni.

1.7 Tipi di Machine Learning

1.7.1 Supervised Learning e Unsupervised Learning

Gli approcci di Machine Learning possono essere approssimativamente divisi in **supervised** e **unsupervised learning**.

Supervised Learning: L'algoritmo viene addestrato su un insieme di esempi di input e **output desiderati**. L'obiettivo è allenare un modello a mappare gli input agli output corretti. Il punto chiave è conoscere gli output desiderati: i dati del dataset dovranno quindi essere preventivamente **etichettati**.

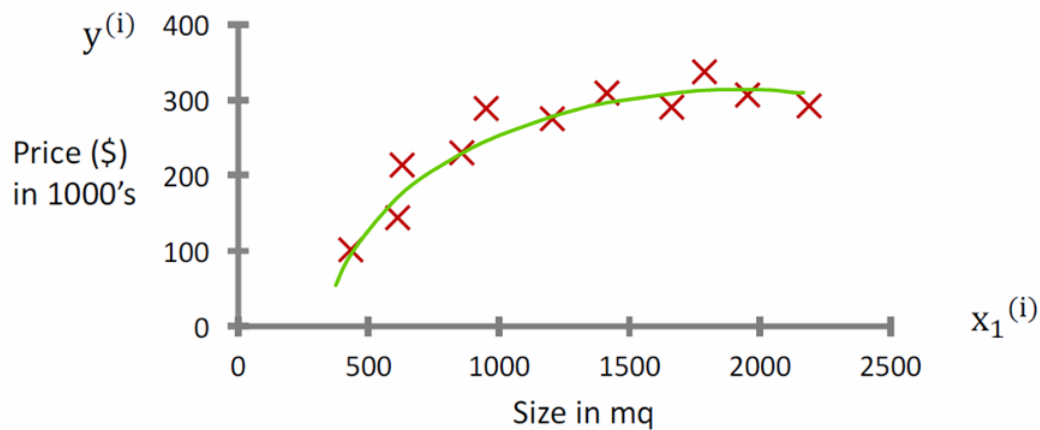


Figura 1.2: Relazione tra dimensione dell’immobile ($x_1^{(i)}$, in mq) e prezzo ($y^{(i)}$, in migliaia di \$): i punti rossi sono i dati osservati, la linea blu rappresenta un modello di regressione lineare che non approssima bene l’andamento non lineare.

Unsupervised Learning: L’algoritmo viene addestrato solo su esempi di input, senza output desiderati. L’obiettivo è trovare struttura, pattern e associazioni nei dati, spesso molto eterogenei:

$$\{x^{(1)}, x^{(2)}, \dots, x^{(m)}\} \quad \text{dove } x^{(i)} \in \mathbb{R}^d \forall i.$$

Questi tipi di compiti mirano generalmente a **modellare la struttura dei dati**. Un esempio di unsupervised learning è il **clustering**, in cui non viene fornita alcuna informazione aggiuntiva oltre agli esempi.

Gli approcci supervised sono generalmente più facili da gestire, ma richiedono la presenza di **labels**. Ottenere labels è spesso un problema costoso in termini di tempo, poiché richiede che le persone annotino manualmente i dati. Ad esempio, se dobbiamo costruire uno spam-detector con un approccio supervised, è necessario che qualcuno etichetti manualmente diverse e-mail come “spam” o “non-spam”.

1.7.2 Reinforcement Learning

Alcuni autori fanno riferimento anche a una terza classe di algoritmi di Machine Learning: il **Reinforcement Learning**.

Il Reinforcement Learning mira a **scoprire la soluzione a un problema** attraverso il metodo *trial and error*, piuttosto che tramite istruzioni esplicite su come risolvere il compito. Questo avviene permettendo all’algoritmo di **interagire con un environment** e ricevere **positive rewards** quando compie azioni che portano a un buon risultato e **negative rewards** quando compie azioni che portano a un risultato negativo.

L’obiettivo degli algoritmi di Reinforcement Learning è apprendere una policy π , che possa essere utilizzata per **determinare quale azione intraprendere** quando si acquisisce un’osservazione del mondo o . Questo processo **ricorda il modo naturale in cui gli animali imparano a risolvere problemi**. Ad esempio, si può pensare a un topo che deve trovare l’uscita da un labirinto (immagine 1.3).

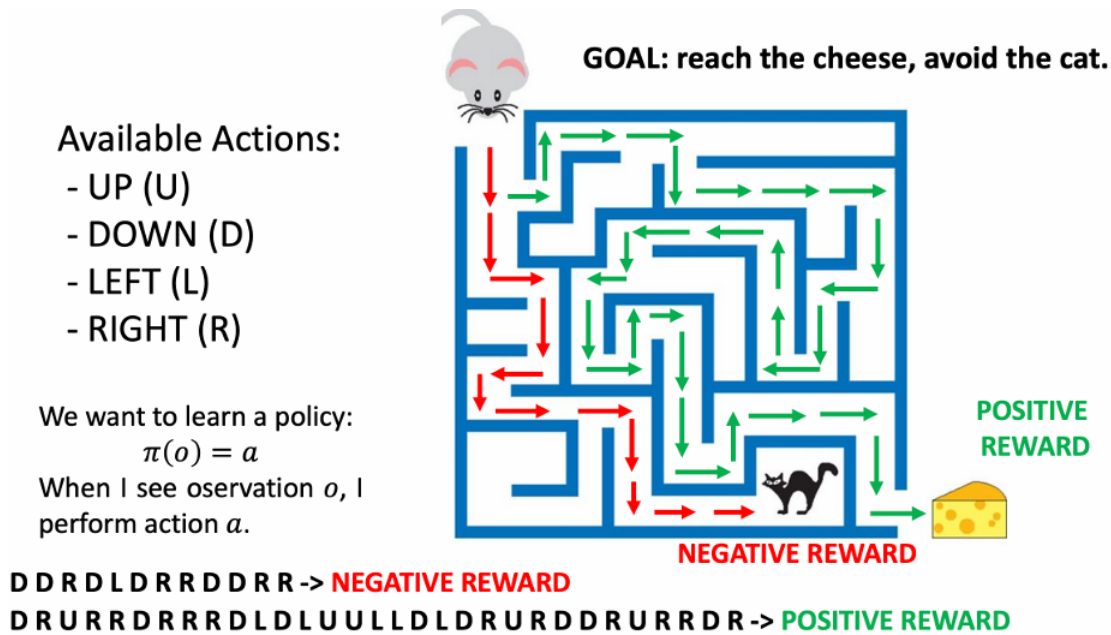


Figura 1.3: Reinforcement Learning nel labirinto: l'agente sceglie tra azioni U/D/L/R e apprende una policy $\pi(o) = a$ che massimizza la ricompensa, raggiungendo il formaggio (positiva) ed evitando il gatto (negativa).

1.8 Misura di Performance (P)

Per valutare le capacità di un algoritmo di Machine Learning nel risolvere un determinato compito, è necessaria una misura quantitativa delle sue prestazioni. Solitamente, questa **performance measure** P è **specifica per il task** T che il sistema sta eseguendo.

Per compiti come la classificazione, spesso si misura la performance utilizzando l'**accuracy**, ovvero la percentuale di esempi classificati correttamente dal modello. Nel caso della regressione, invece, si possono usare altre metriche come il mean squared error.

Le misure di performance sono utilizzate per due motivi principali:

- Capire quando un algoritmo di Machine Learning sta migliorando in un determinato compito.
- Valutare la performance dell'algoritmo una volta finalizzato.

Quando P è una metrica normalizzata (come l'accuracy), l'error rate si può calcolare come $1 - P$. Per metriche non normalizzate si usa una loss ℓ .

1.8.1 Esempio

Un spam detector analizza cinque e-mail. Le prime tre sono spam, le ultime due non lo sono. L'algoritmo classifica come spam le prime due e-mail e come non spam le ultime tre. In questo caso, la prima e le ultime due classificazioni sono corrette, mentre la terza è errata. La accuracy si calcola come la percentuale di esempi classificati correttamente:

$$\frac{4}{5} = 0.8 \quad \text{ovvero} \quad 80\%.$$

1.9 Elementi fondamentali del Machine Learning

1.9.1 Experience (E)

Un algoritmo di Machine Learning apprende dall'**experience** per migliorare una performance measure su un determinato task.

Nel supervised learning, l'experience coincide con un dataset D di esempi osservati:

$$D = \{(x^{(i)}, y^{(i)})\}_{i=1}^m,$$

con $(x^{(i)}, y^{(i)})$ realizzazioni delle variabili aleatorie (X, Y) , spesso assunte i.i.d. secondo una distribuzione $\mathcal{D}$. Nel caso unsupervised, il dataset contiene solo gli input $\{x^{(i)}\}_{i=1}^m$.

1.9.2 Dataset (D)

Le performance measures vengono generalmente calcolate rispetto a un insieme di esempi, piuttosto che su singoli esempi. Un insieme di esempi (eventualmente con labels) è chiamato **dataset**. I dataset sono generalmente omogenei, nel senso che i dati contenuti al loro interno hanno un formato simile. Ad esempio:

- Nel Fisher's Iris dataset, tutti gli esempi hanno 4 features e una label corrispondente a una delle tre classi.
- In un dataset di immagini di food, ogni immagine è associata a una classe che indica il piatto specifico.

1.9.3 Design Matrix

Un modo comune per rappresentare un dataset è utilizzare una design matrix. Poiché ogni esempio è una collezione di d features, un dataset di m elementi può essere rappresentato tramite una matrice:

$$X \in \mathbb{R}^{m \times d}, \quad m, d \in \mathbb{N},$$

di dimensione $m \times d$.

- Ogni riga della design matrix rappresenta un esempio.
- Ogni colonna rappresenta una delle features.

Nel caso del supervised learning, si considera spesso anche un vettore di etichette:

$$Y \in \{1, \dots, K\}^m,$$

dove K è il numero di classi (nel caso di classificazione). Nel caso di regressione, invece, $Y \in \mathbb{R}^m$.

1.9.4 Esempio

Supponiamo di avere un dataset composto da 1000 e-mail, alcune classificate come spam e altre come not spam. Assumiamo che ogni e-mail sia rappresentata da due features, come discusso nei precedenti esempi.

		j^{th} feature											
$X =$	i^{th} example {	2.2	4.7	3.8	9.2	4.7	3.2	-1.2	8.9	-0.11	4.12	2	
		-0.11	-1.2	-0.11	-1.2	-0.11	4.7	-1.2	4.7	-0.11	9.2	3	
		9.2	-0.11	9.2	4.12	9.2	9.2	-1.2	6.9	-1.2	9.2	8	
		9.2	9.2	-0.11	-1.2	4.12	-8.6	9.2	-0.11	4.7	9.2	2	
		-0.15	9.2	-1.2	-0.11	4.7	-0.11	-1.2	8.7	9.2	-87.5	1	
		-0.11	-1.2	4.7	4.7	9.2	4.7	4.12	9.2	9.2	-1.2	0	
		9.2	-0.11	9.2	9.2	-1.2	4.7	4.7	-0.11	-1.2	-0.11	8	
		9.2	-0.11	4.12	-1.2	32.5	-1.2	9.2	9.8	4.12	9.2	1	
												$Y =$	

Figura 1.4: Design Matrix: rappresentazione di un dataset con m esempi e d features. Ogni riga corrisponde a un esempio, ogni colonna a una feature.

La design matrix che rappresenta il dataset è una matrice:

$$X \in \mathbb{R}^{1000 \times 2}.$$

- Ogni elemento della matrice rappresenta una delle features di un esempio nel dataset.
- Ad esempio, $X_{i,1}$ indica il numero di **errori** ortografici nell' **i -esima** e-mail, mentre $X_{i,2}$ rappresenta il numero di **occorrenze** di parole chiave nella stessa e-mail.

Le labels sono contenute in un vettore:

$$Y \in \{0, 1\}^{1000},$$

dove Y_i rappresenta la label associata all' **i -esimo** esempio (ad esempio, 0 = not spam, 1 = spam).

1.9.5 Learning

Un algoritmo di Machine Learning **utilizza un dataset di esempi per migliorare la sua performance** in un determinato **task**. Il processo di miglioramento della performance dell'algoritmo è chiamato **learning** o **training**.

In cosa consiste il training?

- Un algoritmo di Machine Learning ha alcuni parametri (parameters), che possono essere regolati per modificarne il comportamento. Questi parametri sono legati a un model (una funzione matematica) utilizzata per risolvere il task.

- Un algoritmo chiamato *training procedure* utilizza gli esempi forniti per trovare i valori ottimali per questi *parameters*.
- Alcuni parametri non possono essere regolati automaticamente dal training. Questi sono detti *hyperparameters* e devono essere ottimizzati al di fuori della *training procedure*, spesso attraverso un metodo *trial and error*.

Esempio. Consideriamo un semplice spam detector che classifica le e-mail come spam o non-spam in base al numero di errori ortografici.

L'algoritmo può essere scritto come segue:

```

1 def classify(x):
2     if x > a:
3         return 1 # Spam
4     else:
5         return 0 # Non-spam

```

Listing 1.1: Soglia sul numero di errori ortografici, chiaramente un approccio naive

L'algoritmo dipende da un singolo parametro a . La domanda è: quale valore dovremmo assegnare ad a ? La *training procedure* permette di trovare un valore ottimo presumibilmente adatto per a . Una semplice *training procedure* consisterebbe nel provare diversi valori per a e registrare le performance dell'algoritmo per ciascun valore di a . Alla fine, possiamo scegliere il valore di a che massimizza la performance measure P (o, in alternativa, minimizza una loss ℓ).

1.9.6 Rischio atteso e rischio empirico

Per formalizzare l'idea di "buone prestazioni" si introduce una loss ℓ e si definisce il **rischio atteso** (o *true risk*):

$$R(\theta) = \mathbb{E}_{(x,y) \sim \mathcal{D}} [\ell(h_\theta(x), y)].$$

Poiché la distribuzione $\mathcal{D}$ è sconosciuta, in pratica si minimizza il **rischio empirico** sul dataset osservato D :

$$\hat{R}(\theta) = \frac{1}{m} \sum_{i=1}^m \ell(h_\theta(x^{(i)}), y^{(i)}).$$

La differenza $R(\theta) - \hat{R}(\theta)$ è detta **generalization gap**: minimizzare $\hat{R}$ non garantisce necessariamente di minimizzare R , specialmente con modelli molto complessi (overfitting). Per questo si valutano le prestazioni su dati non visti (testing) e si usano strategie come la cross-validation per stimare la capacità di generalizzazione.

1.9.7 Il paradigma Training/Testing

Ciascun algoritmo di Machine Learning presenta al suo interno dei parametri. Una scelta opportuna dei parametri dovrebbe garantire il funzionamento del modello. La procedura generale per la scelta e la verifica dei parametri dell'algoritmo è suddivisa in due parti:

1. Training dell'algoritmo.

Effettuato usando i dati del **training set**, consiste nella scelta dei parametri del modello,

osservando i valori della *loss function* (o funzione di costo), che offre un indice dell'errore da parte del modello e permette di capire se i parametri vanno modificati, e di quanto.

2. Testing dell'algoritmo - Inferenza.

Si passa poi ai dati di testing, usati per verificare se i parametri scelti sul dataset di training sono opportuni su dati mai visti prima. Il modello viene quindi valutato secondo le **misure di valutazione**.

Suddivisione dei dati

Distinguiamo due insiemi di dati: i **dati di training** e i **dati di testing**. È importante che i dati usati in fase di training e i dati usati in fase di inferenza siano **totalmente disgiunti**:

$$D_{\text{train}} \cap D_{\text{test}} = \emptyset.$$

Tuttavia, entrambi i set di dati appartengono in origine allo stesso dataset D . La suddivisione dei dati in training e testing sarà effettuata randomicamente, in modo da evitare bias di qualsiasi tipo. Dobbiamo inoltre assicurarci che il numero di dati, per ogni etichetta, sia bilanciato. Prendere l'80% di un dataset in fase di testing, con due classi (cane, non-cane), significa prendere l'80% di ciascuna delle due classi. L'obiettivo è **evitare favoritismi** durante la classificazione. Parleremo più avanti di tecniche che stabiliscono come effettuare la suddivisione tra training e testing sfruttando al meglio i propri dati, come la k -fold cross-validation.

1.9.8 Normalizzazione

I nostri dati $x \in \mathbb{R}^d$ possono avere valori appartenenti a range molto ampi, con scale diverse per ciascuna feature. Per creare spazi in cui le features hanno più o meno la stessa scala, si usano strategie di normalizzazione, come di seguito.

Normalizzazione in $[0, 1]$. Per ogni j -esima feature, dobbiamo stabilire il minimo e il massimo del dataset. Dato un dataset di dimensione m :

$$\begin{aligned} x_j^{\min} &= \min\{x_j^{(i)} \mid i = 1, 2, \dots, m\}, \\ x_j^{\max} &= \max\{x_j^{(i)} \mid i = 1, 2, \dots, m\}. \end{aligned}$$

Possiamo quindi normalizzare secondo la seguente formula:

$$\hat{x}_j = \frac{x_j - x_j^{\min}}{x_j^{\max} - x_j^{\min}},$$

e ogni dato sarà $\hat{x}_j \in [0, 1]$.

Standardizzazione.

$$\hat{x}_j = \frac{x_j - \mu_j}{\sigma_j}$$

con σ_j deviazione standard (la varianza è σ_j^2). Il risultato è un centramento rispetto alla media: l'origine coincide con la media, e i dati risultano su una scala comparabile.

Quantità statistiche per la j -esima feature

Per completezza, riportiamo le definizioni delle quantità usate sopra:

$$\text{Media: } \mu_j = \frac{1}{m} \sum_{i=1}^m x_j^{(i)},$$

$$\text{Varianza: } \text{Var}(x_j) = \frac{1}{m} \sum_{i=1}^m (x_j^{(i)} - \mu_j)^2,$$

$$\text{Deviazione standard: } \sigma_j = \sqrt{\frac{1}{m} \sum_{i=1}^m (x_j^{(i)} - \mu_j)^2} = \sqrt{\text{Var}(x_j)}.$$

Bisognerà conservare rispettivamente $x_j^{\min}$, $x_j^{\max}$ oppure μ_j, σ_j , per permettere la normalizzazione del testing set, in quanto la normalizzazione è fatta in fase di training.

1.10 Iperparametri

Sono parametri che stabiliscono aspetti del modello e che non sono calcolati in fase di training. In fase di training, questi vengono fissati e **non vengono stimati** dalla training procedure. Nonostante ciò, gli iperparametri hanno un effetto sulla loss function e quindi sulle prestazioni finali. Bisognerà variare gli iperparametri usando un **validation set**, un terzo set di dati ottenuto dal dataset D .

1.10.1 k-fold Cross Validation

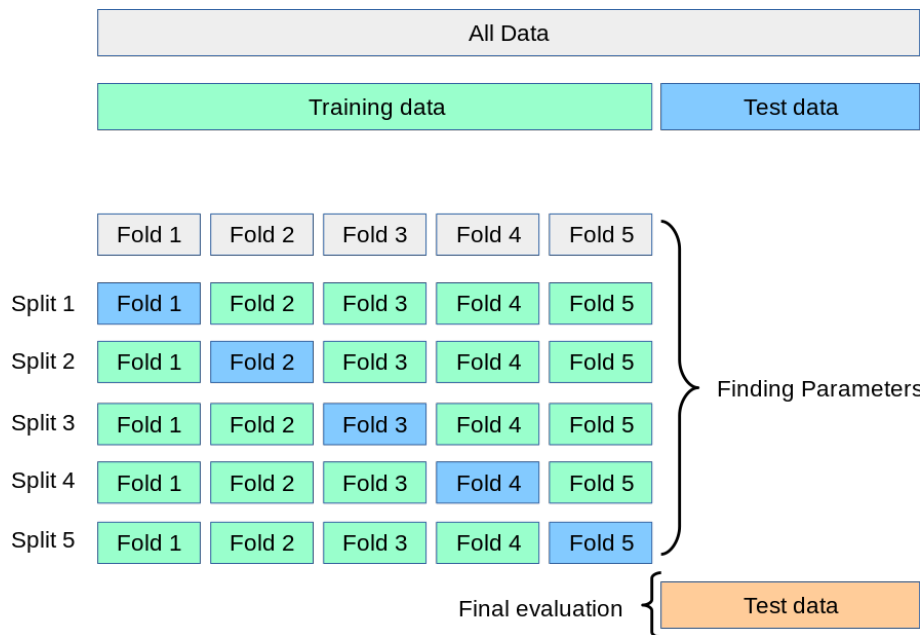
È una strategia che permette di ottenere buoni risultati anche con dataset più piccoli. Dividiamo in due parti il dataset D , ottenendo l'insieme di training e l'insieme di testing. Dividiamo nuovamente l'insieme di **training** in k parti. Supponiamo $k = 3$: utilizzeremo una porzione $\frac{1}{k}$ per la validazione e le $\frac{k-1}{k}$ rimanenti come training. In questo caso, $1/3$ e $2/3$. Effettueremo quindi tre cicli di training e validazione, variando ogni volta il subset usato per la validazione. Da ciascuno dei processi di training emergerà un valore della loss function: sceglieremo gli iperparametri associati a valori della loss più bassi (o, in alternativa, a metriche di performance più alte).

1.11 Overfitting e underfitting

Sono tra i problemi più grandi che possono emergere con i propri algoritmi di machine learning e riguardano spesso la **scelta del modello**.

Overfitting Si parla di overfitting quando il modello è così complesso da offrire performance ottime in fase di training, ma fortemente **peggiori in fase di testing**: il modello ha imparato troppo bene (anche dettagli e rumore) i dati su cui ha appreso, ma generalizza male sui dati inediti.

Underfitting È caratterizzato da **pesse performance in fase di training**: troppe poche variabili o modello troppo semplice. L'apprendimento non è tale da permettere al modello di catturare la struttura dei dati. Il modello non impara.

Figura 1.5: Procedura di cross-validation con $k = 5$.

Bisogna quindi trovare lo *sweet spot*, il compromesso tra modelli più complessi e modelli più semplici. Il numero di parametri è un fattore da tenere sempre in considerazione.

1.11.1 Bias e Variance

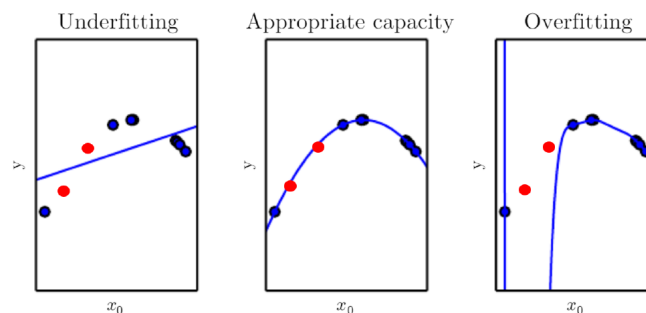


Figura 1.6: In blu i dati di training, in rosso quelli di testing. Nel caso dell'underfitting, vediamo una funzione troppo semplice (una funzione lineare) per approssimare una funzione più complessa. Nel caso dell'overfitting, il modello dà output assurdi.

Osservando i risultati dei nostri modelli, è possibile individuare facilmente due tipologie di errori:

Bias Error. Le previsioni non sono corrette a causa di assunzioni errate nel modello. Un alto bias può portare il modello a non rilevare importanti relazioni tra le features e gli output target, portando a underfitting (es. un modello che cerca di approssimare una funzione polinomiale con una lineare).

Variance Error. Le previsioni del modello sono fortemente influenzate da piccole fluttuazioni nel training set. Ad esempio, se rimuoviamo un punto dati e riaddestriamo l'algoritmo, otteniamo previsioni molto diverse. Questo può essere dovuto al fatto che l'algoritmo sta

cercando di modellare il rumore casuale presente nei dati di training, il che di solito porta a overfitting.

La **decomposizione bias-varianza** si riferisce tipicamente all'**errore atteso** (ad esempio per la MSE) e non va confusa con la differenza tra errori di training e testing.

Riferimenti

Il materiale visto in questo capitolo comprende:

- Materiale visto a lezione.
- Note scritte dal prof. Antonino Furnari.

Capitolo 2

I primi modelli di Machine Learning

2.1 Neurone computazionale - modello di McCulloch-Pitt

L'idea iniziale era quella di replicare la struttura di un neurone biologico con un modello matematico, in modo da poter simulare il funzionamento del cervello umano.

Neurone biologico. Un neurone biologico è costituito da:

- Dendriti: ricevono segnali da altri neuroni.
- Corpo cellulare: elabora i segnali ricevuti.
- Assone: trasmette il segnale elaborato ad altri neuroni.

Il primo modello di neurone computazionale fu proposto da McCulloch e Pitts nel 1943. Si basa su tre componenti principali:

- Degli input $x_1, x_2, \dots, x_d$ binari, che possono essere **eccitatori** e **inibitori**.
- Una threshold v , una soglia
- Un output binario.

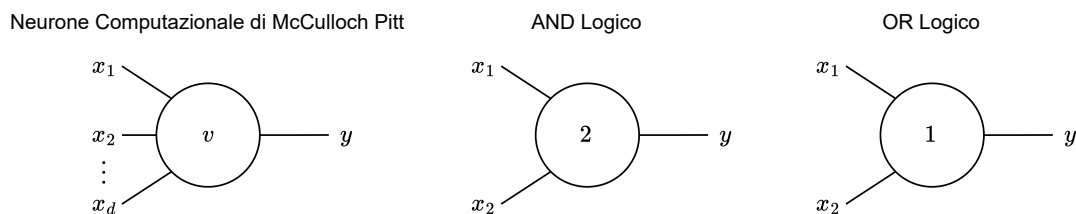


Figura 2.1: Modello di McCulloch Pitt e due computazioni possibili: AND logico e OR logico

Supponiamo che $x_1, \dots, x_j$ siano eccitatori, $x_{j+1}, \dots, x_d$ sono inibitori. Se $j \geq 1$ e almeno un inibitore è $= 1$, allora il neurone ritorna 0. Un inibitore è sufficiente per bloccare l'output. Altrimenti, si calcola $z = x_1 + \dots + x_j = x_1 + \dots + x_d$, ossia la somma di tutti gli input¹. Se la somma è $\geq v$, allora l'output sarà 1, altrimenti 0.

¹posso considerare anche gli input degli inibitori in quanto $= 0$.

Questo modello è tale da poter computare un *AND* ($v = n$ e n input eccitatori), un *OR* ($v = 1$ e n input eccitatori, vedere figura 2.1), ma non uno *XOR*!

2.1.1 Limitazioni del modello di McCulloch-Pitt

- Non esiste un modo automatico di fare training.
- Gli input sono binari.
- Gli input hanno tutti lo stesso peso.
- Tutte le funzioni computabili sono linearmente separabili.

2.2 Percettrone - Rosenblatt

Il successore del modello di McCulloch-Pitt è il **percettrone**. Questo modello risolve il problema dei pesi, assegnando ad ogni ingresso un peso differente, e introduce una procedura di apprendimento automatico per stabilire i pesi in maniera opportuna.

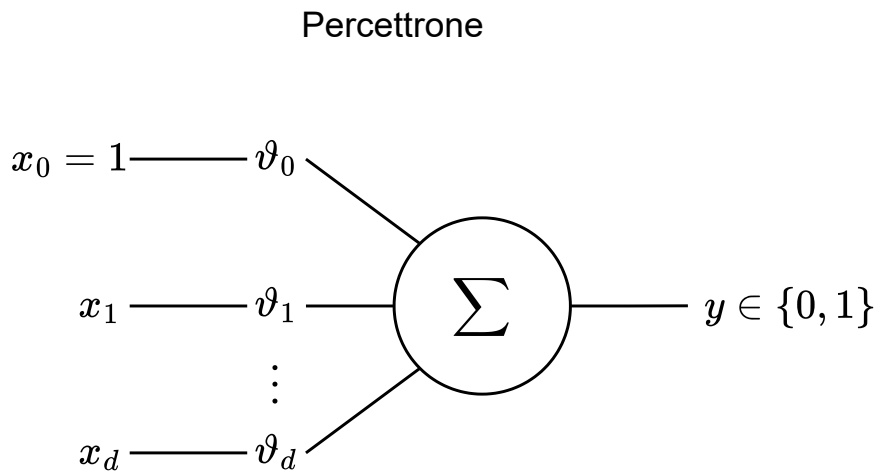


Figura 2.2: Modello di Rosenblatt, noto come Percettrone

Le componenti sono:

- Features $x_1, x_2, \dots, x_d$ normalizzate in $[0, 1]$. Ciascuno di questi input ha associato un peso $v_1, \dots, v_d$.
- Una threshold v_0 , una soglia.
- Un output binario.
- Una **procedura di learning automatico** per stabilire i parametri (il peso di ciascun input).

Il comportamento del percettrone è il seguente:

$$f(x_1, \dots, x_d) = \begin{cases} 1 & \text{se } \sum_{j=0}^d \vartheta_j x_j \geq \vartheta_0 \\ 0 & \text{altrimenti.} \end{cases}$$

Per semplificare la notazione, introduciamo una feature *dummy* $x_0 = 1$ e definiamo il vettore di input esteso

$$\tilde{x} = (x_0, x_1, \dots, x_d)^\top,$$

in modo da poter riscrivere la regola di decisione come una singola disuguaglianza. Per farlo, incorporiamo la soglia nella somma definendo un vettore di pesi esteso

$$\vartheta = (\vartheta_0, \vartheta_1, \dots, \vartheta_d)^\top \quad \text{con} \quad \vartheta_0 = -\vartheta_0^{(\text{soglia})}.$$

In questo modo otteniamo:

$$f(x) = \mathbf{1}\{\vartheta^\top \tilde{x} \geq 0\} = \text{sign}(\vartheta^\top x),$$

dove $\mathbf{1}\{\cdot\}$ vale 1 se la condizione è vera e 0 altrimenti.

Supponiamo ora di avere $d = 2$. Il decision boundary² è definita da

$$\vartheta_0 + \vartheta_1 x_1 + \vartheta_2 x_2 = 0,$$

ossia, risolvendo rispetto³ a x_2 (con $\vartheta_2 \neq 0$)

$$x_2 = -\frac{\vartheta_1}{\vartheta_2} x_1 - \frac{\vartheta_0}{\vartheta_2}.$$

Questa retta, in un task di classificazione, suddivide lo spazio in due classi. In uno spazio 2D è una retta, in uno spazio 3D un piano. Lo spazio dovrà essere **linearmente separabile**.

2.2.1 Processo generale di training del percettrone

Descriviamo la procedura di training nel seguente modo:

1. Inizializzazione casuale dei pesi $\vartheta_1, \dots, \vartheta_d \in \mathbb{R}$.
2. Computa $\forall x^{(i)}$ il valore di $\hat{y}^{(i)}$.
3. Confronta $\hat{y}^{(i)}$ con $y^{(i)}$
 - Se $\hat{y}^{(i)} = y^{(i)}$: non fare nulla.
 - Se $\hat{y}^{(i)} \neq y^{(i)}$: vanno aggiornati i pesi. Analizziamo come modificare i pesi, in funzione dei risultati.
4. Aggiornamento dei pesi:

²Il decision boundary è il luogo geometrico dei punti in cui la funzione di decisione cambia valore, quindi è indecisa tra i due valori 0 e 1.

³Si risolve rispetto a x_2 perché così si ha una forma simile a una retta $y = mx + q$.

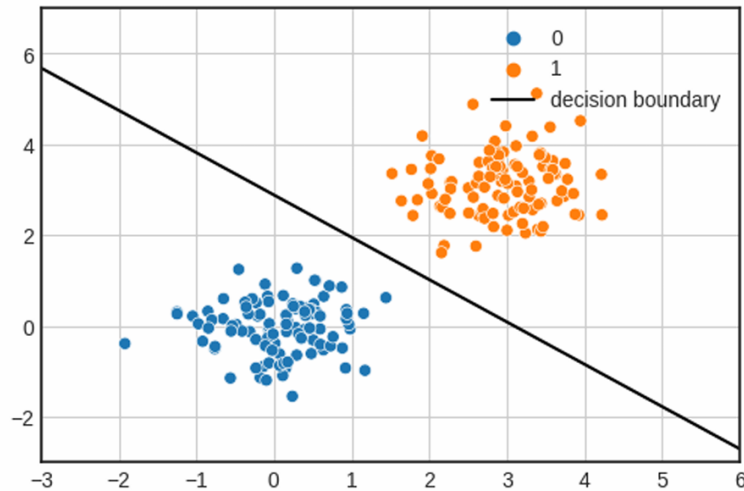


Figura 2.3: Decision boundary del percettrone in 2D. I punti arancioni sono classificati come 1, quelli blu come 0 e la retta al centro è la *decision boundary*.

$\hat{y}^{(i)}$	$y^{(i)}$	Cosa fare
0	0	ok!
0	1	riduci i pesi.
1	0	aumenta i pesi.
1	1	ok!

Questo sposterà la retta in maniera opportuna, convergendo ad un risultato opportuno per il dataset di training.

Questa descrizione generale riflette a grandi linee il processo, tuttavia è opportuno conoscere la procedura dal punto di vista matematico, che riflette poi l'implementazione effettiva.

2.2.2 Implementazione del training

L'aggiornamento dei pesi avviene nel seguente modo:

$$\underbrace{\hat{\vartheta}_j}_{\text{nuovo peso}} \leftarrow \vartheta_j + (y^{(i)} - \hat{y}^{(i)})x_j$$

Chiaramente, il percettrone non dovrà modificare i suoi pesi se $\hat{y}^{(i)} = y^{(i)}$. Questa formula lo contempla, in quanto $y^{(i)} - \hat{y}^{(i)} = 0$: il peso non verrà aggiornato. Si può aggiungere un parametro α , detto **learning rate**. Il percettrone standard non lo prevede.

$$\hat{\vartheta}_j \leftarrow \vartheta_j + \alpha(y^{(i)} - \hat{y}^{(i)})x_j$$

Scelto in maniera opportuna, potrebbe migliorare l'apprendimento e la convergenza.

2.2.3 Versione di Adaline

Una versione successiva dell'algoritmo di ricalcolo dei pesi, per stabilire con maggiore precisione di quanto incrementare o decrementare i pesi, stabilisce che

$$\hat{\vartheta}_j \leftarrow \vartheta_j + \left(y^{(i)} - \sum_i \vartheta_i \cdot x_i \right) x_j$$

Osserviamo che, per ogni esempio i -esimo, l'uscita desiderata $y^{(i)}$ può assumere solo due valori, 0 oppure 1. Allo stesso tempo, la somma pesata

$$\sum_i \vartheta_i \cdot x_i$$

rappresenta una combinazione lineare normalizzata degli ingressi, e pertanto assume valori compresi nell'intervallo $[0, 1]$. Ne consegue che la differenza tra il valore target $y^{(i)}$ e la somma pesata non potrà che appartenere all'intervallo $[-1, 1]$. In altre parole, se il neurone commette l'errore massimo possibile, questo sarà pari a 1 in valore assoluto, cioè la distanza più grande tra un'uscita binaria (0 o 1) e un valore previsto all'interno di $[0, 1]$.

2.3 Modelli lineari

Il perceptrone funziona correttamente solo se i dati sono *linearmente separabili*, ovvero se esiste una frontiera lineare che separa le due classi. Ad esempio, il problema AND è linearmente separabile.

Nota. può essere trovato un approfondimento sulla separabilità lineare nell'appendice B, sezione B.1.

2.3.1 Problema dello XOR

Supponiamo però di voler computare lo XOR logico. Per farlo, costruiamo la tabella:

x_1	x_2	$XOR(x_1, x_2)$
0	0	0
0	1	1
1	0	1
1	1	0

Il problema XOR non è linearmente separabile (immagine 2.4): i quattro punti $(0, 0)$, $(1, 0)$, $(0, 1)$, $(1, 1)$ con etichetta di uscita 0, 1, 1, 0 non possono essere separati da una singola frontiera lineare. Un perceptrone (che produce solo confini di decisione lineari) non riesce quindi a risolverlo.

Dimostrazione della non-linearità dello XOR. Assumendo la tabella di verità dello XOR e un perceptrone con pesi ϑ_1, ϑ_2 e soglia ϑ_0 , le condizioni di attivazione diventano

$$\begin{aligned} 1 \cdot \vartheta_1 + 0 \cdot \vartheta_2 &> \vartheta_0, \\ 1 \cdot \vartheta_1 + 1 \cdot \vartheta_2 &< \vartheta_0, \\ 0 \cdot \vartheta_1 + 1 \cdot \vartheta_2 &> \vartheta_0, \\ 0 \cdot \vartheta_1 + 0 \cdot \vartheta_2 &< \vartheta_0. \end{aligned}$$

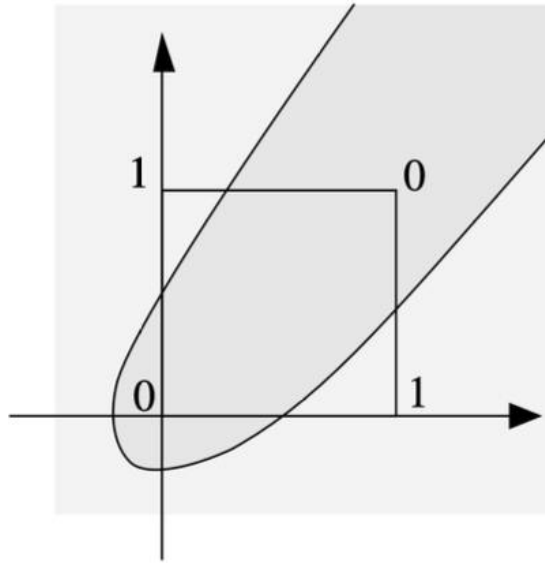


Figura 2.4: Schema del problema XOR: i vertici $(0, 1)$ e $(1, 0)$ appartengono alla classe 1, $(0, 0)$ e $(1, 1)$ alla classe 0. Nessuna frontiera lineare può separare le classi; è necessaria una frontiera non lineare (ottenibile con uno strato nascosto).

Da cui si ottiene

$$2\vartheta_0 < \vartheta_1 + \vartheta_2 < \vartheta_0,$$

ovvero una contraddizione: non esiste configurazione dei parametri di un singolo percettrone che risolva XOR.

2.3.2 Rete di percettroni

La soluzione consiste nell'usare una *rete di percettroni* (uno strato nascosto) che trasformi lo spazio in modo da rendere il problema linearmente separabile all'uscita.

Un esempio minimale è:

$$\begin{aligned} f_1(x_1, x_2) &= [x_1 - x_2 \geq 0.5], \\ f_2(x_1, x_2) &= [x_2 - x_1 \geq 0.5], \\ f_3(f_1, f_2) &= [1 \cdot f_1 + 1 \cdot f_2 \geq 0.5], \end{aligned}$$

dove $[\cdot]$ indica una funzione soglia (vale 1 se la condizione è vera, 0 altrimenti). Le funzioni f_1 e f_2 realizzano una trasformazione non lineare degli input; nello spazio così trasformato l'uscita f_3 è ottenuta tramite una semplice soglia lineare.

Infine, sebbene reti di percettroni possano risolvere *XOR*, l'algoritmo di apprendimento del singolo percettrone non è direttamente applicabile a reti multistrato; storicamente ciò contribuì al primo *AI winter* e la situazione migliorò con la formalizzazione della *retropropagazione* (*backpropagation*).

Caso studio. Consideriamo ora la rete mostrata in figura 2.5, con pesi esplicitamente indicati. I nodi a_1 e a_2 costituiscono lo strato nascosto, mentre a_3 rappresenta il neurone di uscita. Il nodo

costante 1 implementa il bias (termine di soglia) tramite pesi negativi.

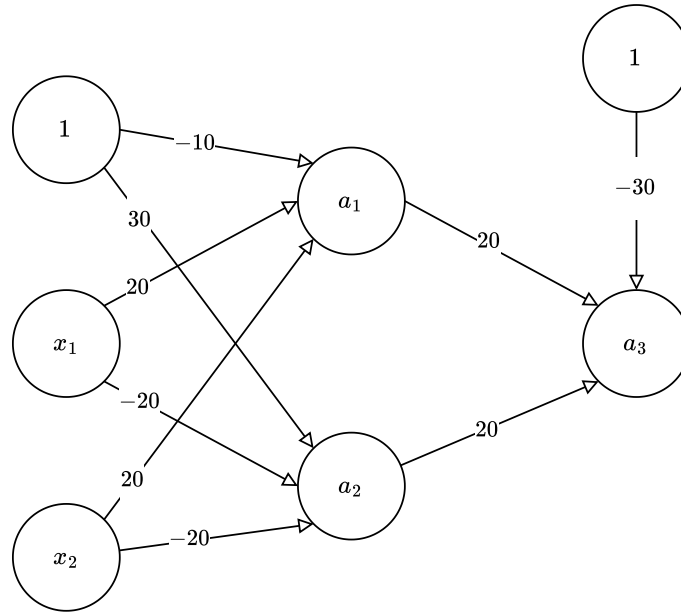


Figura 2.5: Rete di perceptron per computare lo *XOR*

I neuroni nascosti sono definiti da:

$$a_1 = \mathbf{1}\{-10 \cdot 1 + 20x_1 + 20x_2 \geq 0\},$$

$$a_2 = \mathbf{1}\{30 \cdot 1 - 20x_1 - 20x_2 \geq 0\}.$$

Il neurone di uscita è:

$$a_3 = \mathbf{1}\{-30 \cdot 1 + 20a_1 + 20a_2 \geq 0\}.$$

Verifichiamo ora i quattro casi possibili di (x_1, x_2) :

- $(0, 0)$: $a_1 = 0, a_2 = 1 \Rightarrow a_3 = 0$.
- $(0, 1)$: $a_1 = 1, a_2 = 0 \Rightarrow a_3 = 1$.
- $(1, 0)$: $a_1 = 1, a_2 = 0 \Rightarrow a_3 = 1$.
- $(1, 1)$: $a_1 = 0, a_2 = 0 \Rightarrow a_3 = 0$.

Si ottiene quindi esattamente la tabella di verità dello XOR.

Questo esempio mostra in modo concreto come uno strato nascosto realizzi una trasformazione non lineare dello spazio di input: i neuroni a_1 e a_2 costruiscono due regioni linearmente separabili, che il neurone finale combina linearmente per ottenere una frontiera di decisione non lineare nello spazio originale.

Riferimenti

Il materiale visto per questo capitolo comprende:

- Materiale visto a lezione.
- Note del prof. Antonino Furnari.
- Materiale sul percettrone dal libro *Neural Networks: A Systematic Introduction* [9].

Capitolo 3

Regressione

Definiamo il compito di regressione in termini di come un algoritmo elabora un esempio di input e produce un valore (o vettore) reale di output. Formalmente, vogliamo apprendere una funzione

$$h_{\vartheta} : \mathbb{R}^{d_1} \rightarrow \mathbb{R}^{d_2}, \quad d_1 \geq 1, d_2 \geq 1,$$

a partire da un dataset supervisionato

$$D = \{(x^{(i)}, y^{(i)})\}_{i=1}^m, \quad x^{(i)} \in \mathbb{R}^{d_1}, y^{(i)} \in \mathbb{R}^{d_2}.$$

In altre parole, individuare una funzione nello spazio dei dati, passante per tutti i valori del dataset in input, e presumibilmente capace di predire il valore associato ai dati di testing. Permette quindi di **stabilire relazioni tra i dati, individuare trend** e fare predizioni.

3.1 Ingredienti del task

- **Task:** predire valori reali a partire da input reali.
- **Modello:** ipotesi parametrica h_{ϑ} che mappa input in output.
- **Dati:** coppie etichettate $(x^{(i)}, y^{(i)})$.
- **Algoritmo di learning:** metodo per stimare ϑ (ottimizzazione).
- **Funzione di loss:** misura lo scarto tra predizioni e target.
- **Valutazione:** metriche su validation/test per giudicare il modello.

3.1.1 Tipi di regressione

1. **Regressione lineare**, $h_{\vartheta} : \mathbb{R} \rightarrow \mathbb{R}$, da spazio a una dimensione a una dimensione.
2. **Multiple regression**, $h_{\vartheta} : \mathbb{R}^{d_1} \rightarrow \mathbb{R}$, $d_1 > 1$, con d_1 features di input e un output.
3. **Multivariate regression**, $h_{\vartheta} : \mathbb{R}^{d_1} \rightarrow \mathbb{R}^{d_2}$, $d_1, d_2 > 1$, con d_1 features di input e d_2 output. Un modello di multivariate regression, viene allenato come un modello di multiple regression su ciascuno degli output.

3.2 Regressione lineare semplice

Nel caso $d_1 = d_2 = 1$ modelliamo la relazione tra un singolo ingresso $x \in \mathbb{R}$ e un'uscita reale $y \in \mathbb{R}$ con una retta:

$$h_{\vartheta}(x) = \vartheta_0 + \vartheta_1 x,$$

dove ϑ_0 è l'intercetta e ϑ_1 la pendenza. “Imparare” significa scegliere $\vartheta = (\vartheta_0, \vartheta_1)$ in modo che le predizioni siano vicine ai corrispondenti target.

3.2.1 Funzione di loss

Misuriamo la qualità della retta con l'errore medio quadratico (MSE):

$$J(\vartheta) = \frac{1}{2m} \sum_{i=1}^m (h_{\vartheta}(x^{(i)}) - y^{(i)})^2.$$

Il fattore $1/2$ è una costante di comodo che semplifica le derivate; il quadrato rende positivi tutti i contributi ed enfatizza gli errori grandi. In regressione lineare J è convessa rispetto ai parametri, quindi il minimo è unico (se c'è variabilità negli $x^{(i)}$). Otteniamo un grafico a tazza.

3.3 Regressione lineare multipla

Estendiamo al caso con più variabili in ingresso. Dato il vettore $x = (x_1, \dots, x_n)$, usiamo un parametro per ogni dimensione più il bias:

$$f(x) = \vartheta_0 + \vartheta_1 x_1 + \vartheta_2 x_2 + \dots + \vartheta_n x_n.$$

Per semplicità poniamo $x_0 = 1$ e definiamo il vettore esteso $x = (x_0, x_1, \dots, x_n)^\top$; allora

$$f(x) = \sum_{i=0}^n \vartheta_i x_i = \boldsymbol{\vartheta}^\top x.$$

Questo modello è, in sostanza, un *perceptrone senza soglia*: la combinazione lineare $\boldsymbol{\vartheta}^\top x$ è l'uscita continua del regressore.

3.4 Feature scaling

Per far funzionare bene (e in fretta) la discesa del gradiente, le feature vanno portate su scale simili. Due opzioni comuni:

$$\text{z-scoring: } x_j \leftarrow \frac{x_j - \mu_j}{\sigma_j}, \quad \text{min-max: } x_j \leftarrow \frac{x_j - x_j^{\min}}{x_j^{\max} - x_j^{\min}}.$$

Le statistiche $(\mu_j, \sigma_j, x_j^{\min}, x_j^{\max})$ si calcolano solo sul training e si riusano (senza ricalcolarle) su validation/test, per evitare data leakage.

Osservazione sulla rappresentazione delle features. Nel seguito indicheremo con X la design matrix costruita a partire dalle osservazioni $x^{(i)}$. Qualora venga applicata una trasformazione $\Phi : \mathbb{R}^n \rightarrow \mathbb{R}^d$, si intende che

$$X_{i\cdot} = \Phi(x^{(i)})^\top.$$

Pertanto tutte le formule successive (predizioni, loss, gradiente e metriche di valutazione) mantengono la forma

$$\hat{y} = X\vartheta,$$

intendendo che X possa già includere eventuali trasformazioni non lineari delle feature originali.

3.5 Algoritmo di discesa del gradiente

L'obiettivo è scegliere ϑ che minimizza $J(\vartheta)$ funzione di loss. La *discesa del gradiente* è un metodo iterativo che aggiorna i parametri nella direzione di massima diminuzione di J . Nel prossimo esempio a due parametri (ϑ_0, ϑ_1), supporremo l'intercetta $\vartheta_0 = 0$.

3.5.1 Idea dell'algoritmo

Partiremo da dei valori casuali dei parametri, in questo caso ϑ_1 , in quanto ϑ_0 è già fissata a 0 per semplicità. Osserveremo il coefficiente angolare della retta tangente alla funzione di loss, ossia la sua derivata. Quando questa è:

- negativa, incrementare il valore.
- positiva, decrementare il valore,

Il numero di iterazioni da ripetere dipende dalla convergenza che si vuole ottenere.

3.5.2 Algoritmo generale

Sia $X \in \mathbb{R}^{m \times (n+1)}$ la design matrix con prima colonna di 1, $y \in \mathbb{R}^m$ il vettore dei target e $\hat{y} = X\vartheta$ le predizioni. La loss è

$$J(\vartheta) = \frac{1}{2m} \|X\vartheta - y\|_2^2, \quad \nabla_{\vartheta} J(\vartheta) = \frac{1}{m} X^\top (X\vartheta - y).$$

Pseudocodice.

1. **Inizializza** in modo casuale: $\vartheta^{(0)} = (\vartheta_0, \dots, \vartheta_n)^\top$.
2. **Calcola le derivate parziali:**

$$\frac{\partial J(\vartheta)}{\partial \vartheta_j} = \frac{1}{m} \sum_{i=1}^m (\vartheta^\top \tilde{x}^{(i)} - y^{(i)}) \tilde{x}_j^{(i)}, \quad j = 0, \dots, n,$$

dove $\tilde{x}^{(i)} = (1, x_1^{(i)}, \dots, x_n^{(i)})^\top$.

3. **Aggiorna** i parametri (passo di ampiezza $\alpha > 0$):

$$\vartheta_j \leftarrow \vartheta_j - \alpha \frac{\partial J(\vartheta)}{\partial \vartheta_j}, \quad j = 0, \dots, n.$$

4. **Ripeti** i passi 2-3 finché non è soddisfatto un criterio d'arresto (iterazioni massime, $\|\nabla J(\vartheta)\|$ sotto soglia, variazione di J piccola).

In forma compatta: $\vartheta \leftarrow \vartheta - \frac{\alpha}{m} X^T (X\vartheta - y)$.

Il termine $X\vartheta - y$ è il vettore dei residui, ossia le differenze tra predizioni e target. Da qui moltiplicare per X^T "riporta" questi errori nello spazio dei parametri, ritornando una somma pesata degli errori per ogni parametro. Il passo di aggiornamento α controlla la velocità di apprendimento: se è troppo grande, si rischia di divergere; se è troppo piccolo, l'algoritmo potrebbe impiegare troppo tempo per convergere.

3.6 Regressione polinomiale e feature mapping

Parlando della rete di perceptron per computare lo *XOR*, abbiamo già anticipato l'utilizzo di trasformazioni per semplificare spazi di partenza non linearmente separabili, col fine ultimo di renderli compatibili a modelli con parametri di tipo lineare. Indichiamo la trasformazione con $\Phi(x) \rightarrow (x')$. Il nostro modello lavorerà su $h_{\vartheta}(\Phi(x))$. Prendiamo come esempio la seguente trasformazione, che ci fa passare da uno spazio a 2 dimensioni, ad uno a 5 dimensioni.

$$\phi \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \rightsquigarrow \begin{pmatrix} x_1 \\ x_2 \\ x_1^2 \\ x_2^2 \\ x_1 x_2 \end{pmatrix}$$

Ottenendo un polinomio non lineare rispetto alle features, ma lineare rispetto ai parametri (ovvero che è lineare nella combinazione lineare dei parametri ϑ).

$$\vartheta_0 + \vartheta_1 x_1 + \vartheta_2 x_2 + \vartheta_3 x_1^2 + \vartheta_4 x_2^2 + \vartheta_5 x_1 x_2$$

3.7 Regularizzazione

Per ridurre l'overfitting penalizziamo pesi grandi. In **Ridge** (L2) la loss è

$$J_{\lambda}(\vartheta) = \frac{1}{2m} \|X\vartheta - y\|_2^2 + \underbrace{\frac{\lambda}{2m} \sum_{j=1}^n \vartheta_j^2}_{\text{Termine di regolarizzazione}},$$

dove tipicamente ϑ_0 non si penalizza. Aggiornamenti:

$$\vartheta_0 \leftarrow \vartheta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}), \quad \vartheta_j \leftarrow \vartheta_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) \tilde{x}_j^{(i)} + \frac{\lambda}{m} \vartheta_j \right] \quad (j \geq 1).$$

In questa regolarizzazione il quadrato rende ogni contributo non negativo ed enfatizza errori grandi, mentre il fattore λ controlla l'intensità della penalizzazione. In generale, un λ più grande porta a modelli più semplici (con pesi più piccoli), ma se è troppo grande si rischia di sottostimare i dati (underfitting).

Questa regolarizzazione non annulla i pesi, ma li spinge verso zero. Esiste un altro tipo di regolarizzazione, chiamata **Lasso** (L1), che invece utilizza il valore assoluto dei pesi:

$$J_{\lambda}(\vartheta) = \frac{1}{2m} \|X\vartheta - y\|_2^2 + \underbrace{\frac{\lambda}{m} \sum_{j=1}^n |\vartheta_j|}_{\text{Termine di regolarizzazione}},$$

che ha l'effetto di annullare completamente alcuni pesi, portando a modelli più semplici e interpretabili, ma anche a una maggiore difficoltà di ottimizzazione (a causa della non differenziabilità del termine di regolarizzazione).

3.8 Valutazione

Metriche tipiche:

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2, \quad \text{RMSE} = \sqrt{\text{MSE}}, \quad \text{MAE} = \frac{1}{m} \sum_{i=1}^m |\hat{y}^{(i)} - y^{(i)}|.$$

R^2 (coefficiente di determinazione):

$$R^2 = 1 - \frac{\sum_i (y^{(i)} - \hat{y}^{(i)})^2}{\sum_i (y^{(i)} - \bar{y})^2}.$$

3.8.1 REC curve

Ordinando gli errori assoluti $e_i = |\hat{y}^{(i)} - y^{(i)}|$ e tracciando, al variare di ε , la frazione cumulativa di esempi con errore $\leq \varepsilon$, si ottiene una curva di facilissima lettura; un'area sotto la curva¹ maggiore indica in genere prestazioni migliori. Analogamente, un'area sopra la curva² maggiore indica maggiore errore atteso. Quanto più velocemente cresce la curva, tanto più il modello dovrebbe essere accurato.

Riferimenti

Il materiale per questo capitolo comprende:

- Materiale visto a lezione.
- Note del prof. Antonino Furnari.

¹AUC, area under the curve

²AOC, area over the curve

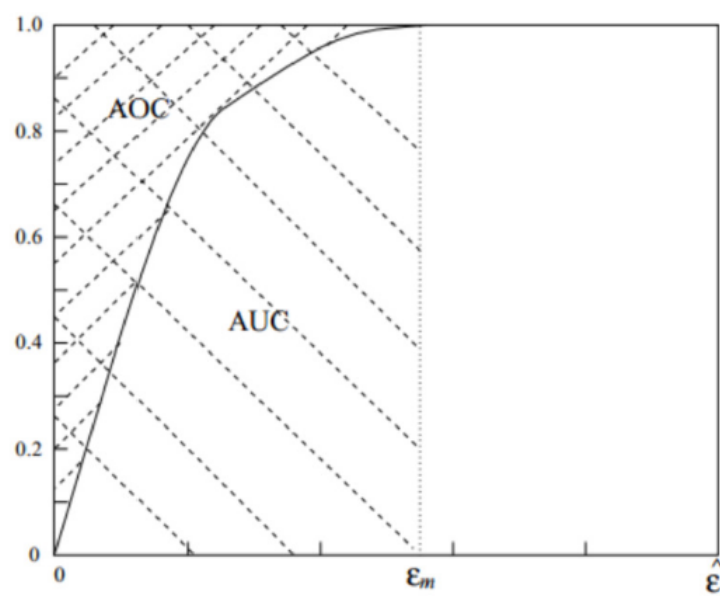


Figura 3.1: Esempio di curva REC.

Capitolo 4

Classificazione

4.1 Introduzione alla classificazione

La **classificazione** è un altro dei problemi tipici che può essere risolto tramite un algoritmo di Machine Learning. Riguarda lo **stabilire se un dato input appartiene ad una tra delle classi** prestabilite dal problema. Il **target non sarà più un valore** $\in \mathbb{R}$, ma una tra k classi. Per studiare la classificazione, andremo a stabilire i requisiti e i nostri obiettivi.

- Un modello ben definito.
- Una funzione di loss per stabilire lo scarto con i risultati attesi.
- Un algoritmo di learning per trovare i parametri.
- Delle misure di valutazione.
- Niente overfitting!

4.1.1 Classificazione: binaria e di classe

Identifichiamo due tipi di task di classificazione:

- **Classificazione binaria.**
Le etichette $y \in \{0, 1\}$. Va associata ad x una delle due etichette, o le probabilità relative a ciascuna delle due.
- **Classificazione di classe.**
Le etichette $y \in \{0, 1, \dots, k - 1\}$. Va associata ad x una delle etichette, o la probabilità relativa a ciascuna di esse. In realtà $\hat{y}$ (la predizione) è un vettore di k elementi, in cui ogni elemento rappresenta la probabilità che x appartenga alla classe associata a quell'elemento.

È fondamentale sottolineare l'importanza che gli output di questi algoritmi siano sempre $\in [0, 1]$, in quanto da interpretare come **probabilità**.

4.2 Classificazione binaria

Nonostante i classificatori binari possano sembrare limitati, questi trovano applicazioni in vari task: solitamente, questi si basano sulla suddivisione di immagini più grandi in *patches* più piccole,

con una successiva, appunto, classificazione, per individuare determinati elementi. Nel **medical imaging**, i classificatori binari possono individuare piccoli tumori. Nei **controlli qualità** in ambito industriale, possono individuare **imperfezioni** nei materiali. Nel campo più generale della **computer vision**, gli algoritmi di classificazione (binaria e non) sono fondamentali.

4.2.1 Perché non usare la regressione lineare per la classificazione?

Si potrebbe pensare di usare un modello di regressione lineare per effettuare classificazione binaria, e l'intuizione non sarebbe nemmeno totalmente sbagliata: potremmo usare, ad esempio, la regressione per trovare una retta che unisce i dati in maniera opportuna, e in funzione della pendenza di questa retta, stabilire un valore di sogliatura per distinguere due classi.

Forniamo il seguente esempio: vogliamo addestrare un modello per prevedere se un tumore è benigno o maligno usando una singola feature (dimensione del tumore).

Usiamo la regressione lineare e facciamo il fitting di una retta nello spazio (troviamo la retta, cioè la funzione che meglio si posiziona a media tra tutti i punti). Possiamo trovare il miglior $y = \vartheta^T x$ dal nostro set di dati e quindi selezionare una **soglia** su y per classificare quando il tumore è maligno o meno: $h_{\vartheta}(x) \leq 0.5 \rightarrow 0$, $h_{\vartheta}(x) \geq 0.5 \rightarrow 1$.

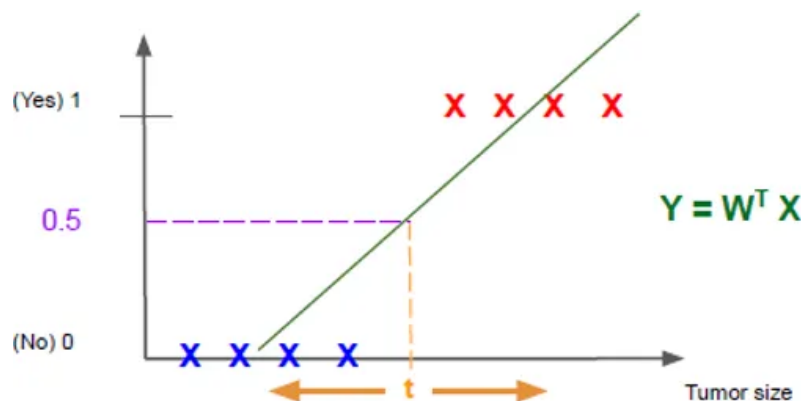


Figura 4.1: Utilizzo della regressione lineare per classificazione binaria: viene stimata una retta $y = \vartheta^T x$ sui dati e si introduce una soglia (ad esempio 0.5) per distinguere le due classi. I punti sotto la soglia vengono classificati come 0 (tumore benigno), mentre quelli sopra come 1 (tumore maligno).

Ci sono **due criticità** relative all'utilizzo della regressione. La prima, riguarda la versatilità del modello: il fitting della retta non è opportuno, in quanto la regressione lineare dipende fortemente dalla distribuzione dei dati.

La seconda criticità riguarda invece la y , non sempre limitata tra 0 e 1, come ci aspettiamo in un modello di classificazione binaria. Come ci comportiamo con valori più grandi di 1? E con valori minori di 0? Possiamo risolvere il problema introducendo delle funzioni, dette funzioni **di linking**. Queste ci permettono di **adattare i nostri valori** a modelli di regressione che altrimenti non sarebbero opportuni per la classificazione. Vediamo un esempio relativo alla regressione logistica usando la funzione **sigmoide**.



Figura 4.2: Sensibilità della regressione lineare alla distribuzione dei dati: l'aggiunta o lo spostamento di alcuni esempi modifica significativamente la retta stimata, causando errori di classificazione rispetto alla soglia fissata. Questo evidenzia una delle principali criticità dell'uso della regressione lineare per compiti di classificazione.

4.3 Regressione logistica - Sigmoide

Se la regressione lineare restituisce una y non limitata tra 0 e 1, la **regressione logistica** risolve il problema con una funzione, detta **sigmoide**, applicata sul risultato $z = \vartheta^T X$ del modello di regressione. La **sigmoide** è una funzione del tipo

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-\vartheta^T X}}$$

con $z = \vartheta^T X$. Questa funzione ha un'interpretazione probabilistica molto banale, opportuna per **adattare i modelli di regressione lineare su problemi di classificazione**¹. Osserviamo che:

- $1 - \sigma = \sigma(-z)$
- $\frac{\partial \sigma(z)}{\partial z} = \sigma(z)(1 - \sigma(z)) = \sigma(z)\sigma(-z)$
- È inoltre dimostrabile che $\sigma = \frac{1}{2} + \frac{1}{2} \tanh\left(\frac{z}{2}\right)$. Questo ci apre le porte a implementazioni molto più semplici.

Da qui il modello di regressione logistica è definito come:

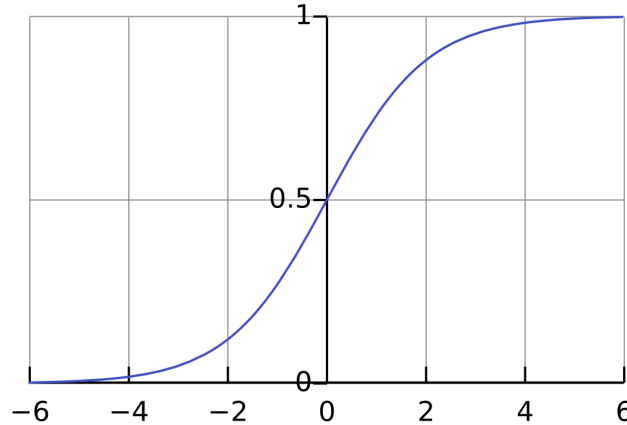
$$h_{\vartheta}(x) = \sigma(\vartheta^T x) = \frac{1}{1 + e^{-\vartheta^T x}}$$

Decision boundaries. Il modello di regressione logistica, come quello di regressione lineare, è un classificatore lineare. La decision boundary è definita come l'insieme dei punti per cui il classificatore è incerto, ovvero quando le probabilità restituite sono uguali per entrambe le classi (nel caso binario, quando $P(y = 1|x) = P(y = 0|x)$). Nella regressione logistica questo avviene quando

$$\sigma(\vartheta^T x) = 0.5 \quad \Rightarrow \quad \vartheta^T x = 0$$

¹Non si può pretendere di usare un modello di regressione lineare su task di classificazione, senza fare alcun tipo di modifica, come quelle che introduciamo con la sigmoide.

il che porta a un decision boundary lineare definito dall'iperpiano $\vartheta^T x = 0$. I punti su questa linea rappresentano la soglia di decisione del classificatore, dove è incerto se assegnare l'etichetta 0 o 1.



4.3.1 Vantaggi relativi all'uso della sigmoide

- Interpretazione probabilistica semplice. Da valori $\in [0, 1]$.
- Derivabile, più liscia rispetto ad una step function.
- Introduce non linearità, migliorando la classificazione.

4.4 Funzione di Loss

Avere una funzione di Loss associata al modello è fondamentale per definire il criterio di apprendimento e determinare i parametri ϑ ottimali per il problema di classificazione.

Nel caso della regressione logistica, la funzione di Loss è definita in funzione della **vera etichetta** $y \in \{0, 1\}$ e della predizione del modello $h_\vartheta(x)$.

Per un singolo esempio (x, y) , definiamo la loss come:

$$\text{Loss}(h_\vartheta(x), y) = \begin{cases} -\log(h_\vartheta(x)) & \text{se } y = 1 \\ -\log(1 - h_\vartheta(x)) & \text{se } y = 0 \end{cases}$$

Questa definizione penalizza fortemente le predizioni errate:

- se $y = 1$ e $h_\vartheta(x)$ è vicino a 0, la loss diverge;
- se $y = 0$ e $h_\vartheta(x)$ è vicino a 1, la loss diverge.

Possiamo riscrivere la funzione in forma compatta, sfruttando il fatto che y è binaria:

$$\text{Loss}(h_\vartheta(x), y) = -[y \log(h_\vartheta(x)) + (1 - y) \log(1 - h_\vartheta(x))]$$

Questa espressione è equivalente alla definizione precedente ma consente una trattazione analitica più semplice, in particolare nel calcolo del gradiente.

Definita la loss per un singolo esempio, possiamo introdurre la **funzione di costo** (cost function), che rappresenta la media della loss su tutti i m esempi del training set:

$$J(\vartheta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(h_{\vartheta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\vartheta}(x^{(i)})) \right]$$

La funzione di costo viene minimizzata tramite l'algoritmo di discesa del gradiente:

$$\vartheta_j^{\text{new}} = \vartheta_j^{\text{old}} - \alpha \frac{\partial J(\vartheta)}{\partial \vartheta_j} \quad \forall j$$

dove α è il learning rate, iperparametro che controlla l'ampiezza del passo di aggiornamento.

4.4.1 Derivata parziale della funzione di costo

Necessaria per applicare la discesa del gradiente.

$$\frac{\partial J(\vartheta)}{\partial \vartheta_j} = \frac{1}{m} \sum_{i=1}^m \underbrace{(h_{\vartheta}(x^{(i)}) - y^{(i)})}_{(*)} x_j^{(i)} \quad \forall j = 0, \dots, d$$

(*) Questo membro nasconde la funzione sigmoide $\sigma(\vartheta^T X^{(i)})$.

4.4.2 Entropia dell'informazione

La funzione di loss usata nella regressione logistica (Binary Cross Entropy) ha una chiara interpretazione informativa: misura quanto la distribuzione di probabilità predetta dal modello si discosta dalla distribuzione “vera” delle etichette. Per introdurre questa idea, partiamo dal concetto di **entropia** di una variabile discreta, che quantifica il grado di incertezza associato a una distribuzione di probabilità.

Sia X una variabile aleatoria discreta che può assumere K valori con probabilità $p_1, \dots, p_K$. L'entropia è definita come:

$$H(X) = - \sum_{k=1}^K p_k \log_2(p_k).$$

Intuitivamente, l'entropia è:

- **alta** quando la distribuzione è “uniforme” (massima incertezza);
- **bassa** quando la distribuzione è concentrata su pochi valori (bassa incertezza).

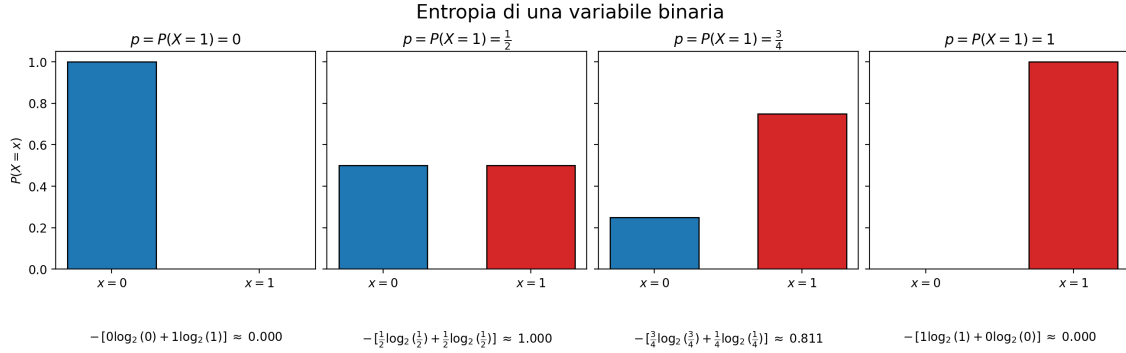
Nel caso binario ($K = 2$), con probabilità p_1 e $p_2 = 1 - p_1$, otteniamo:

$$H(X) = -p_1 \log_2(p_1) - p_2 \log_2(p_2).$$

NB Il termine $\log_2(0)$ non è definito, ma nel calcolo dell'entropia (e della cross-entropy) si usa il limite:

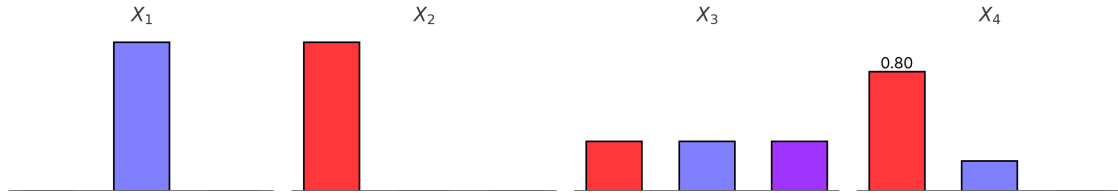
$$\lim_{p \rightarrow 0^+} p \log(p) = 0,$$

per cui assegnare probabilità esattamente nulle non crea problemi teorici. In pratica, per evitare instabilità numeriche, le implementazioni “clippano” le probabilità in $(\varepsilon, 1 - \varepsilon)$ con ε molto piccolo.



Inoltre, previsioni estremamente confidenti ($p \approx 0$ o $p \approx 1$) non implicano di per sé overfitting: l'overfitting riguarda la *generalizzazione* (prestazioni su dati non visti), non il valore di una singola probabilità stimata.

Esercizio sull'entropia. Riordina le seguenti distribuzioni in ordine crescente di entropia.



$$H(X) = -p_1 \log_2(p_1) - p_2 \log_2(p_2) = -[p_1 \log_2(p_1) + p_2 \log_2(p_2) + p_3 \log_2(p_3)]$$

1. $H(X_1) = -[0 \log(0) + 1 \log(1) + 0 \log(0)] = 0$
2. $H(X_2) = -[1 \log(1) + 0 \log(0) + 0 \log(0)] = 0$
3. $H(X_3) = -[0,33 \log(0,33) + 0,33 \log(0,33) + 0,33 \log(0,33)] \approx 1,5848$
4. $H(X_4) = -[0,80 \log(0,80) + 0,20 \log(0,20) + 0 \log(0)] \approx 0,722$

$$X_1 = X_2 < X_4 < X_3$$

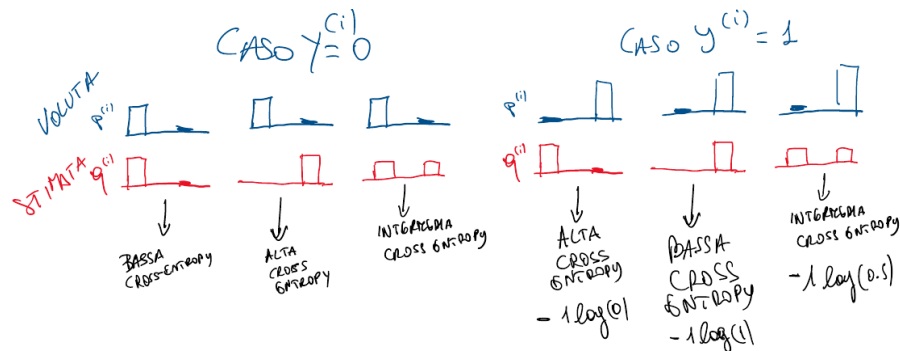
4.4.3 Binary Cross Entropy Loss

La seguente formula è equivalente alla nostra funzione di Loss, ma fornisce un'interpretazione più intuitiva. Definiamo la **Binary Cross Entropy Loss** come:

$$H_{BCE}(X) = \frac{1}{m} \sum_{i=1}^m \left(- \sum_{k=1}^K \underbrace{p_k^{(i)}}_{(a)} \log_2 \left(\underbrace{q_k^{(i)}}_{(b)} \right) \right)$$

Dove (a) è la distribuzione vera della classificazione, detta **ground truth**, ovvero quella con probabilità massima sulla classe attesa, mentre (b) è la distribuzione data dal classificatore. Misura

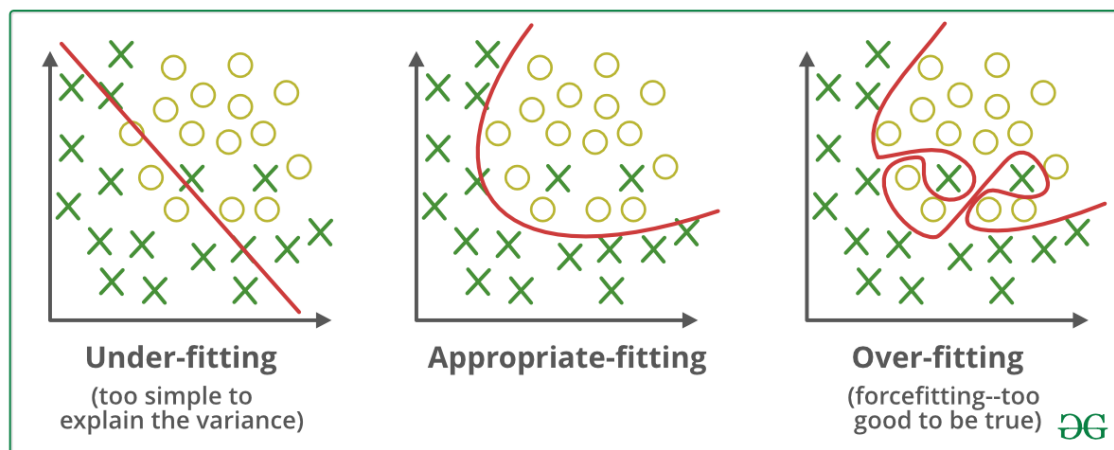
quindi la **distanza tra le due distribuzioni**. Minimizzare $H_{BCE}(X)$, significherà rendere le distribuzioni ideali e quelle effettive del classificatore, quanto più simili possibile.



4.5 Overfitting nella classificazione

Rischiamo di cadere in overfitting nell'utilizzo di classificatori polinomiali. Basterà usare i termini di regolarizzazione per diminuire o annullare l'impatto di alcuni termini di grado eccessivamente alto, ottenendo così:

$$J(\vartheta) = -\frac{1}{m} \sum_{i=1}^m (y^{(i)} \log(h_{\vartheta}(x)) + (1 - y^{(i)}) \log(1 - h_{\vartheta}(x))) + \frac{\lambda}{2m} \sum_{j=1}^n \vartheta_j^2$$



4.6 Reject Region, regione d'incertezza

Sia nella classificazione binaria, che in quella multiclasse, è possibile osservare casi in cui la probabilità stimata che un input appartenga ad una tra le classi specificate, non superi un determinato valore di certezza. Sono dei casi limite che vanno gestiti in maniera opportuna:

1. Prendere il valore con certezza più alta, se è proprio obbligatorio stabilire una classe, e il task non è particolarmente delicato.
2. Scartare l'input. Un'opzione migliore in campi più delicati, come quello medico.

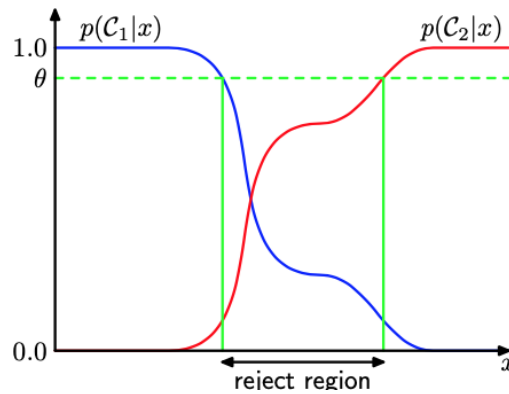


Figura 4.3: La zona d'incertezza (in cui $P(C_1|x) \sim P(C_2|x)$)

4.7 Classificazione multiclasse, approccio naive OVA e OVO

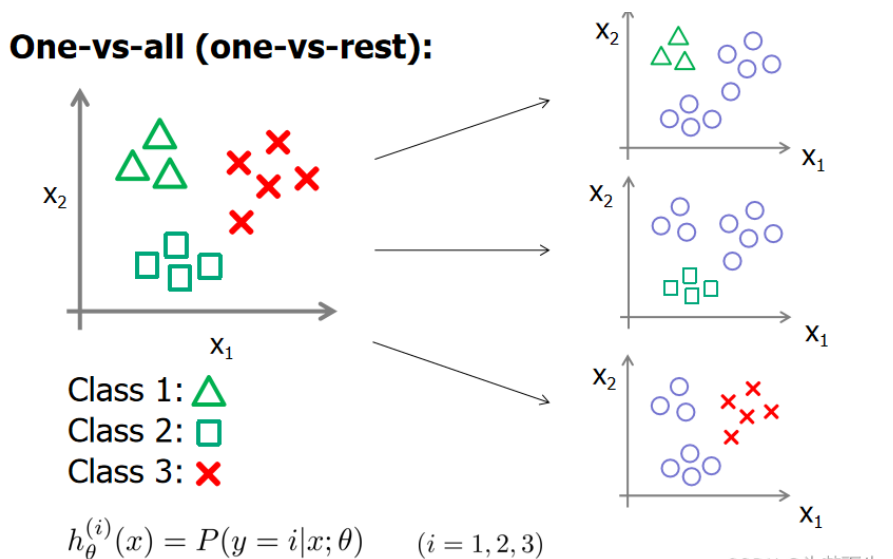
La classificazione multiclasse si distingue da quella binaria per il numero di classi. Il dataset di input non avrà più etichette binarie, ma k etichette. A ogni classe si associa un numero, ai fini di semplicità.

4.7.1 Metodo One vs All

Possiamo ottenere una classificazione multiclasse da dei classificatori binari, usando la metodologia **one vs all**. Immaginiamo di avere un sistema di classificazione figure geometriche, con le classi *triangolo*, *quadrato* e *cerchio* (c_1, c_2, c_3). Avremo bisogno di tre classificatori, $h_{\theta}^1, h_{\theta}^2, h_{\theta}^3$, capaci di misurare $P(\text{triangolo}|x)$, $P(\text{quadrato}|x)$, $P(\text{cerchio}|x)$. Prenderemo poi l'etichetta associata al valore di probabilità più alto

$$\hat{k} = \arg \max_k h_{\theta}^k(\bar{x})$$

ottenendo effettivamente una classificazione multiclasse.



Il training è effettuato sui singoli classificatori binari. Osserviamo inoltre che, con k classi:

Numero di classificatori richiesti in one vs all = k

4.7.2 Metodo One vs One

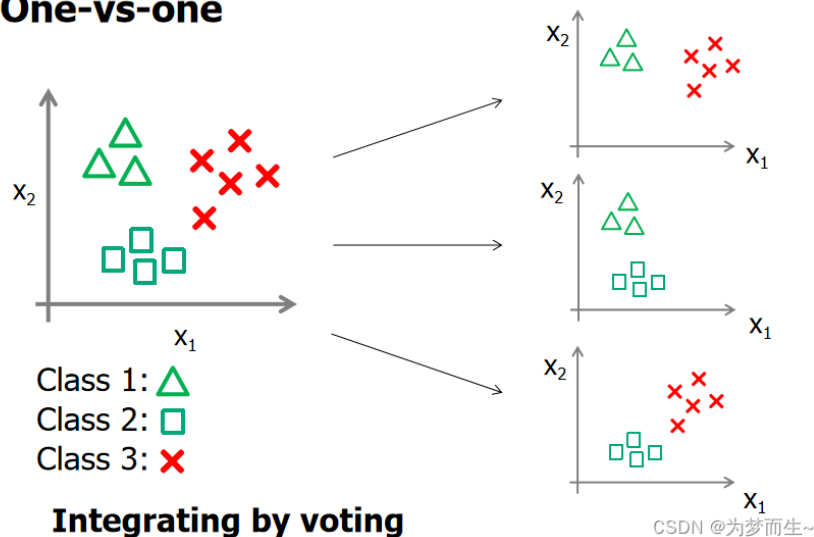
In questo caso, i classificatori distinguono classi a due a due. Si fanno poi le opportune valutazioni per capire quale classe assegnare. Il numero di classificatori risulta essere più alto. Con k classi abbiamo:

$$\text{Numero di classificatori richiesti in one vs one} = \frac{k(k-1)}{2}$$

Immaginiamo un classificatore per lo stesso problema precedentemente esposto: otteniamo le classi (c_1, c_2, c_3). Questi classificatori su coppie, daranno poi dei risultati. La classe più votata vince:

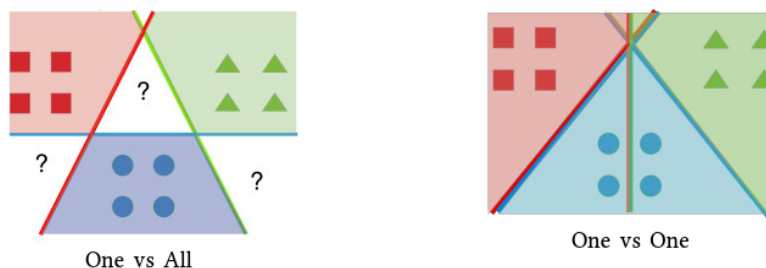
$$c_1 \text{ vs } c_2, \quad c_2 \text{ vs } c_3, \quad c_1 \text{ vs } c_3$$

One-vs-one



4.7.3 Confronto tra One vs All e One vs One

Il modello one vs one, per quanto più oneroso in termini di numero di classificatori binari richiesti, non presenta l'area di incertezza su tutte le classi, che invece possiamo trovare nel classificatore one vs all.



4.8 Stochastic Gradient Descent

L'algoritmo di discesa del gradiente, computa $J(\vartheta)$ su **tutti** i campioni del training set: associando al numero di campioni il valore m , a d il numero di feature e ad e le epoche di training, otteniamo un numero di operazioni complessive pari a

$$m \times d \times e$$

Questo prodotto raggiunge facilmente valori molto alti. Un modo per ridurre il numero di operazioni, senza intaccare il risultato del training, e arrivare a convergenza, è l'uso della **SGD**, ossia della **discesa del gradiente stocastica**. Ad ogni iterazione del training si campiona un mini-**batch** di campioni di dimensione $b \leq m$. Questo campione è scelto randomicamente a ogni iterazione.

$$b \times d \times e$$

L'algoritmo deve arrivare a convergenza: se non arriva a convergenza, si ripete il campionamento e si ripete il training.

4.9 Softmax - Classificazione Multiclasse

Abbiamo quindi visto com'è possibile implementare un classificatore multiclasse utilizzando più classificatori binari. Tuttavia, questa strategia (a prescindere da OVA e OVO) presenta delle criticità:

- Numero di classificatori elevato, che porta a tempi di training lunghi.
- Possibili aree di incertezza.
- Difficoltà nell'interpretare le probabilità restituite dai classificatori.

Si può ovviare a questi problemi con il modello **Softmax**, che estende la regressione logistica alla classificazione multiclasse.

4.9.1 Definizione

Sia K il numero di classi da prevedere. Il modello Softmax restituisce un vettore $P_{\vartheta}^{(k)}$ di lunghezza K , in cui ogni elemento $P_{\vartheta}^{(i)}$ rappresenta **la probabilità** che l'input x appartenga alla classe i . L'obiettivo è quindi, cercare di **stimare**:

$$P_{\vartheta}^{(k)}(y = k|x) \quad \forall k \in \{1, \dots, K\}$$

la somma di ogni elemento del vettore, è sempre = 1.

4.9.2 Funzione Softmax

Partiamo dall'idea che il softmax restituisce un vettore di probabilità:

$$h_{\vartheta}^{(c)}(x) = \begin{pmatrix} P_{\vartheta}^{(1)}(y = 1|x) \\ P_{\vartheta}^{(2)}(y = 2|x) \\ \vdots \\ P_{\vartheta}^{(K)}(y = K|x) \end{pmatrix} = \frac{1}{\underbrace{\sum_{k=0}^{K-1} e^{\vartheta^{(k)T}x}}_{\text{Normalizzatore della distribuzione}}} \cdot \begin{pmatrix} e^{\vartheta^{(1)T}x} \\ e^{\vartheta^{(2)T}x} \\ \vdots \\ e^{\vartheta^{(K)T}x} \end{pmatrix} = \frac{e^{\vartheta^{(c)T}x}}{\sum_{k=0}^{K-1} e^{\vartheta^{(k)T}x}}$$

Dove c è la classe di cui vogliamo calcolare la probabilità, k è l'indice che scorre tutte le classi, e $\vartheta^{(k)}$ è il vettore dei parametri associati alla classe k .

N.B. Ogni classe k ha il proprio vettore di parametri $\vartheta^{(k)}$ (i pesi del modello). Inoltre, una volta finito il learning del modello, la somma delle probabilità restituite dal softmax sarà sempre pari a 1 ($\sum_{k=1}^K P_{\vartheta}^{(k)}(y = k|x) = 1$).

4.9.3 Funzione di Costo

La funzione di costo del softmax è una funzione chiamata Cross Entropy Loss. Prima di poter definire la funzione di costo, definiamo una variabile ausiliaria $1\{\text{proposizione}\}$ che vale 1 se la proposizione è vera, 0 altrimenti. La funzione di costo del softmax è quindi:

$$J(\vartheta) = -\frac{1}{m} \left[\sum_{i=1}^m \sum_{c=1}^K 1\{y^{(i)} = c\} \log P_{\vartheta}^{(c)}(y = c|x^{(i)}) \right] + \underbrace{\frac{\lambda}{2} \sum_{c=1}^K \sum_{j=1}^n (\vartheta_j^{(c)})^2}_{\text{Termine di regolarizzazione}}$$

Dove il termine di regolarizzazione serve a prevenire l'overfitting del modello.

4.9.4 Obiettivo di Learning

L'obiettivo di learning è quello di minimizzare la funzione di costo, trovando i parametri ottimali ϑ :

$$\vartheta = \arg \min_{\vartheta} J(\vartheta)$$

Ovvero, trovare i parametri che minimizzano la distanza tra la distribuzione di probabilità predetta dal modello e la distribuzione di probabilità reale (ground truth).

Per fare questo possiamo usare la discesa del gradiente, per cui abbiamo bisogno della derivata parziale della funzione di costo rispetto ai parametri $\vartheta_j^{(c)}$:

$$\frac{\partial J(\vartheta)}{\partial \vartheta_j^{(c)}} = -\frac{1}{m} \sum_{i=1}^m \left[\left(1\{y^{(i)} = c\} - P_{\vartheta}^{(c)}(y = c|x^{(i)}) \right) x_j^{(i)} \right] + \underbrace{\lambda \vartheta_j^{(c)}}_{\text{Termine di regolarizzazione}}$$

E da questa, aggiorniamo i pesi $\vartheta_j^{(c)}$ utilizzando la regola di aggiornamento della discesa del

gradiente:

$$\vartheta_j^{(c)} \leftarrow \vartheta_j^{(c)} - \alpha \frac{\partial J(\vartheta)}{\partial \vartheta_j^{(c)}}, \quad \forall j \in [0, n], \quad \forall c \in [0, K - 1]$$

4.9.5 Proprietà del Softmax

Il modello softmax è **overparametrizzato**: per ogni ipotesi che potrebbe essere fatta sui dati, esistono infinite combinazioni di parametri che a partire dal vettore di feature $X \in \mathbb{R}^n$ producono la stessa predizione nel vettore di probabilità Y .

Per dimostrarlo, partiamo da:

$$\begin{aligned} h_{\vartheta}^{(c)}(x) &= \frac{e^{\vartheta^{(c)\top} x}}{\sum_{k=0}^{K-1} e^{\vartheta^{(k)\top} x}} && \text{Formula del Softmax} \\ &= \frac{e^{(\vartheta^{(c)} - \psi)^{\top} x}}{\sum_{k=0}^{K-1} e^{(\vartheta^{(k)} - \psi)^{\top} x}} && \text{Sottraiamo lo stesso } \psi \\ &= \frac{e^{\vartheta^{(c)\top} x} e^{-\psi^{\top} x}}{\sum_{k=0}^{K-1} e^{\vartheta^{(k)\top} x} e^{-\psi^{\top} x}} && \text{Separiamo gli esponenziali} \\ &= \frac{e^{\vartheta^{(c)\top} x}}{\sum_{k=0}^{K-1} e^{\vartheta^{(k)\top} x}} && \text{Si cancella il fattore comune } e^{-\psi^{\top} x} \end{aligned}$$

Dove $\psi \in \mathbb{R}^n$ è un vettore arbitrario. Quindi, sottraendo lo stesso vettore ψ da tutti i vettori di parametri $\vartheta^{(k)}$, otteniamo gli stessi valori di probabilità predetti dal modello, perciò esistono infinite combinazioni di parametri che producono la stessa predizione.

Nota. Può essere trovato un approfondimento di questa parte nell'appendice B alla sezione B.2.

4.9.6 Rete neurale Softmax

Il softmax può essere visto come un particolare tipo di rete neurale:

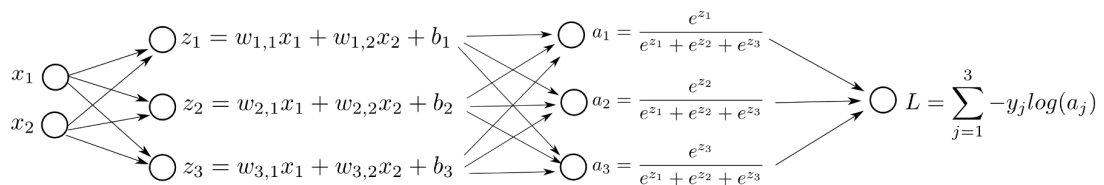


Figura 4.4: Flusso della regressione softmax con due ingressi e tre classi: dai logit $z_i = w_{i,1}x_1 + w_{i,2}x_2 + b_i$ alle probabilità $a_i = \frac{e^{z_i}}{\sum_{k=1}^3 e^{z_k}}$, fino alla loss a entropia incrociata $L = \sum_{j=1}^3 -y_j \log(a_j)$.

4.9.7 Funzione di Loss

La funzione di Loss usata nel softmax normalmente è una variante di **cross entropy loss**. Il softmax, infatti, restituisce un vettore di probabilità, e la cross entropy loss misura la distanza tra questa distribuzione di probabilità e la distribuzione di probabilità reale (ground truth).

Si possono usare alternative, in particolare la **Hinge Loss** trasforma la classificazione multi-classe in un problema di massimizzazione del margine tra le classi, come nelle SVM. La Hinge Loss è definita come:

$$\text{Loss}(h_{\vartheta}(x), y) = \sum_{i \neq y} \max(0, h_{\vartheta}(x)_i - h_{\vartheta}(x)_y + \Delta)$$

In questo contesto massimizzare il margine significa cercare di rendere la probabilità della classe corretta $h_{\vartheta}(x)_y$ significativamente maggiore rispetto alle probabilità delle classi errate $h_{\vartheta}(x)_i$, con un margine di sicurezza Δ .

Riferimenti

Il materiale di questo capitolo comprende:

- Materiale visto a lezione.

Capitolo 5

Design, valutazione e scelta dei parametri di un algoritmo di ML

5.1 Momento e gradiente

Una variante dell'algoritmo di discesa del gradiente, ovvero **la discesa del gradiente con momento**, modifica la nostra formula del gradiente nel seguente modo:

$$\begin{aligned}\vartheta_j^{\text{new}} &\leftarrow \vartheta_j^{\text{old}} - \alpha z^{\text{new}} \\ z^{\text{new}} &\leftarrow \beta z^{\text{old}} + \frac{\partial J(\vartheta^{\text{old}})}{\partial \vartheta^{\text{old}}}\end{aligned}$$

L'idea è quella di mantenere in z una **media dei gradienti recenti**, in modo da mantenere coerenza durante la convergenza: valori incoerenti del gradiente sono **rumore**, e questo **può causare zig-zag e rallentamenti nella convergenza**. Tenendo uno storico dei valori del gradiente (usando il **momentum** β come parametro che stabilisce l'influenza dei vecchi valori su quelli nuovi), diventa possibile **annullare parzialmente o totalmente l'effetto del rumore**. Accelera anche la convergenza stessa, **rinforzando i passi coerenti**. Si accorcia quindi il training, e si migliorano le performance a parità di epoche. Osservazioni statistiche consigliano l'utilizzo di $\beta = 0.90$ per ottenere risultati migliori.

5.2 Design di Algoritmi

Progettare bene un algoritmo di ML permette di ottenere migliori risultati in meno tempo e con meno risorse computazionali. Si pensi al cercare di inserire a casaccio i parametri di un modello, senza una logica precisa: si rischia di perdere tempo, se invece si pensa a come variare i parametri in maniera sistematica, per capire l'impatto di ciascuno di essi sul modello si potrebbe ottenere un modello migliore in meno tempo.

5.2.1 Strategie di miglioramento

Supponiamo di aver implementato la regressione lineare con regolarizzazione, ma gli errori su nuovi campioni sono comunque troppo alti. Potremmo pensare ad alcune strategie per migliorare

il modello:

Aumentare i campioni di Training. Non è sempre una scelta utile, anche perché se il modello non funziona su nuovi campioni significa che il problema (molto probabilmente) è di overfitting. Aumentare i campioni di training potrebbe aiutare, ma non è detto che lo faccia.

Ridurre il set di Features. Si potrebbe pensare di ridurre il numero di features in input al modello, in modo da semplificare il problema e ridurre il rischio di overfitting. Questa tecnica può essere efficace se alcune features sono ridondanti o non rilevanti per il task. Tuttavia, è importante fare attenzione a non eliminare informazioni utili.

Cercare nuove features. L'opposto della riduzione delle features: il problema è di underfitting, in quanto il modello non ha abbastanza informazioni per generalizzare bene. Aggiungere nuove features può aiutare a migliorare le prestazioni del modello ma richiede un'attenta selezione delle stesse per evitare di introdurre rumore.

Utilizzare features polinomiali. Trasformare le features esistenti in polinomiali può aiutare a catturare relazioni non lineari tra le variabili. Questa tecnica può essere utile se si sospetta che il modello lineare non sia sufficiente per rappresentare i dati. Tuttavia, l'aggiunta di termini polinomiali aumenta la complessità del modello e il rischio di overfitting, quindi è importante bilanciare questa scelta con tecniche di regolarizzazione.

Far variare il parametro di regolarizzazione λ . Modificare il parametro di regolarizzazione può influenzare significativamente le prestazioni del modello. Un valore più alto di λ penalizza maggiormente i pesi del modello, riducendo il rischio di overfitting, mentre un valore più basso consente al modello di adattarsi meglio ai dati di training, ma aumenta il rischio di overfitting. È importante trovare un equilibrio ottimale attraverso tecniche come la validazione incrociata (che vedremo più avanti).

Solitamente si cerca di effettuare test di diagnostica applicati al modello, per permettere di ottenere informazioni dettagliate su cosa funziona o non funziona con l'obiettivo di ottenere indicazioni per migliorare le prestazioni.

5.3 Valutazione

La valutazione di un algoritmo di ML è fondamentale per capire se il modello è adatto al task.

5.3.1 Valutazione incrociata

Un modo di fare valutazione incrociata¹ è la **grid search**, che sfrutta la k-Fold Cross Validation: si suddivide il dataset in t splits e in k folds, e si allena il modello su $t - 1$ splits e si valuta su uno split di validation, ripetendo il processo per tutti i t splits. In questo modo si ottengono t stime delle prestazioni del modello, che vengono poi mediate per ottenere una stima complessiva. Questo processo viene ripetuto per tutti i possibili set di iperparametri del modello, selezionando quelli che producono la *migliore prestazione* media sul validation set. Infine, si valuta il modello finale con gli iperparametri selezionati sul test set per ottenere una stima finale delle prestazioni del modello.

¹Per valutazione incrociata si intende la tecnica di suddividere il dataset in più parti per allenare e testare il modello in modo iterativo, garantendo una stima più robusta delle prestazioni.

Procedura. La procedura è la seguente:

1. Si suddivide il dataset originale in training/validation set e un test set che non verrà usato fino alla fine della valutazione.
2. Si suddivide il dataset rimanente in t splits e in k folds.
3. Si seleziona uno split come validation set e gli altri $t - 1$ come training set.
4. Si allena il modello sui $t - 1$ training splits.
5. Si valuta il modello con una funzione J_{VAL} sul validation fold (non usato per il training).
6. Si ripete il processo per tutti i t splits, ottenendo t valori di J_{VAL} .
7. Si calcola la media dei t valori di J_{VAL} per ottenere una stima complessiva delle prestazioni del modello.
8. Si ripete l'intero processo per tutti i possibili set di iperparametri del modello.
9. Si selezionano gli iperparametri che hanno prodotto la migliore prestazione media sul validation set.
10. Infine, si valuta il modello finale con gli iperparametri selezionati sul test set per ottenere una stima finale delle prestazioni del modello.

In questo modo il modello è robusto perché è stato allenato su diversi training set per i dati, allenato sui validation set per gli iperparametri e testato su un test set **mai visto prima**.

Esempio. Ipotizziamo di avere un certo dataset, dividiamolo in un test set (20% dei dati) e in un training/validation set (80% dei dati). Definiamo le funzioni di costo:

- **Funzione di costo di training:**

$$J_{\text{TR}}(\vartheta) = \frac{1}{2m_{\text{TR}}} \sum_{i=1}^{m_{\text{TR}}} \left(h_{\vartheta}(x_{\text{TR}}^{(i)}) - y_{\text{TR}}^{(i)} \right)^2$$

Dove m_{TR} è il numero di campioni nel training set, $x_{\text{TR}}^{(i)}$ e $y_{\text{TR}}^{(i)}$ sono rispettivamente l'input e l'output del i -esimo campione del training set.

- **Funzione di costo di validation:**

$$J_{\text{VAL}}(\vartheta) = \frac{1}{2m_{\text{VAL}}} \sum_{i=1}^{m_{\text{VAL}}} \left(h_{\vartheta}(x_{\text{VAL}}^{(i)}) - y_{\text{VAL}}^{(i)} \right)^2$$

Dove m_{VAL} è il numero di campioni nel validation set, $x_{\text{VAL}}^{(i)}$ e $y_{\text{VAL}}^{(i)}$ sono rispettivamente l'input e l'output del i -esimo campione del validation set.

- **Funzione di costo di test:**

$$J_{\text{TEST}}(\vartheta) = \frac{1}{2m_{\text{TEST}}} \sum_{i=1}^{m_{\text{TEST}}} \left(h_{\vartheta}(x_{\text{TEST}}^{(i)}) - y_{\text{TEST}}^{(i)} \right)^2$$

Dove m_{TEST} è il numero di campioni nel test set, $x_{\text{TEST}}^{(i)}$ e $y_{\text{TEST}}^{(i)}$ sono rispettivamente l'input e l'output del i -esimo campione del test set.

Supponiamo di voler utilizzare $t = 5$ splits e $k = 5$ folds. La suddivisione del training/validation set sarà la seguente:

	FOLD 1	FOLD 2	FOLD 3	FOLD 4	FOLD 5
SPLIT 1					
SPLIT 2					
SPLIT 3					
SPLIT 4					
SPLIT 5					

Figura 5.1: Dataset diviso in split e folds per la valutazione incrociata.

Dopo aver suddiviso i dati, ognuno dei dati produrrà una stima di $J_{\text{VAL}}^H(\vartheta^{(i)})$ per un certo i -esimo set valutata sull'iperparametro H . Da queste andiamo a fare la media:

$$\bar{J}_{\text{VAL}}^H(\vartheta) = \frac{1}{t} \sum_{i=1}^t J_{\text{VAL}}^H(\vartheta^{(i)})$$

Ottenuta la media $\bar{J}_{\text{VAL}}^H(\vartheta)$ per ogni set di iperparametri H , si seleziona quello che minimizza la funzione di costo media:

$$H^* = \arg \min_H \bar{J}_{\text{VAL}}^H(\vartheta)$$

Una volta fissato il miglior iperparametro H^* , si allena il modello sul test set per ottenere la stima finale:

$$J_{\text{TEST}}^{H^*}(\vartheta)$$

5.3.2 Bias e Varianza

Per valutare le prestazioni di un modello possiamo usare i valori del bias e della varianza per fare delle stime su come il modello si comporta sui dati.

Bias. Ricordiamo che il bias è l'**errore sistematico** che il modello commette sui dati. Un modello con alto bias tende a sottostimare la complessità del problema, portando a errori elevati sia sul training set che sul test set. Questo fenomeno è noto come **underfitting**. Nell'immagine 5.2, il grafico a sinistra mostra un esempio di underfitting, dove il modello lineare non riesce a catturare la relazione tra le variabili e commette un errore sistematico sia sul training set (punti blu) che sul test set (punti rossi).

Varianza. La varianza rappresenta la sensibilità del modello alle variazioni nei dati di training. Un modello con alta varianza tende a sovradattarsi ai dati di training, catturando il rumore invece della vera relazione tra le variabili. Questo porta a errori bassi sul training set ma elevati sul test set, fenomeno noto come **overfitting**.

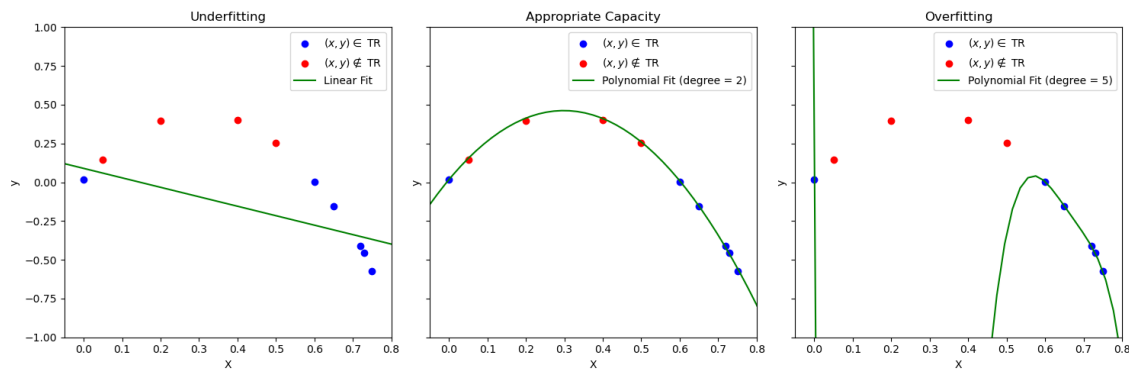


Figura 5.2: Underfitting—capacità adeguata—overfitting in regressione polinomiale: punti blu = dati di training (TR), punti rossi = fuori da TR; la linea verde mostra il fit del modello: lineare (sinistra), polinomio di grado 2 (centro) e polinomio di grado 5 (destra).

Nella figura 5.2, il grafico a destra mostra un esempio di overfitting, dove il modello polinomiale è troppo complesso e si adatta troppo strettamente ai dati di training, risultando in prestazioni scadenti sui dati di test². Come si vede dall'immagine al centro della figura 5.2, un modello con capacità adeguata riesce a bilanciare bias e varianza, ottenendo buone prestazioni sia sul training set che sul test set. Per valutare e risolvere i problemi sul modello, possiamo andare a controllare come Bias e Varianza variano al variare dei parametri del modello³ e notiamo che si può costruire una correlazione tra l'errore commesso dal modello (quindi o in J_{TEST} o in J_{VAL}) e la complessità del polinomio (il grado):

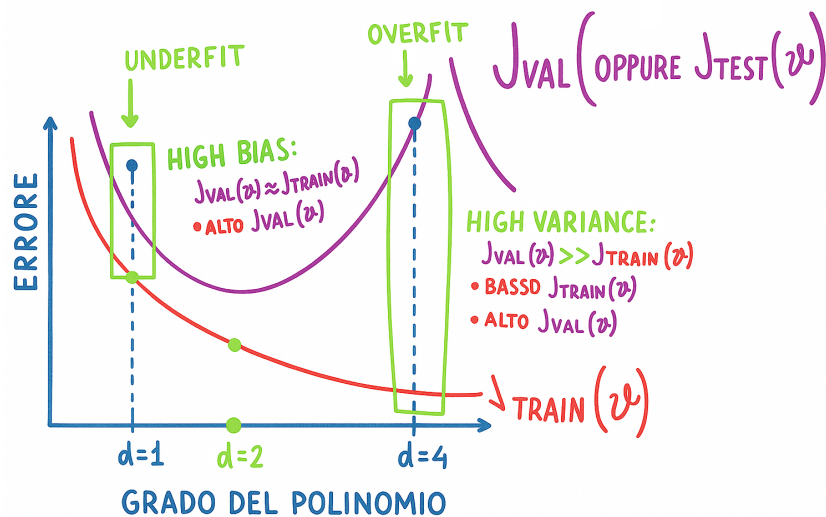


Figura 5.3: Trade-off tra bias e varianza in funzione della complessità del modello.

Uso del momentum. Per risolvere questi problemi, o comunque limitarli, si potrebbe utilizzare la discesa del gradiente con momento, che aiuta a stabilizzare la convergenza e a ridurre l'effetto

²Si noti che basterebbe rimuovere un punto del training set per far cambiare completamente il modello, in quanto altamente sensibile ai dati di input.

³In quella che molti chiamano **bias-variance trade-off**.

del rumore nei dati di training. Inoltre, è possibile utilizzare tecniche di regolarizzazione per penalizzare i modelli troppo complessi e prevenire l'overfitting.

5.3.3 Parametri di regolarizzazione

Anche i parametri di regolarizzazione, esattamente come il modello (i suoi parametri), devono essere stabiliti sperimentalmente. Il valore di λ influenza molto la regolarizzazione del modello:

- Un valore alto di λ penalizza maggiormente i pesi del modello, riducendo il rischio di overfitting, ma potrebbe portare a un modello troppo semplice (high bias).
- Un valore basso di λ consente al modello di adattarsi meglio ai dati di training, ma aumenta il rischio di overfitting (high variance).

Si parte da $\lambda^0 = 0$, testando valori di $\lambda^{(1)}, \lambda^{(2)} \dots \lambda^{(k)}$ sempre più grandi, verificando come cambia $J_{\text{VAL}}(\vartheta^i)$ in funzione di λ^i :

$$\begin{bmatrix} \lambda^{(0)} \\ \lambda^{(1)} \\ \lambda^{(2)} \\ \lambda^{(3)} \\ \vdots \\ \lambda^{(k)} \end{bmatrix} \Rightarrow \min_{\vartheta} J(\vartheta) \Rightarrow \begin{bmatrix} \vartheta^{(0)} \\ \vartheta^{(1)} \\ \vdots \\ \vartheta^{(k)} \end{bmatrix} \Rightarrow \begin{bmatrix} J_{\text{VAL}}(\vartheta^{(0)}) \\ J_{\text{VAL}}(\vartheta^{(1)}) \\ \vdots \\ J_{\text{VAL}}(\vartheta^{(k)}) \end{bmatrix} \Rightarrow \underbrace{\text{BEST } J_{\text{VAL}}}_{\min} \Rightarrow J_{\text{TEST}}(\vartheta^{\text{BEST}})$$

Interpretazione della procedura. Per ogni valore candidato di λ , si ottiene un modello diverso (poiché la funzione di costo include il termine di regolarizzazione). Ciascun modello viene valutato sul validation set tramite J_{VAL} , che fornisce una stima della capacità di generalizzazione. Si seleziona quindi il valore di λ che minimizza l'errore di validazione.

In questo modo si riesce a trovare il miglior parametro di regolarizzazione che minimizza l'errore sul validation set, e si può usare questo parametro per valutare il modello sul test set.

5.3.4 Altre curve di valutazione

Un'altra curva potrebbe essere quella che mostra l'**errore** su *train set* e *validation set* in funzione del numero di campioni di training (figura 5.4). In questo modo si può capire se il modello soffre di overfitting o underfitting:

La figura conferma il fatto che esiste il trade-off bias-varianza, in quanto:

- Se siamo in presenza di *high bias*, quindi un alto gap tra la curva di train e l'asse delle ascisse, aumentare il numero di dati di training non aiuta molto.
- Se siamo in presenza di *high variance*, quindi un grande gap tra l'errore commesso tra il training set e il validation set, aumentare il numero di dati di training potrebbe aiutare a migliorare i risultati (non sempre).

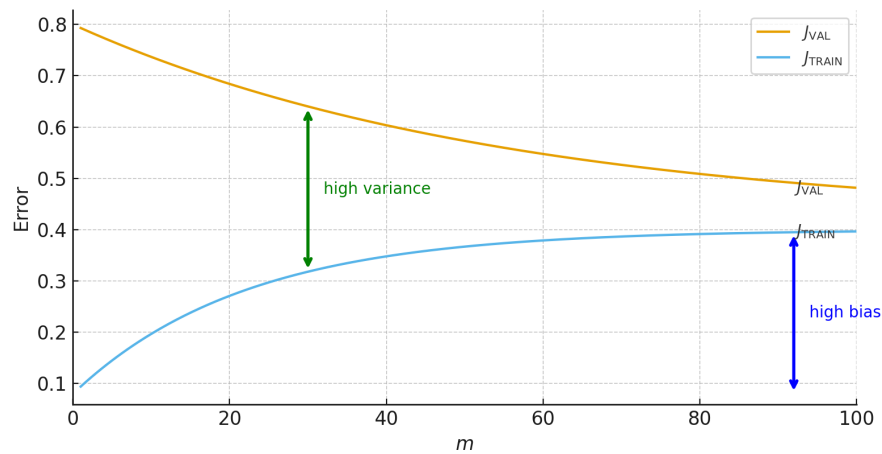


Figura 5.4: Learning curves: errore su training J_{TRAIN} e su validation J_{VAL} in funzione del numero di esempi m . J_{VAL} decresce, J_{TRAIN} cresce leggermente e il *generalization gap* (freccia) si riduce; le curve si avvicinano all'errore irreducibile.

Riferimenti

Il materiale di questo capitolo comprende:

- Materiale visto a lezione.
- Presentazione del prof. Giovanni M. Farinella.

Capitolo 6

Introduzione alle reti neurali

Le reti neurali, grazie alla loro capacità di gestire dati non lineari e molto rumorosi, e di riconoscere pattern complessi, ritrovano moltissime applicazioni per la risoluzione di task di vario tipo. Esistono inoltre numerose varianti delle reti neurali (profonde e non), tra cui la MLP (di cui parleremo) e la CNN.

6.1 Definizione di rete neurale

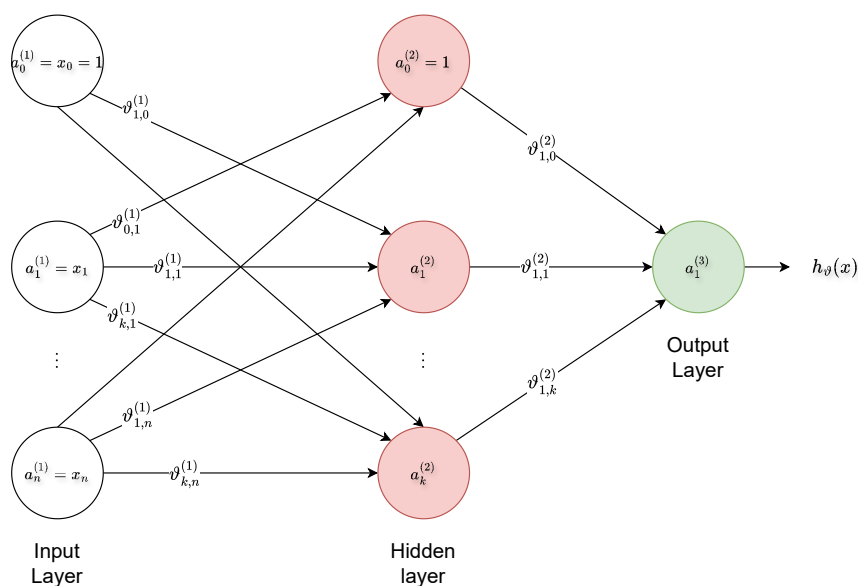


Figura 6.1: Un multilayer perceptron, ossia una particolare rete neurale.

Una rete neurale è una funzione matematica *annidata* del tipo:

$$f_n(f_{n-1}(\dots(f_1(x))))$$

dove n coincide col numero di strati, o *layer*. Questa rete neurale ritorna uno scalare y . Una rete neurale è quindi ottenuta da una **composizione di funzioni derivabili**. Le funzioni sono dette **funzioni di attivazione**, e andremo a vederne presto alcuni esempi. Ogni funzione f_i

è detta **layer** che è composto da un certo numero di **neuroni**, o unità, che sono a loro volta composti da:

- Una trasformazione lineare $z = Wx + b$, in cui W è la matrice dei pesi, b è il bias, e x è l'input del layer. Il numero di neuroni di un layer determina il numero di righe della matrice dei pesi W .
- Una funzione di attivazione $a = \sigma(z)$, che introduce non linearità nella rete. La funzione di attivazione viene applicata elemento per elemento al vettore z .

I neuroni di un layer sono connessi a quelli del layer successivo tramite i pesi W e i bias b . La rete neurale è quindi una struttura a strati, in cui ogni strato trasforma l'input ricevuto dal precedente e lo passa al successivo, fino a raggiungere l'output finale.

Gli strati si suddividono in:

- **Input Layer:** $a_0^{(1)}, a_1^{(1)}, \dots, a_n^{(1)}$. Ciascuno degli input viene moltiplicato per il rispettivo peso della coppia input-hidden layer. Questo viene realizzato tramite un prodotto righe per colonne tra la matrice dei pesi di ciascuno degli elementi dell'hidden layer, con i parametri d'input.
- **Hidden Layer:** sono tutti i layer che non sono né input né output. Il numero di hidden layer definisce la profondità della rete neurale. Una rete neurale si dice profonda (deep) quando si hanno $2 \leq$ hidden layers. Il numero di neuroni può cambiare da layer a layer. Il layer n -esimo comunica col layer $n + 1$ -esimo.
- **Output Layer:** riunisce i dati in output. La funzione di attivazione dell'output layer dipende fortemente dal tipo di task. Tra queste funzioni, abbiamo ad esempio la sigmoide, la tangente iperbolica, la softplus o la ReLU.

Nelle reti neurali, individuiamo due parti fondamentali:

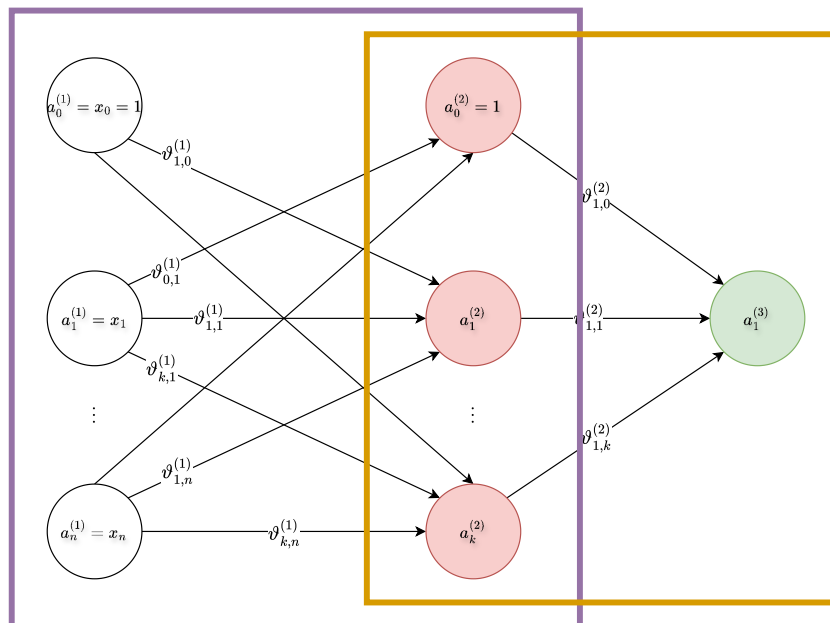


Figura 6.2: Un multilayer perceptron

Nella rete neurale fornita in figura 6.2, individuiamo due parti.

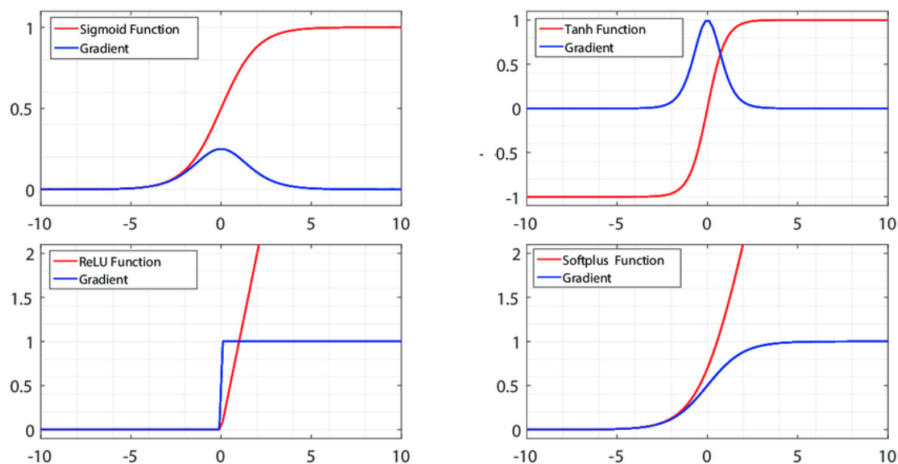
- La parte in viola della rete è dedicata alla **trasformazione degli input grezzi** in qualcosa di utile per il task. La trasformazione viene appresa in fase di training grazie alle etichette note dell'apprendimento supervisionato. In questo caso, permette di trasformare $x \rightarrow a^{(2)}$.
- La parte arancione invece è dedicata alla **classificazione** (tramite softmax, o regressore logistico nella classificazione binaria), prendendo come input i dati trasformati dal resto della rete, in questo caso $a^{(2)}$. Basta sostituire il softmax con un regressore lineare per usare la rete su task di regressione.

6.2 Le funzioni di attivazione

Le funzioni di attivazione devono essere derivabili per consentire il training della rete tramite algoritmo di discesa del gradiente e di backpropagation. Tra le funzioni più usate, abbiamo:

Nome	Equazione	Derivata
Identità	$f(x) = x$	$f'(x) = 1$
Gradino binario (Binary step)	$f(x) = \begin{cases} 0 & \text{per } x < 0, \\ 1 & \text{per } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{per } x \neq 0, \\ ? & \text{per } x = 0 \end{cases}$
Logistica (o Soft step)	$f(x) = \frac{1}{1 + e^{-x}}$	$f'(x) = f(x)(1 - f(x))$
TanH	$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$
ArcTan	$f(x) = \tan^{-1}(x)$	$f'(x) = \frac{1}{x^2 + 1}$
Unità lineare rettificata (ReLU)	$f(x) = \begin{cases} 0 & \text{per } x < 0, \\ x & \text{per } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{per } x < 0, \\ 1 & \text{per } x \geq 0 \end{cases}$
Unità lineare rettificata parametrica (PReLU)	$f(x) = \begin{cases} \alpha x & \text{per } x < 0, \\ x & \text{per } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} \alpha & \text{per } x < 0, \\ 1 & \text{per } x \geq 0 \end{cases}$
Unità lineare esponenziale (ELU)	$f(x) = \begin{cases} \alpha(e^x - 1) & \text{per } x < 0, \\ x & \text{per } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} f(x) + \alpha & \text{per } x < 0, \\ 1 & \text{per } x \geq 0 \end{cases}$
SoftPlus	$f(x) = \log_e(1 + e^x)$	$f'(x) = \frac{1}{1 + e^{-x}}$

Riportiamo alcuni dei plotting delle funzioni di attivazione e le loro derivate.



6.3 Il teorema di approssimazione universale per le reti neurali

Il *Teorema di Approssimazione Universale* afferma che una rete neurale feed-forward¹ con un solo *hidden layer*, dotata di un numero finito ma sufficientemente grande di neuroni e di una funzione di attivazione non lineare (ad esempio sigmoide o tangente iperbolica), è in grado di approssimare qualunque funzione continua definita su un insieme compatto $K \subset \mathbb{R}^n$, con errore arbitrariamente piccolo.

Più formalmente, data una funzione continua

$$f : K \subset \mathbb{R}^n \rightarrow \mathbb{R}$$

e fissato un qualunque $\varepsilon > 0$, esiste una rete neurale con un solo strato nascosto tale che

$$\sup_{x \in K} |f(x) - \hat{f}(x)| < \varepsilon,$$

dove $\hat{f}(x)$ rappresenta la funzione implementata dalla rete neurale.

Interpretazione. Il risultato implica che una rete neurale con un solo hidden layer possiede una capacità espressiva teoricamente universale: non è limitata, in linea di principio, nel tipo di funzione che può rappresentare. La limitazione non è dunque strutturale, ma riguarda piuttosto il processo di apprendimento dei parametri e la quantità di neuroni necessari per ottenere una buona approssimazione.

Esempio. Consideriamo la funzione

$$f(x) = \sin(x), \quad x \in [0, 2\pi].$$

Una rete neurale con attivazioni sigmoide può combinare molte sigmoide traslate e scalate per costruire una curva che approssima la sinusoidale con errore arbitrariamente piccolo. Aumentando il numero di neuroni nello strato nascosto, la rete riesce a rappresentare dettagli sempre più fini dell'andamento della funzione.

Limiti del teorema. È importante sottolineare che il teorema non fornisce un metodo costruttivo per determinare i parametri della rete, né garantisce che un algoritmo di ottimizzazione (come la discesa del gradiente) riesca effettivamente a trovare la configurazione che realizza l'approssimazione desiderata. Inoltre, il teorema non dice nulla sulla capacità di generalizzazione del modello né fornisce un limite superiore sul numero di neuroni necessari, che potrebbe essere molto elevato. In pratica, una rete con un solo hidden layer potrebbe richiedere un numero enorme di neuroni per rappresentare funzioni complesse, risultando inefficiente e potenzialmente soggetta a overfitting.

Questo risultato teorico fornisce quindi una motivazione concettuale all'uso delle reti *deep*: l'introduzione di più strati nascosti consente spesso di rappresentare le stesse funzioni in modo più compatto ed efficiente.

¹Per feed-forward si intende una rete neurale in cui ogni layer riceve input esclusivamente dal layer precedente e lo trasmette a quello successivo, senza cicli o connessioni ricorrenti.

6.4 Training e inferenza di una rete neurale

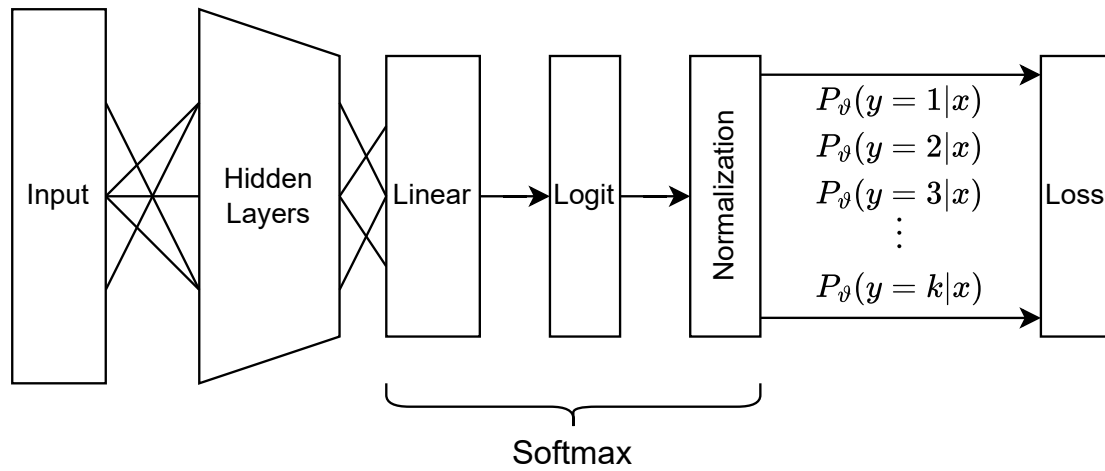


Figura 6.3: Struttura tipica di una rete neurale per la classificazione.

Durante la fase di training, il nostro obiettivo è quello di stabilire i migliori parametri per il nostro modello sui dati di training. Usiamo quindi la Loss function e la nostra procedura di training. Usiamo poi il modello con i parametri ottenuti per fare inferenza su nuovi dati, e osservare le performance del modello.

6.4.1 Funzione di costo

Definiamo la nostra funzione di costo.

$$J(\vartheta) = -\frac{1}{m} \sum_{i=1}^m \underbrace{\sum_{c=1}^k \mathbf{1}\{y^{(i)} = c\} \log(h_{\vartheta}^{(c)}(x^{(i)}))}_{\text{Cross Entropy Loss Function}}$$

in cui

- m = numero di esempi nel dataset
- k = numero di classi
- $(x^{(i)})$ = i -esimo input
- $(y^{(i)})$ = etichetta (classe corretta) dell' i -esimo esempio
- $(\mathbf{1}\{y^{(i)} = c\})$ = funzione indicatrice: vale 1 se l'esempio (i) appartiene alla classe (c) , altrimenti 0
- $(h_{\vartheta}^{(c)}(x^{(i)}))$ = output della rete (probabilità predetta) per la classe (c) sull'esempio (i)

Il nostro obiettivo è quello di trovare $\hat{\vartheta}$, ossia i valori dei parametri ϑ , tali che

$$\hat{\vartheta} = \min(J(\vartheta))$$

6.4.2 Chain Rule

Considerando le reti neurali come delle funzioni composte, o annidate, è possibile usare la chain rule per le derivate di funzioni composte.

$$\frac{\partial J(\vartheta)}{\partial \vartheta_{i,j}^{(l)}} \quad \forall j, i, l$$

6.4.3 Inizializzazione dei parametri

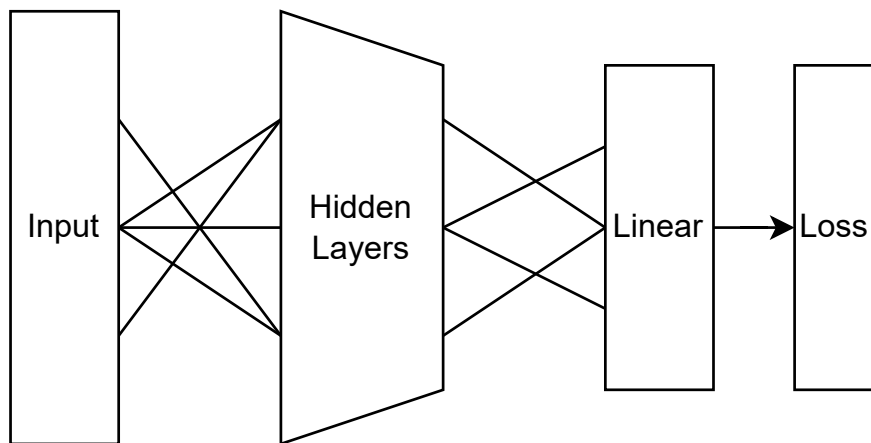
L'inizializzazione dei parametri avviene con numeri casuali molto piccoli $\in [-\varepsilon, +\varepsilon]$. I parametri dei nodi che dipendono dallo stesso nodo, devono essere diversi tra loro, effettuando quella che è nota come **symmetry breaking**.

In pratica, si assegna a neuroni dello stesso layer inizializzazioni diverse, così che non producano le stesse attivazioni e non ricevano gradienti identici.

Perché è importante? Senza symmetry breaking, tutti i neuroni apprenderebbero la stessa funzione attraverso gli stessi gradienti durante la backpropagation, rendendo inefficace l'avere più neuroni nello stesso layer. Questo annullerebbe i benefici della profondità e della larghezza della rete, poiché ogni neurone comporterebbe solo una moltiplicazione ridondante dei pesi identici.

6.4.4 Reti neurali per regressione

Usiamo come Loss function l'errore quadratico medio (MSE). Ogni output sarà un regressore lineare, e la Loss function valuterà gli output di tutti i regressori.



6.5 Overfitting e Underfitting nelle reti neurali

Underfitting. Le reti neurali più piccole, su dati molto complessi, sono soggette ad underfitting. I pochi parametri non permettono di descrivere (o separare) i pattern complessi dei dati. Si riscontrano alti errori già in fase di training. Reti più semplici sono però più veloci da allenare.

Overfitting. Le reti neurali più complesse, con tanti parametri, imparano meglio i dati in fase di training, minimizzando o addirittura azzerando la loss function. Il rischio è opposto: il modello

diventa troppo specifico, e non riesce a generalizzare sui dati di testing, cadendo in overfitting. Diventa quindi necessario introdurre termini di regolarizzazione (λ) o cambiare modello. Modelli più vasti richiedono inoltre risorse computazionali più importanti.

6.6 Caso studio: MLP

Un MLP (Multi-Layer Perceptron) è una rete neurale feed-forward composta da più strati di neuroni, in cui ogni neurone di un layer è connesso a tutti i neuroni del layer successivo. Un esempio può essere visto in figura 6.1, in cui abbiamo un input layer, un hidden layer e un output layer. L'input layer riceve i dati in ingresso, l'hidden layer elabora le informazioni e l'output layer produce la predizione finale.

6.6.1 Definizione del modello

Consideriamo una rete neurale feed-forward completamente connessa, composta da un input layer, un hidden layer e un output layer. Indichiamo con $a^{(l)}$ il vettore delle attivazioni del layer l e con $\Theta^{(l)}$ la matrice dei pesi che connette il layer $l - 1$ al layer l . L'input della rete è $a^{(1)} = x$. Ogni neurone del layer successivo riceve in ingresso una combinazione lineare delle attivazioni del layer precedente; formalmente, per il generico layer l definiamo

$$z^{(l)} = a^{(l-1)} \Theta^{(l)},$$

dove $z^{(l)}$ rappresenta il vettore delle combinazioni lineari. Il singolo neurone j del layer l è quindi dato da

$$z_j^{(l)} = \sum_{i=1}^{n_{l-1}} a_i^{(l-1)} \theta_{i,j}^{(l)}.$$

Applicando la funzione di attivazione f elemento per elemento otteniamo

$$a^{(l)} = f(z^{(l)}).$$

Nel caso rappresentato in Figura 6.1, ciascun neurone del layer nascosto riceve contributi da tutti i neuroni dell'input layer, mentre il neurone di output riceve contributi da tutte le unità del layer nascosto; la rete è quindi completamente connessa. L'output finale della rete è

$$\hat{y} = a^{(L)}.$$

6.6.2 Funzione di costo

Nel caso di classificazione multiclasse utilizziamo la Cross-Entropy Loss

$$J(\vartheta) = -\frac{1}{m} \sum_{i=1}^m \sum_{c=1}^k \mathbf{1}\{y^{(i)} = c\} \log(h_{\vartheta}^{(c)}(x^{(i)})),$$

dove $h_{\vartheta}(x^{(i)}) = a^{(L)}$ rappresenta l'output della rete per l'esempio i -esimo. Nel caso di regressione utilizziamo invece l'errore quadratico medio

$$J(\vartheta) = \frac{1}{2m} \sum_{i=1}^m \|\hat{y}^{(i)} - y^{(i)}\|^2.$$

L'obiettivo del training è determinare

$$\hat{\vartheta} = \arg \min_{\vartheta} J(\vartheta).$$

6.6.3 Backpropagation

Poiché la rete neurale è una composizione di funzioni, il calcolo dei gradienti avviene tramite la chain rule. Definiamo il termine di errore del layer l come

$$\delta^{(l)} = \frac{\partial J}{\partial z^{(l)}}.$$

Per l'output layer si ottiene

$$\delta^{(L)} = a^{(L)} - y.$$

Per un generico layer interno vale invece

$$\delta^{(l)} = \left(\delta^{(l+1)} (\Theta^{(l+1)})^\top \right) \odot f'(z^{(l)}),$$

dove $\odot$ indica il prodotto elemento per elemento. Il gradiente rispetto ai pesi del layer l risulta

$$\frac{\partial J}{\partial \Theta^{(l)}} = (a^{(l-1)})^\top \delta^{(l)}.$$

6.6.4 Aggiornamento dei parametri

Una volta calcolati i gradienti, i parametri vengono aggiornati tramite discesa del gradiente secondo la regola

$$\Theta^{(l)} \leftarrow \Theta^{(l)} - \alpha \frac{\partial J}{\partial \Theta^{(l)}},$$

dove $\alpha > 0$ rappresenta il learning rate. Il processo viene ripetuto iterativamente fino alla convergenza, ossia fino a quando la funzione di costo non varia in modo significativo oppure viene raggiunto un numero massimo di iterazioni.

Interpretazione

Un MLP realizza una sequenza di trasformazioni lineari seguite da funzioni di attivazione non lineari. Le trasformazioni lineari combinano le informazioni provenienti dal layer precedente, mentre le non linearità permettono alla rete di rappresentare funzioni complesse e di modellare relazioni non lineari tra input e output. Il training consiste nella regolazione iterativa dei pesi $\Theta^{(l)}$ in modo da minimizzare la funzione di costo; grazie alla composizione di più layer, la rete è in grado di apprendere rappresentazioni interne progressivamente più astratte dei dati in ingresso.

Riferimenti

I riferimenti per questo capitolo includono:

- Materiale visto a lezione.
- Teorema di approssimazione universale per le reti neurali 6.3 dal libro *Deep Learning* [5], sezione 6.4.1.
- Caso studio MLP scritto da Paolo Volpini, <https://github.com/paolovolpini>.

Capitolo 7

Decision Tree

Gli alberi decisionali (decision tree) sono modelli utilizzati per la classificazione e la regressione. Essi rappresentano un insieme di regole decisionali organizzate in una struttura ad albero, dove ogni nodo interno rappresenta una condizione su una caratteristica dei dati, ogni ramo rappresenta l'esito della condizione e ogni foglia rappresenta una classe o un'etichetta di output.

7.1 Alberi di classificazione

Un albero decisionale per la classificazione è una struttura gerarchica composta da nodi interni e foglie. Ogni nodo interno rappresenta un test su una caratteristica specifica del dataset, mentre ogni foglia rappresenta una classe di output. L'obiettivo principale di un albero decisionale è suddividere il dataset in modo tale che le tuple appartenenti alla stessa classe siano raggruppate insieme nelle foglie dell'albero.

Questo approccio, comunemente chiamato **divide and conquer**, suddivide iterativamente il dataset in sottoinsiemi più piccoli basati su condizioni specifiche, fino a quando non si raggiungono le foglie dell'albero che rappresentano le decisioni finali.

7.1.1 Divisione ricorsiva

La classe di una tupla q si ottiene seguendo il cammino radice $\rightarrow$ foglia guidato dai blocchi condizionali sui nodi interni. Ogni nodo interno applica un test su una caratteristica A e, in base al risultato del test, si procede lungo il ramo corrispondente fino a raggiungere una foglia che fornisce la classificazione finale. L'insieme di regole *deve* essere **esaustivo e mutuamente esclusivo**¹ (ogni tupla deve essere classificata da una e una sola regola).

La costruzione dell'albero decisionale è molto semplice:

1. Se tutte le tuple del nodo X hanno la *stessa* classe C , crea una foglia C .
2. Altrimenti scegli un attributo A (non ancora usato) e *ramifica* X (*splitting*) secondo i valori/soglia di A ; crea i figli.
3. Per ogni figlio X_i : se puro, fermati; se impuro, ripeti ricorsivamente.

¹Se così non fosse, alcune tuple potrebbero non essere classificate o potrebbero essere classificate da più regole contemporaneamente

Pruning. Se le tuple nel nodo sono poche o la profondità è elevata, si può fermare prima e rendere il nodo una foglia (vedere figura 7.1 con l'attributo "overcast").

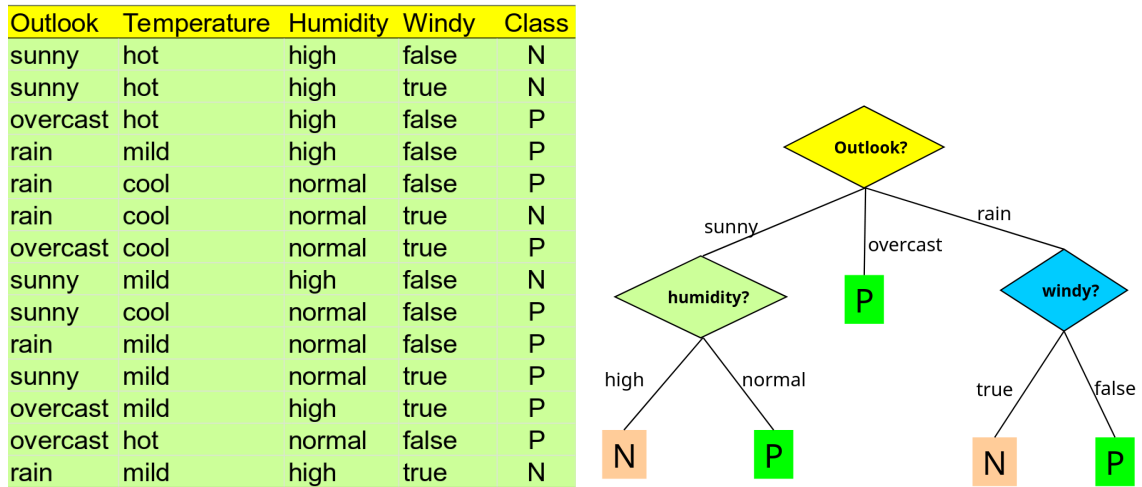


Figura 7.1: Dataset *weather* (a sinistra) e albero decisionale appreso (a destra). La tabella contiene 14 esempi con quattro attributi descrittivi (Outlook, Temperature, Humidity, Windy) e la classe binaria P/N. L'albero (stile ID3/C4.5) sceglie come radice Outlook; il ramo overcast porta direttamente alla classe P, mentre per sunny si testa Humidity e per rain si testa Windy. L'esempio illustra il passaggio da dati tabellari a regole interpretabili.

7.1.2 Migliore divisione dei nodi

Diciamo che per il nodo m , N_m è il numero di tuple nel training set che raggiungono il nodo m (per la radice, $N_{root} = N$, il numero totale di tuple). Sia N_m^i il numero di tuple di classe C_i che raggiungono il nodo m , con $\sum_i N_m^i = N_m$. Sapendo che una tupla x raggiunge il nodo m , la probabilità stimata che appartenga alla classe C_i è:

$$\hat{P}(C_i|x, m) \equiv p_m^i = \frac{N_m^i}{N_m}$$

Definiamo un **nodo puro** come un nodo in cui tutte le tuple appartengono alla stessa classe:

$$\exists i : N_m^i = N_m \implies p_m^i = 1 \quad \text{e} \quad \forall j \neq i : p_m^j = 0$$

ovvero la probabilità stimata di una classe è 1, mentre per tutte le altre è 0. Un nodo è **impuro** se contiene tuple di classi diverse. Un modo frequente per misurare l'impurità di un nodo è utilizzare l'entropia di Shannon:

$$H(m) = - \sum_i p_m^i \log_2(p_m^i)$$

L'entropia misura l'incertezza associata alla distribuzione delle classi nel nodo. Un nodo puro ha *entropia zero*, mentre un nodo con una distribuzione uniforme delle classi ha *entropia massima*.

Minimizzazione dell'impurità. Sapendo che l'entropia è, quindi, una misura dell'impurità del nodo, per costruire un albero decisionale efficace è necessario scegliere l'attributo di split che minimizza l'entropia dei nodi figli dopo la divisione.

7.1.3 Algoritmo di costruzione dell'albero

Procedura ricorsiva

Algorithm 1 GENERATETREE($\mathcal{X}$)

```

1: if NODEENTROPY( $\mathcal{X}$ ) <  $\theta_I$  then
2:   return leaf labelled with majority class in  $\mathcal{X}$ 
3: end if
4:  $i \leftarrow \text{SPLITATTRIBUTE}(\mathcal{X})$ 
5: for each branch of attribute  $x_i$  do
6:   find  $\mathcal{X}_\ell$  falling in branch  $\ell$ 
7:   GENERATETREE( $\mathcal{X}_\ell$ )
8: end for

```

Scelta dell'attributo ottimale di split

Algorithm 2 SPLITATTRIBUTE($\mathcal{X}$)

```

1: MinEnt  $\leftarrow +\infty$ 
2: for  $i = 1, \dots, d$  do
3:   if  $x_i$  is discrete with  $n$  values then
4:     split  $\mathcal{X}$  into  $\mathcal{X}_1, \dots, \mathcal{X}_n$  by  $x_i$ 
5:      $e \leftarrow \text{SPLITENTROPY}(\mathcal{X}_1, \dots, \mathcal{X}_n)$ 
6:     if  $e < \text{MinEnt}$  then
7:       MinEnt  $\leftarrow e$ ; bestf  $\leftarrow i$ 
8:     end if
9:   else  $\triangleright x_i$  numeric
10:    for all possible splits  $t$  of  $x_i$  do
11:      split  $\mathcal{X}$  into  $\mathcal{X}_1, \mathcal{X}_2$  using threshold  $t$ 
12:       $e \leftarrow \text{SPLITENTROPY}(\mathcal{X}_1, \mathcal{X}_2)$ 
13:      if  $e < \text{MinEnt}$  then
14:        MinEnt  $\leftarrow e$ ; bestf  $\leftarrow i$ 
15:      end if
16:    end for
17:   end if
18: end for
19: return bestf

```

Per trovare la miglior combinazione dei nodi eseguiamo la seguente procedura ricorsiva:

- Se il nodo è *puro*, creiamo una foglia con la classe corrispondente.
- Altrimenti, per ogni attributo A non ancora usato, si divide.

Per calcolare l'impurità dei nodi dopo lo splitting (un nodo impuro) bisogna diminuire il valore di impurità. Per calcolarlo:

$$\hat{P}(C_i|x, m, j) \equiv p_{mj}^i = \frac{N_{mj}^i}{N_{mj}}$$

dove N_{mj} è il numero di tuple che raggiungono il figlio j del nodo m e N_{mj}^i è il numero di tuple di classe C_i che raggiungono il figlio j del nodo m . Questo si traduce in una nuova entropia calcolata come:

$$H^j(m) = - \sum_{j=1}^n \frac{N_{mj}}{N_m} \sum_i^k p_{mj}^i \log_2(p_{mj}^i)$$

dove n è il numero di figli del nodo m e k è il numero di classi. L'entropia dopo lo splitting è una media pesata delle entropie dei figli, ponderata per la proporzione di tuple che raggiungono ciascun figlio.

7.2 Alberi di regressione

Un albero di regressione è un modello predittivo utilizzato per stimare valori continui basati su un insieme di caratteristiche. A differenza degli alberi decisionali per la classificazione, che suddividono i dati in classi discrete, gli alberi di regressione suddividono i dati in intervalli continui e forniscono una stima numerica come output.

7.2.1 Costruzione dell'albero di regressione

Uno dei problemi di un albero di regressione è che non abbiamo classi su cui basarci per la divisione dei nodi, ma abbiamo solo valori continui. Il problema quindi diventa "Come faccio a dividere i dati in modo che dentro ogni nodo i valori target siano il più simili possibile?" In altre parole, vogliamo che i valori target all'interno di ogni nodo siano il più possibile vicini tra loro, in modo da poter fornire una stima accurata per quel nodo.

Ipotizziamo di avere un nodo m , con $\mathcal{X}_m$ che rappresenta l'insieme $\mathcal{X}$ delle tuple che raggiungono il nodo m . Quindi è l'insieme di tutti quei nodi x tali che $x \in \mathcal{X}$ e x raggiunge il nodo m . Possiamo definire:

$$b_m(x) = \begin{cases} 1 & \text{se } x \in \mathcal{X}_m \\ 0 & \text{altrimenti} \end{cases}$$

ovvero una funzione indicatrice che vale 1 se la tupla x raggiunge il nodo m e 0 altrimenti.

Nella regressione, la *goodness*² di una divisione viene misurata utilizzando la somma dei quadrati degli errori³ (SSE, Sum of Squared Errors). Per un nodo m , la SSE è definita come:

$$E_m = \frac{1}{N_m} \sum_t (y^t - \hat{y}_m)^2 b_m(x^t)$$

dove y^t è il valore reale della tupla t , $\hat{y}_m$ è la stima associata al nodo m , e N_m è il numero di tuple che raggiungono il nodo m . Si moltiplica per la funzione indicatrice $b_m(x^t)$ per considerare solo le tuple che effettivamente raggiungono il nodo m . La stima media $\hat{y}_m$ per il nodo m è calcolata come:

$$\hat{y}_m = \frac{\sum_t b_m(x^t) y^t}{\sum_t b_m(x^t)} = \frac{1}{N_m} \sum_t b_m(x^t) y^t$$

²Qualità

³O come alternativa, la varianza in quanto $SSE = N \cdot \text{Var}$.

dove la somma è calcolata su tutte le tuple t che raggiungono il nodo m . In pratica, $\hat{y}_m$ è la media dei valori di output delle tuple che raggiungono il nodo m .

7.2.2 Divisione di un nodo

Per dividere un nodo m , si seleziona un attributo A e una soglia di divisione. La divisione crea più figli per il nodo m , ciascuno corrispondente a un intervallo di valori dell'attributo A . Dopo la divisione, si calcola l'errore E_m^j per ciascun figlio j del nodo m .

Se l'errore E_m viene ritenuto accettabile, ovvero $E_m < \theta_r$ per una certa soglia, si crea una foglia con la stima $\hat{y}_m$. Altrimenti, se l'errore non è accettabile, il nodo m viene diviso tante volte finché non si raggiunge una stima accettabile o si esauriscono gli attributi disponibili per la divisione. Si definisce $\mathcal{X}_{mj}$ come l'insieme delle tuple che raggiungono il figlio j del nodo m sottoinsieme di $\mathcal{X}_m$. Definiamo ulteriormente:

$$b_{mj}(x) = \begin{cases} 1 & \text{se } x \in \mathcal{X}_{mj} \\ 0 & \text{altrimenti} \end{cases}$$

La stima media per il figlio j del nodo m è calcolata come:

$$\hat{y}_{mj} = \frac{\sum_t b_{mj}(x^t) y^t}{\sum_t b_{mj}(x^t)} = \frac{1}{N_{mj}} \sum_t b_{mj}(x^t) y^t$$

dove N_{mj} è il numero di tuple che raggiungono il figlio j del nodo m . L'errore dopo lo split è calcolato come:

$$E_m^j = \frac{1}{N_m} \sum_t \sum_j (y^t - \hat{y}_{mj})^2 b_{mj}(x^t)$$

dove la somma esterna è calcolata su tutte le tuple t che raggiungono il nodo m e la somma interna è calcolata su tutti i figli j del nodo m . Ipotizziamo di avere un nodo m , con X_m che rappresenta l'insieme X delle tuple che raggiungono il nodo m . Quindi è l'insieme di tutti quei nodi x tali che $x \in X$ e x raggiunge il nodo m .

7.3 Pruning: riduzione dell'albero

Il pruning è una tecnica utilizzata per ridurre la complessità di un albero decisionale (di classificazione o di regressione), al fine di migliorare la sua capacità di generalizzazione sui dati non visti. Durante la costruzione dell'albero, è possibile che si creino nodi che si adattano troppo strettamente ai dati di addestramento, portando ad *overfitting*. Il pruning mira a rimuovere questi nodi superflui, semplificando l'albero e migliorando le prestazioni sui dati di test.

7.3.1 Pre-pruning

Il pre-pruning consiste nell'interrompere la crescita dell'albero prima che raggiunga la sua massima profondità. Durante la costruzione dell'albero, si valuta la bontà⁴ di ogni divisione e si decide di fermarsi se la divisione non migliora significativamente le prestazioni del modello. Questo

⁴La bontà indica la qualità di misura.

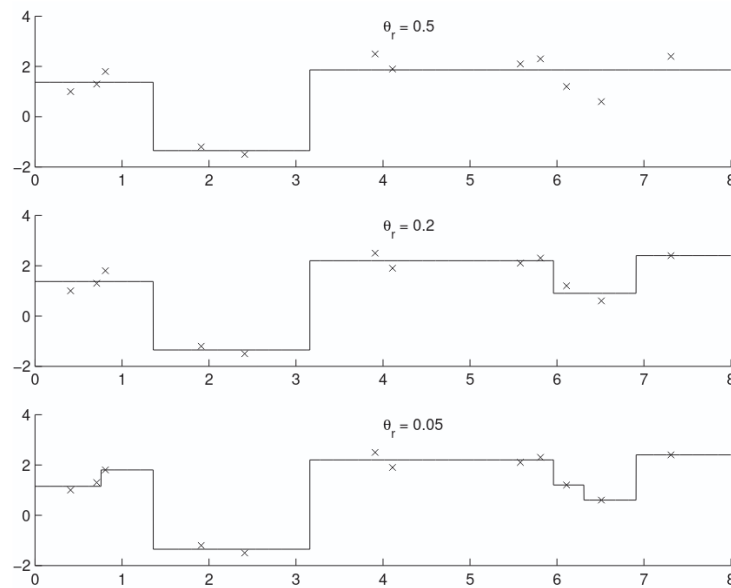


Figura 7.2: Effetto della soglia di arresto θ_r nella costruzione di un albero di regressione. Ogni grafico mostra la funzione stimata (a gradini) ottenuta con diversi valori del parametro di complessità. Con $\theta_r = 0.5$ (in alto) l'albero si arresta precocemente, producendo poche suddivisioni e una funzione più regolare ma meno aderente ai dati (underfitting). Riducendo la soglia a $\theta_r = 0.2$ (centro) aumentano gli split e la funzione segue più fedelmente l'andamento dei punti. Con $\theta_r = 0.05$ (in basso) l'albero continua a suddividere i nodi fino a ottenere una funzione molto frammentata, che interpola quasi perfettamente i dati di training ma rischia overfitting.

può essere fatto utilizzando criteri come la riduzione dell'impurità o la diminuzione dell'errore di regressione.

7.3.2 Post-pruning

Il post-pruning, invece, avviene dopo che l'albero è stato completamente costruito. In questa fase, si esaminano i nodi dell'albero e si valutano le prestazioni del modello su un set di validazione. I nodi che non contribuiscono significativamente alla precisione del modello vengono rimossi, in quanto contribuiscono all'*overfitting*, e i loro figli vengono sostituiti con foglie che rappresentano la stima media o la classe più frequente delle tuple che raggiungevano quel nodo.

7.4 Regole di learning

Generalmente il metodo IF-THEN funziona per addestrare un albero decisional e dopo convertire questo in regole. Altri metodi riguardano l'apprendimento sulle **regole stesse**: invece di costruire un albero e poi estrarre le regole, si *imparano* le regole dai dati e dopo si costruisce l'albero, procedura nota come **rule induction**.

7.4.1 Rule induction

Una regola è generalmente espressa nella forma:

IF condition(s) THEN conclusion

dove le *condition(s)* sono un insieme di test su attributi e la *conclusion* è una classe (per classificazione) o una stima (per regressione). L'obiettivo dell'apprendimento delle regole è identificare un insieme di regole che descrivano accuratamente i dati di addestramento.

Sequential Covering. Le *condition* possono essere combinate utilizzando operatori logici come AND e OR per formare regole più complesse. Ad esempio, una regola potrebbe essere:

IF ($A_1 > 5$) AND ($A_2 = 'Yes'$) THEN Class = Positive

e si aggiungono **una alla volta** per ottimizzare una certa *euristica* (per esempio l'entropia o la riduzione dell'errore). Nell'esempio di prima, si inizia con la prima regola $R_1 = A_1 > 5$, si valuta la bontà di R_1 e si continua ad aggiungere condizioni finché non si raggiunge un certo criterio di arresto (ad esempio, la regola copre un numero sufficiente di tuple positive senza coprire troppe tuple negative). Una volta che una regola è stata appresa, le tuple coperte da quella regola vengono rimosse dal dataset e il processo continua con le tuple rimanenti fino a quando tutte le tuple sono state coperte o non è possibile apprendere ulteriori regole utili.

Differenza con alberi decisionali. Mentre gli alberi decisionali suddividono i dati in modo gerarchico, le regole di apprendimento si concentrano sull'identificazione di condizioni specifiche che portano a conclusioni. Le regole possono essere più flessibili e interpretabili rispetto agli alberi decisionali, poiché possono essere applicate in modo indipendente l'una dall'altra. Si parla infatti di:

Breadth-First Questo approccio, utilizzato dagli alberi decisionali, esplora tutte le possibili divisioni, in modo parallelo, ad ogni livello dell'albero prima di procedere al livello successivo (quindi cresce in *larghezza*).

Depth-First Questo approccio, utilizzato dalle regole di apprendimento, esplora una singola divisione alla volta, procedendo in profondità lungo un percorso specifico prima di tornare indietro e esplorare altre divisioni (quindi cresce in *profondità*).

7.4.2 Copertura delle regole

Si dice che una regola **copre** una tupla t se le condizioni della regola sono soddisfatte dalla tupla t . L'insieme delle tuple coperte da una regola è chiamato **coverage** (copertura) della regola. La copertura aiuta a **valutare** l'efficienza di una regola: una regola con una copertura elevata è generalmente considerata più utile, poiché si applica a un numero maggiore di tuple nel dataset.

7.4.3 Algoritmo RIPPER

RIPPER (Repeated Incremental Pruning to Produce Error Reduction) è un algoritmo di *rule induction* basato sulla strategia di **sequential covering**. Costruisce iterativamente un insieme di

regole che coprono le istanze positive e successivamente ottimizza l'insieme tramite un criterio di descrizione minima (MDL).

Algorithm 3 RIPPER(Pos, Neg, k)

```

1:  $RuleSet \leftarrow \text{LEARNRULESET}(Pos, Neg)$ 
2: for  $t = 1$  to  $k$  do
3:    $RuleSet \leftarrow \text{OPTIMIZERULESET}(RuleSet, Pos, Neg)$ 
4: end for
5: return  $RuleSet$ 

```

Pos e Neg rappresentano rispettivamente le istanze positive e negative del dataset, e k è il numero di iterazioni di ottimizzazione da eseguire. L'algoritmo inizia con l'apprendimento di un insieme iniziale di regole utilizzando la funzione `LEARNRULESET`, che genera regole basate sui dati di addestramento. Successivamente, l'insieme di regole viene ottimizzato iterativamente attraverso la funzione `OPTIMIZERULESET`, che mira a migliorare la qualità delle regole riducendo l'errore e semplificando le regole stesse.

Nota: `LEARNRULESET` e `OPTIMIZERULESET` sono funzioni che implementano rispettivamente la fase di apprendimento delle regole e la fase di ottimizzazione del set di regole. Non sono dettagliate qui, ma si basano su tecniche di pruning e valutazione delle regole per migliorare la generalizzazione del modello.

7.5 Alberi decisionali multivariati

Nel caso di alberi decisionali univariati, ogni nodo interno effettua un test su un singolo attributo alla volta. Si considera quindi una funzione del tipo:

$$f(x) = \begin{cases} \text{figlio destro} & \text{se } A_i \leq \theta \\ \text{figlio sinistro} & \text{se } A_i > \theta \end{cases}$$

dove A_i è un singolo attributo e θ è una soglia. Questo approccio è semplice e facile da interpretare, ma potrebbe non catturare tutte le relazioni complesse tra gli attributi.

7.5.1 Alberi decisionali multivariati lineari

In alcuni casi, potrebbe essere vantaggioso considerare più attributi contemporaneamente per prendere decisioni più complesse, portando alla creazione di *alberi decisionali multivariati lineari* dove si considera una **combinazione lineare** di attributi in ogni nodo:

$$f_m(x) = \mathbf{w}_m^T \cdot x + w_{m0} > 0$$

dove $\mathbf{w}_m$ è un vettore di pesi associati agli attributi, x è il vettore delle caratteristiche della tupla, e w_{m0} è un termine di bias. In questo caso, la decisione su quale figlio seguire dipende dalla valutazione di una combinazione lineare di tutti gli attributi, piuttosto che da un singolo attributo. Questo consente di catturare relazioni più complesse tra gli attributi e può portare a una migliore performance del modello in alcuni scenari.

7.5.2 Interpretazione geometrica

L'uso di combinazioni lineari di attributi in ogni nodo può essere interpretato geometricamente come la suddivisione dello spazio delle caratteristiche in regioni definite da iperpiani. In particolare, la funzione di decisione $f_m(x)$ definisce un iperpiano nello spazio delle caratteristiche, che separa lo spazio in due metà: una metà in cui la condizione è vera (portando al figlio destro) e l'altra metà in cui la condizione è falsa (portando al figlio sinistro) (figura 7.3).

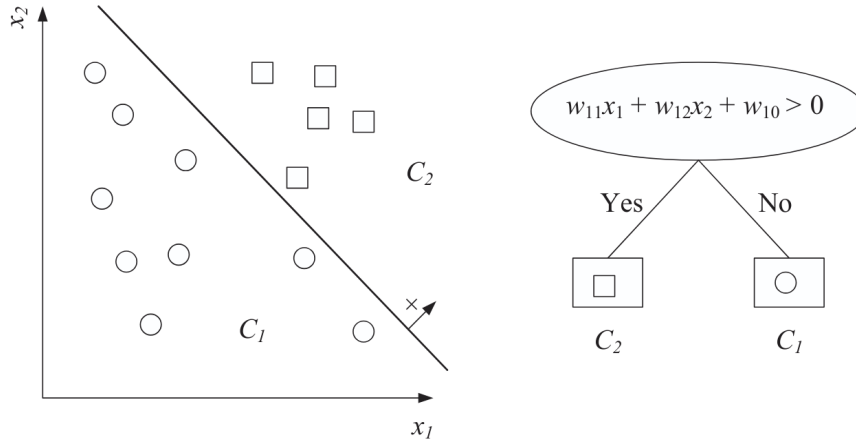


Figura 7.3: Esempio di nodo multivariato: a sinistra, gli esempi delle due classi C_1 (cerchi) e C_2 (quadrati) vengono separati da un iperpiano lineare dove $w_{11}x_1 + w_{12}x_2 + w_{10} = 0$. A destra è mostrato il corrispondente nodo dell'albero di decisione, che instrada gli esempi in base al test $w_{11}x_1 + w_{12}x_2 + w_{10} > 0$.

7.5.3 Alberi decisionali multivariati non lineari

Il concetto di alberi decisionali multivariati può essere esteso ulteriormente per includere funzioni non lineari nei nodi. In questo caso, invece di utilizzare una combinazione lineare di attributi, si possono utilizzare funzioni non lineari più complesse, come polinomi, funzioni radiali o reti neurali. Ad esempio, un nodo potrebbe utilizzare una funzione del tipo:

$$f_m(x) : x^T \mathbf{W}_m x + \mathbf{w}_m^T x + w_{m0} > 0$$

dove $\mathbf{W}_m$ è una matrice di pesi che definisce una funzione quadratica. Questo approccio consente di catturare relazioni ancora più complesse tra gli attributi, ma può rendere l'interpretazione del modello più difficile.

In generale, una funzione non lineare può essere rappresentata come:

$$f_m(x) = g_m(x) > 0$$

dove $g_m(x)$ è una funzione non lineare definita sui dati di input. L'uso di funzioni non lineari nei nodi può migliorare la capacità del modello di adattarsi a dati complessi, ma richiede anche tecniche di ottimizzazione più sofisticate per addestrare l'albero decisionale e non finire in overfitting.

Riferimenti

Il materiale di questo capitolo comprende:

- Materiale visto a lezione.
- Capitolo 9 del libro *Introduction to Machine Learning* [1].

Capitolo 8

Random Forests, Ferns, Meta-Algoritmi

8.1 Meta algoritmi nel machine learning

Un meta-algoritmo, nel contesto del machine learning, rappresenta un modo di combinare più modelli, detti *weak learner* (weak perché sono modelli più semplici e meno potenti). Tra questi, studiamo:

Tree hierarchy. - Definita come una gerarchia di domande (*weak learner*). Ogni nodo è un modello. La struttura ad albero permette di spezzare lo spazio delle features in varie regioni. I decision tree classici rappresentano un caso particolare di questo meta-algoritmo. È una struttura facile da interpretare (*if-then-else*). Un albero decisionale è una particolare istanza di questo.

Bagging (Bootstrap Aggregating). - Migliora i modelli di classificazione, in termini di stabilità e precisione. Riduce la varianza e permette di evitare l'overfitting. Dato un training set I di dimensione n , il bagging genera m nuovi training set I_i , ognuno di essi di dimensione $n' < n$, campionando esempi da I ¹. Ciascuno dei sotto-training set allena un modello differente. L'output complessivo è una media o una votazione.

Boosting. - Gli algoritmi di boosting combinano più modelli deboli per creare un modello forte. A differenza del bagging, il boosting costruisce i modelli in modo sequenziale, dove ogni modello successivo cerca di correggere gli errori del modello precedente.

Trattiamo questi meta-algoritmi, in quanto permettono di ottenere dei modelli strutturalmente semplici, ma sempre più precisi e meno proni a overfitting.

8.2 Decision stump - Ceppo di decisione

È un albero decisionale a singolo nodo. Può essere univariato o multivariato. Data la loro semplicità, sono dei classificatori lineari. Saranno i nostri *weak learners*.

¹Potrebbe ricordare la k -fold validation. Tuttavia, se il bagging è un metodo per ridurre l'overfitting, la cross validation viene utilizzata per stimare l'accuratezza del modello fuori dal campione di training.

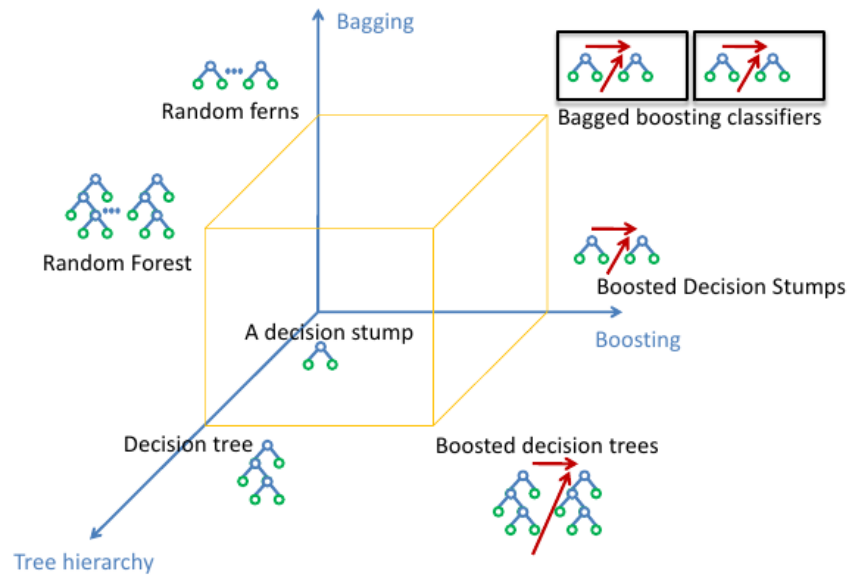


Figura 8.1: Meta-Algoritmi applicati sui decision stump.

8.3 Decision Trees

Applichiamo la **tree hierarchy** sui nostri decision stump, ottenendo una gerarchia di decisioni. Tramite una serie di domande di tipo "yes/no", o varie sogliature, diventa possibile classificare categorie sempre più specifiche. Ogni coppia di risultati di un nodo di un decision tree, è detta *split*. La profondità (direttamente proporzionale al numero di split), definisce la complessità del modello. Alberi decisionali troppo profondi rischiano di cadere in overfitting.

8.4 Random Ferns

Le **Random Ferns** sono un metodo di classificazione progettato per il riconoscimento di patch locali attorno a keypoint, con l'obiettivo di ottenere un sistema estremamente veloce, scalabile e adatto al real-time.

L'idea di fondo è riformulare il problema del matching tra keypoint come un problema di classificazione probabilistica. Ogni keypoint stabile osservato in fase di training viene considerato come una classe c_i . Data una nuova patch x , vogliamo assegnarla alla classe più probabile:

$$\hat{c} = \arg \max_{c_i} P(C = c_i \mid f_1, \dots, f_N)$$

dove $f_1, \dots, f_N$ sono feature binarie calcolate sulla patch.

8.4.1 Feature binarie

Le feature utilizzate sono estremamente semplici: ciascuna consiste in un confronto tra l'intensità di due pixel della patch:

$$f_j = \begin{cases} 1 & \text{se } I(d_{j,1}) < I(d_{j,2}) \\ 0 & \text{altrimenti} \end{cases}$$

Queste feature sono scelte casualmente e in numero elevato (N può essere dell'ordine di alcune centinaia). La discriminatività non deriva dalla singola feature, ma dall'aggregazione di molte di esse.

8.4.2 Formulazione probabilistica

Applicando il teorema di Bayes:

$$P(C = c_i | f_1, \dots, f_N) = \frac{P(f_1, \dots, f_N | C = c_i) P(C = c_i)}{P(f_1, \dots, f_N)}$$

Assumendo prior uniformi sulle classi e ignorando il denominatore (comune a tutte), la decisione si riduce a:

$$\hat{c} = \arg \max_{c_i} P(f_1, \dots, f_N | C = c_i)$$

Il problema è che modellare la distribuzione congiunta completa richiederebbe stimare 2^N configurazioni per classe, il che è costoso per N grande.

8.4.3 Approccio Semi-Naive Bayes

Per rendere il problema trattabile si introduce un compromesso tra indipendenza totale e modellazione completa.

Le N feature vengono suddivise in M gruppi (detti *ferns*), ciascuno di dimensione S :

$$F_k = \{f_{\sigma(k,1)}, \dots, f_{\sigma(k,S)}\}$$

Si assume indipendenza tra gruppi diversi, ma si modella la distribuzione congiunta completa all'interno di ciascun gruppo:

$$P(f_1, \dots, f_N | C = c_i) = \prod_{k=1}^M P(F_k | C = c_i)$$

Ogni fern può assumere 2^S configurazioni binarie. Il numero di parametri per classe diventa quindi:

$$M \cdot 2^S$$

che è gestibile per valori tipici come $S \approx 10-11$ e M dell'ordine di alcune decine.

8.4.4 Stima delle probabilità

Durante il training si stimano le probabilità

$$P(F_k = r | C = c_i)$$

tramite conteggi sulle patch di training (spesso generate applicando trasformazioni affini casuali per aumentare la robustezza).

Per evitare probabilità nulle si utilizza una regolarizzazione di tipo additivo:

$$P(F_k = r \mid C = c_i) = \frac{N_{r,c_i} + \alpha}{N_{c_i} + 2^S \alpha}$$

dove $\alpha > 0$ è un parametro di smoothing.

8.5 Random Tree

Un **Random Tree** è una variante del classico decision tree in cui la costruzione dell'albero non avviene in modo completamente deterministico, ma introduce una componente di casualità nella scelta degli split.

Negli alberi decisionali standard, per ogni nodo si valutano sistematicamente tutte le possibili feature (e spesso tutte le possibili soglie) selezionando quella che massimizza la riduzione dell'impurità. In un random tree, invece, non si esplora l'intero spazio delle possibilità: si genera un insieme limitato di proposte casuali e si sceglie la migliore tra queste.

In particolare, ad ogni nodo:

- si seleziona casualmente un sottoinsieme di feature candidate;
- per ciascuna feature si generano una o più soglie casuali;
- si valuta un criterio di qualità solo su queste proposte;
- si sceglie lo split che produce il miglior miglioramento secondo il criterio adottato.

La struttura dell'albero rimane quella di un decision tree classico, ma la procedura di costruzione è randomizzata. Questo rende il modello meno sensibile a piccole variazioni dei dati e particolarmente adatto a essere utilizzato all'interno di metodi ensemble come le Random Forest.

8.5.1 Randomized Learning

La costruzione del random tree avviene in modo ricorsivo.

Dato un nodo associato a un insieme di indici I_n (cioè i campioni che raggiungono quel nodo), si considera uno split basato su una funzione f applicata ai vettori di feature v_i . Lo split è definito da una soglia t :

$$I_l = \{i \in I_n \mid f(v_i) < t\} \quad I_r = I_n \setminus I_l$$

dove:

- v_i è il vettore delle feature del campione i ,
- f è una feature candidata,
- t è una soglia scelta casualmente nell'intervallo compreso tra il valore minimo e massimo assunto da $f(v_i)$ nel nodo corrente.

Lo scopo è suddividere i dati in due sottoinsiemi il più possibile omogenei rispetto alla variabile target.

8.5.2 Guadagno di informazione

Per valutare la qualità di uno split si utilizza una misura di impurità, indicata con $E(I)$ (come per esempio l'entropia). Il criterio di scelta si basa sulla riduzione di impurità, ossia sul **guadagno di informazione**:

$$\Delta E = E(I_n) - \frac{|I_l|}{|I_n|} E(I_l) - \frac{|I_r|}{|I_n|} E(I_r)$$

Questa quantità misura quanto l'incertezza nel nodo padre viene ridotta dallo split. Lo split selezionato è quello che massimizza ΔE tra le proposte generate casualmente.

Il processo viene poi applicato ricorsivamente ai nodi figli fino a quando si soddisfa un criterio di arresto (profondità massima, numero minimo di campioni, purezza del nodo, ecc.).

8.5.3 Ruolo della casualità

Un singolo decision tree è un modello ad alta varianza: piccole variazioni nei dati possono produrre alberi molto diversi. La randomizzazione:

- riduce la correlazione tra alberi diversi;
- aumenta la diversità nei modelli;
- migliora la robustezza quando più alberi vengono combinati;
- riduce il rischio di overfitting.

Per questo motivo i random trees vengono spesso utilizzati come componenti di metodi ensemble, dove più alberi vengono addestrati su dati o feature diverse e le loro predizioni vengono aggregate (media o voto).

8.5.4 Randomized Learning

Il concetto di **randomized learning** consiste proprio nell'introdurre casualità controllata durante l'apprendimento per ottenere modelli meno correlati e più stabili quando combinati. Nel contesto del riconoscimento di immagini, la randomizzazione può riguardare:

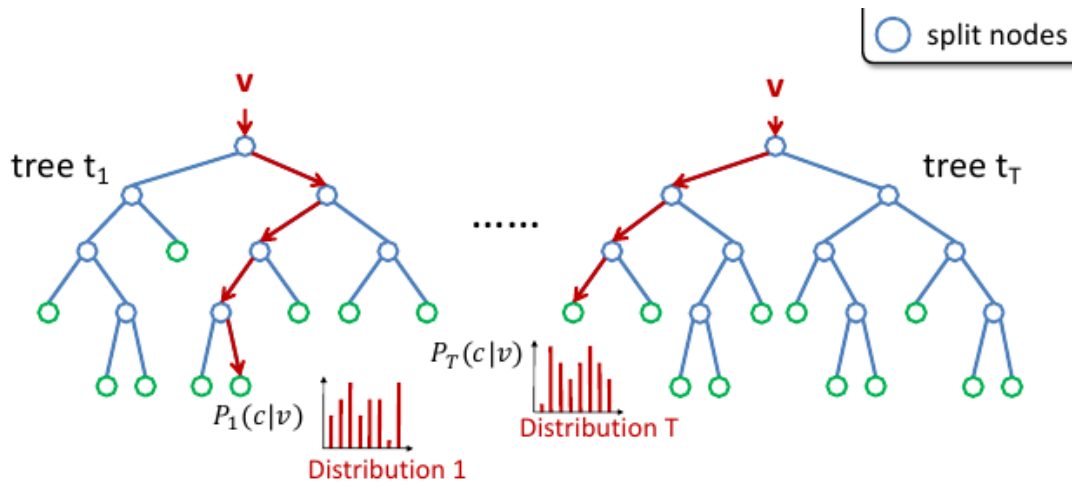
- la scelta delle feature;
- la scelta delle soglie;
- la struttura dell'albero;
- oppure la suddivisione dei dati (come nel bagging).

L'idea generale è che un singolo modello casuale possa essere debole o instabile, ma la combinazione di molti modelli randomizzati produce un sistema complessivamente più robusto ed efficace.

8.6 Random forest

Le random forest nascono dall'unione delle intuizioni relative al bagging e alla tree hierarchy. Si basano sulla combinazione di output di più decision trees (spesso random trees) per ottenere un

unico risultato. La loro facilità d'uso e la flessibilità ne hanno favorito l'adozione, e possono essere usate sia su task di classificazione che di regressione. Offrono un ottimo grado di **precisione**, **cadono meno facilmente in overfitting** e sono assolutamente **multi-core friendly**.



La classificazione è ottenuta considerando la distribuzione media computata dalle foglie.

$$\frac{1}{T} \sum_{t=1}^T P_t(c|v)$$

Riferimenti

I riferimenti per questo capitolo includono:

- Materiale visto a lezione.
- Random Ferns dal paper «Fast Keypoint Recognition using Random Ferns» [7].

Capitolo 9

SVM: Support Vector Machines

Le SVM sono modelli di machine learning supervisionato che possono essere utilizzati sia per problemi di classificazione che di regressione. L'idea alla base delle SVM è trovare un iperpiano che separi i dati di diverse classi con il massimo margine possibile: sono quindi una delle soluzioni più eleganti e semplici da visualizzare per quando riguarda la classificazione. L'iperpiano scelto non solo separa le classi, ma lo fa in modo tale da **massimizzare la distanza tra l'iperpiano e i punti** più vicini di ciascuna classe, noti come vettori di supporto.

9.1 Iperpiani di separazione

Dati i vettori x_i e x_j , un modo di definirne la similarità è il prodotto scalare tra i due¹ :

$$\langle x_i, x_j \rangle = x_i^T x_j = \|x_i\| \|x_j\| \cos \theta$$

dove $\|x_i\|$ e $\|x_j\|$ sono le norme dei vettori x_i e x_j , e θ è l'angolo tra i due vettori. Da questo si evince che la similarità tra i due vettori è massima quando sono paralleli (ovvero quando $\theta = 0$), e minima quando sono ortogonali (ovvero quando $\theta = 90^\circ$).

Il valore di questo prodotto dipende dalla norma² dei due vettori. Questo permette di definire un iperpiano di separazione come l'insieme dei punti x che soddisfano l'equazione:

$$\langle w, x \rangle + b = 0$$

dove w è un vettore normale all'iperpiano e b è l'intercetta.

9.1.1 Funzione di decisione lineare

L'idea principale dietro a diversi algoritmi di classificazione è quello di rappresentare i dati in uno spazio $\mathbb{R}^d$, dove d è il numero di features del piano, e trovare quel *decision boundary* grazie al quale l'algoritmo è capace di separare le diverse classi. Nel caso di una classificazione binaria si separa lo spazio in due regioni corrispondenti alla classe positiva e negativa e si cerca quell'iperpiano

¹In generale, il prodotto scalare tra due vettori a e b è definito come $a^T b = \|a\| \|b\| \cos \theta$.

²La norma di un vettore è una misura della sua lunghezza o grandezza, come il modulo di un vettore in uno spazio euclideo.

che separa le due regioni. Sia $x \in \mathbb{R}^d$ un esempio nello spazio dei dati, consideriamo la funzione:

$$f : \mathbb{R}^d \rightarrow \mathbb{R}, \quad f(x) := \langle w, x \rangle + b$$

dove $w \in \mathbb{R}^d$ e $b \in \mathbb{R}$ sono rispettivamente il vettore dei pesi e il bias. In questo modo, un iperpiano può essere definito come il **separatore** lineare delle due classi, ovvero:

$$\{x \in \mathbb{R}^d : f(x) = 0\} \quad (9.1)$$

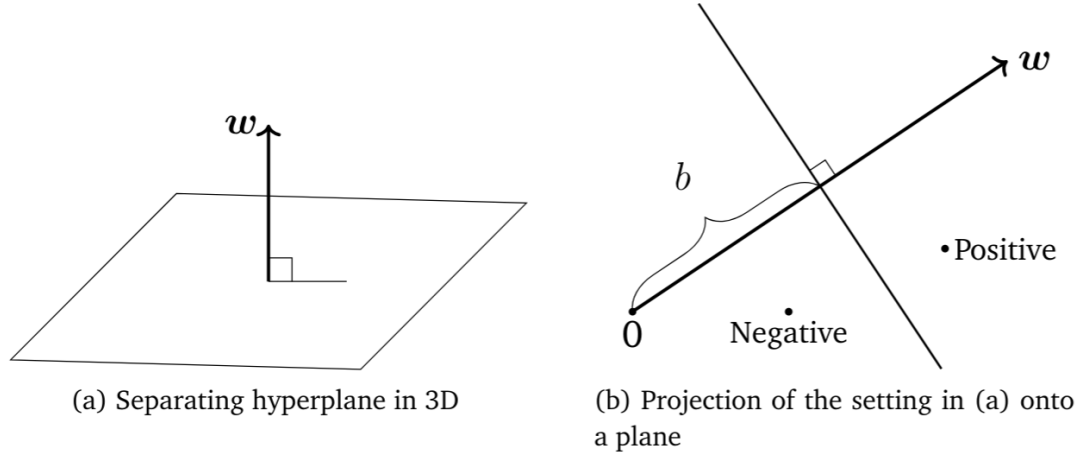


Figura 9.1: Esempio di iperpiano come frontiera di decisione: il vettore w è normale all'iperpiano e ne determina l'orientamento, mentre il termine b ne stabilisce la posizione rispetto all'origine, separando i punti delle due classi sul lato positivo e su quello negativo.

9.1.2 Interpretazione geometrica

Come si vede in figura 9.1, il vettore w è normale³ all'iperpiano e ne determina l'orientamento, mentre il termine b ne stabilisce la posizione rispetto all'origine, separando i punti delle due classi sul lato positivo e su quello negativo. Questo è anche possibile dimostrarlo prendendo in considerazione due esempi x_a, x_b giacenti sull'iperpiano:

$$\begin{aligned} f(x_a) &= f(x_b) \\ \Rightarrow f(x_a) - f(x_b) &= 0 \\ \Rightarrow (\langle w, x_a \rangle + b) - (\langle w, x_b \rangle + b) &= 0 \\ \Rightarrow \langle w, x_a \rangle - \langle w, x_b \rangle &= 0 && (\text{poiché } b - b = 0) \\ \Rightarrow \langle w, x_a - x_b \rangle &= 0 && (\text{per linearità del prodotto scalare}) \end{aligned}$$

Utilizzando l'equazione dell'iperpiano (9.1), possiamo, per un certo esempio x capire se si tratta di una classe positiva o negativa a seconda della regione in cui si trova rispetto all'iperpiano:

$$y = \begin{cases} +1 & \text{se } f(x) \geq 0 \\ -1 & \text{se } f(x) < 0 \end{cases}$$

³Un vettore è normale a un iperpiano se è perpendicolare a ogni vettore che giace sull'iperpiano.

dove y è l'etichetta di classe associata all'esempio x e $f(x)$ è l'equazione dell'iperpiano. Questo si può rappresentare tramite la quantità chiamata **marginale funzionale**:

$$\gamma(x) = y (\langle w, x \rangle + b). \quad (9.2)$$

Un esempio è correttamente classificato se

$$y (\langle w, x \rangle + b) \geq 0.$$

9.2 Primal SVM

Questa formulazione delle SVM ha l'obiettivo di trovare quell'iperpiano che massimizza il margine tra le due classi. Ipotizziamo un dataset $\{(x_1, y_1), \dots, (x_n, y_n)\}$ linearmente separabile, dove $x_i \in \mathbb{R}^d$ sono gli esempi e $y_i \in \{-1, +1\}$ sono le etichette di classe. Esistono infiniti iperpiani che possono separare le due classi, ma possiamo dire che un iperpiano è migliore di un altro se performa meglio sui dati di test. Formuliamo quindi un vero e proprio **problema di ottimizzazione**, cui risoluzione ci offrirà i valori ottimali di w e b .

9.2.1 Concetto di margine

L'idea chiave delle SVM (nel caso linearmente separabile) è scegliere, tra i molti iperpiani separatori possibili, quello che *massimizza il margine*, cioè la distanza tra l'iperpiano di decisione e gli esempi più vicini (i futuri *support vectors*). Per formalizzare questa intuizione dobbiamo però chiarire un punto tecnico: *a quale scala* misuriamo questa distanza?

Invarianza di scala. L'iperpiano

$$\langle w, x \rangle + b = 0$$

non cambia se moltiplichiamo *entrambi* i parametri per una costante positiva $\alpha > 0$:

$$\langle \alpha w, x \rangle + \alpha b = 0 \quad \Longleftrightarrow \quad \langle w, x \rangle + b = 0.$$

Quindi, se non fissiamo una convenzione di scala, quantità come $\langle w, x \rangle + b$ (e derivate) possono essere rese arbitrariamente grandi o piccole senza modificare la frontiera di decisione. Per evitare ambiguità, nella derivazione geometrica fissiamo la scala *normalizzando* il vettore normale all'iperpiano, cioè imponendo $\|w\| = 1$ (norma euclidea).

Distanza punto-iperpiano (proiezione ortogonale). Consideriamo un esempio x_a e la sua proiezione ortogonale x'_a sull'iperpiano (Figura 9.2). Poiché w è normale all'iperpiano, il segmento che unisce x'_a a x_a è parallelo a w . Introduciamo allora una distanza $r > 0$ tale che

$$x_a = x'_a + r \frac{w}{\|w\|}.$$

Se imponiamo $\|w\| = 1$, il termine $\frac{w}{\|w\|}$ è un versore e r coincide con la distanza euclidea tra x_a e l'iperpiano.

Poiché x'_a giace sull'iperpiano, vale $\langle w, x'_a \rangle + b = 0$. Sostituendo $x'_a = x_a - r \frac{w}{\|w\|}$ otteniamo

$$\begin{aligned}
 & \left\langle w, x_a - r \frac{w}{\|w\|} \right\rangle + b = 0 \\
 \Leftrightarrow & \quad \langle w, x_a \rangle - r \left\langle w, \frac{w}{\|w\|} \right\rangle + b = 0 \\
 \Leftrightarrow & \quad \langle w, x_a \rangle - r \frac{\langle w, w \rangle}{\|w\|} + b = 0 \\
 \Leftrightarrow & \quad \langle w, x_a \rangle - r \frac{\|w\|^2}{\|w\|} + b = 0 \\
 \Leftrightarrow & \quad \langle w, x_a \rangle - r \|w\| + b = 0 \\
 \Leftrightarrow & \quad \langle w, x_a \rangle + b = r \|w\|
 \end{aligned}$$

Con la normalizzazione $\|w\| = 1$ segue semplicemente

$$r = \langle w, x_a \rangle + b.$$

In altre parole, quando $\|w\| = 1$, il valore della funzione affine $f(x) = \langle w, x \rangle + b$ (sul lato positivo) misura direttamente la distanza geometrica dall'iperpiano.

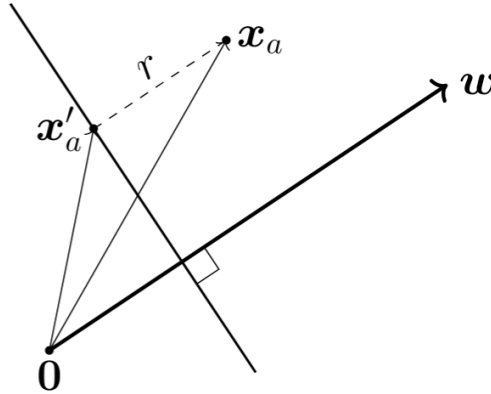


Figura 9.2: Distanza r tra un punto x_a e la sua proiezione ortogonale x'_a sull'iperpiano di separazione.

Il margine come vincolo sui dati. Vogliamo che *tutti* gli esempi siano correttamente classificati e si trovino almeno a distanza r dall'iperpiano, sul lato coerente con l'etichetta. Questo si esprime in un'unica disuguaglianza:

$$y_n (\langle w, x_n \rangle + b) \geq r, \quad n = 1, \dots, N,$$

dove $r > 0$ è il margine (distanza del punto più vicino). A questo punto l'obiettivo è massimizzare r , mantenendo la normalizzazione $\|w\| = 1$. Otteniamo quindi il problema di ottimizzazione

(hard margin, vista geometrica):

$$\max_{w,b,r} r \quad \text{soggetto a} \quad y_n(\langle w, x_n \rangle + b) \geq r, \quad \|w\| = 1, \quad r > 0.$$

Questo formalizza esattamente l'idea: scegliere l'iperpiano che “separa di più” le due classi, cioè quello che lascia il cuscinetto (margin) più ampio attorno alla frontiera di decisione.

9.2.2 Derivazione tradizionale

Un modo equivalente per arrivare alla formulazione *primal* consiste nel fissare la scala in maniera diversa: invece di imporre $\|w\| = 1$, si scala (w, b) in modo che il (o uno dei) punti più vicini all'iperpiano soddisfino

$$\langle w, x_a \rangle + b = 1,$$

cioè si pone il margine “a unità” sulle ipersuperfici parallele $\langle w, x \rangle + b = \pm 1$. Con questa convenzione si ricava che la distanza geometrica del punto più vicino (il margine) risulta proporzionale a $\frac{1}{\|w\|}$, quindi massimizzare il margine equivale a minimizzare la norma di w .

In questa scala, la corretta classificazione con margine unitario si scrive come

$$y_n(\langle w, x_n \rangle + b) \geq 1, \quad n = 1, \dots, N.$$

Il problema di ottimizzazione finale (hard margin SVM) diventa quindi

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad \text{soggetto a} \quad y_n(\langle w, x_n \rangle + b) \geq 1, \quad n = 1, \dots, N,$$

dove il termine $\frac{1}{2} \|w\|^2$ è scelto per comodità analitica (derivate più “pulite”) senza cambiare il minimizzatore.

Nota: si può trovare un approfondimento sulla dimostrazione di questa equivalenza nella sezione B.3 del capitolo B.

9.3 Soft Margin SVM

Nel caso di dati non linearmente separabili, potremmo desiderare di permettere ad alcuni esempi di cadere all'interno dell'area del margine, o addirittura di finire dalla parte sbagliata dell'iperpiano. Il modello che permette alcuni errori di classificazione, è detto **soft margin SVM**. Introduciamo una variabile, detta **variabile di slack**, indicata con ξ_n . Sottrarre questo termine, ci permetterà di rilassare i *constraints* relativi alla classificazione, consentendo errori.

$$\begin{aligned} \min_{w,b,\xi} \quad & \frac{1}{2} \|w\|^2 + C \sum_{n=1}^N \xi_n \\ \text{rispetto a} \quad & y_n(\langle w, x_n \rangle + b) \geq 1 - \xi_n, \\ & \xi_n \geq 0 \end{aligned}$$

La variabile di slack dovrà essere non-negativa. Il parametro $C > 0$ nella funzione obiettivo gestisce il tradeoff tra la dimensione del margine e la dimensione complessiva di rilassamento.

È detto **parametro di regolarizzazione**. Indichiamo invece con il nome **regolarizzatore** il termine $\|w\|^2$, questo controlla la larghezza del margine.

- **C grande:** penalizzi molto le ξ_n . Il modello cercherà di commettere pochissimi errori, anche se questo porta ad avere un margine più stretto, rischiando **overfitting**.
- **C piccolo:** gli errori costano poco, accettando più violazioni, in cambio con un margine più grande, ottenendo un modello più regolare.

9.3.1 Soft Margin SVM e Loss function

Cambiamo il nostro approccio alle SVM, seguendo il **principio di minimizzazione del rischio empirico**⁴. Nel modello SVM, scegliamo come classe di ipotesi⁵ gli iperpiani, ossia funzioni del tipo

$$f(x) = \langle w, x \rangle + b$$

dove il margine, coincide con il termine di regolarizzazione. Affrontiamo adesso l'argomento relativo alla loss function, che dovrà essere mirata al task principale del modello, ossia classificazione binaria.

Zero-One Loss. La **zero-one loss** misura se una predizione è corretta oppure no. L'obiettivo sarebbe minimizzare il numero di esempi per cui la classe predetta $\hat{y}_n = \text{sign}(f(x_n))$ non coincide con l'etichetta reale y_n .

Nel caso di classificazione binaria con $y_n \in \{-1, +1\}$, possiamo esprimere questa loss tramite la funzione indicatrice come

$$\mathbf{1}(y_n f(x_n) \leq 0).$$

Infatti:

- se $y_n f(x_n) > 0$, la predizione è corretta;
- se $y_n f(x_n) \leq 0$, la predizione è errata.

Il rischio empirico associato diventa quindi

$$\sum_{n=1}^N \mathbf{1}(y_n f(x_n) \leq 0).$$

Tuttavia, questa funzione è non continua e non convessa. Minimizzarla porta a un problema di ottimizzazione combinatorio, generalmente difficile da risolvere in modo efficiente.

⁴Nella teoria dell'apprendimento statistico, la minimizzazione del rischio empirico (in inglese empirical risk minimization, da cui la sigla ERM) è un principio che definisce una famiglia di algoritmi di apprendimento e viene utilizzato per fornire limiti teorici alle loro prestazioni. L'idea centrale è che non è possibile sapere esattamente quanto bene funzionerà un algoritmo nella pratica (il "rischio vero") perché è ignota la vera distribuzione dei dati su cui lavorerà l'algoritmo, ma è invece possibile misurarne le prestazioni su un insieme di dati di addestramento noto (il rischio "empirico").

⁵insieme di tutti i modelli possibili che consideriamo

Hinge Loss. La **hinge loss** è una funzione di perdita convessa che penalizza non solo le classificazioni errate, ma anche quelle corrette che si trovano troppo vicine all'iperpiano di separazione. L'idea è sostituire la zero-one loss con una funzione continua e convessa, più semplice da ottimizzare, che favorisca un margine ampio.

Consideriamo l'errore tra l'output del predittore $f(x_n)$ e l'etichetta y_n . Ci aspettiamo loss nulla quando la predizione è corretta e si trova al di fuori del margine (cioè quando il vincolo di margine è soddisfatto). Una predizione corretta ma all'interno del margine viene comunque penalizzata, mentre una predizione errata riceve una penalizzazione maggiore.

La hinge loss è definita come

$$L(t) = \max\{0, 1 - t\}, \quad t = y_n f(x_n) = y_n(\langle w, x_n \rangle + b).$$

Equivalentemente, può essere scritta come funzione a tratti:

$$L(t) = \begin{cases} 0 & \text{se } t \geq 1, \\ 1 - t & \text{se } t < 1. \end{cases}$$

9.4 Dual SVM

La formulazione duale delle SVM si ottiene applicando i moltiplicatori di Lagrange alla formulazione primale. Invece di ottimizzare direttamente sui parametri w e b , il problema viene riscritto in termini dei moltiplicatori α_n , uno per ogni esempio del dataset.

Un risultato fondamentale è che il vettore dei pesi ottimale può essere espresso come combinazione lineare dei dati di training:

$$w = \sum_{n=1}^N \alpha_n y_n x_n.$$

Nel problema duale, la funzione obiettivo dipende esclusivamente dai prodotti scalari tra coppie di esempi:

$$\langle x_i, x_j \rangle.$$

L'ottimizzazione avviene quindi rispetto agli α_n , soggetti ai vincoli

$$\sum_{n=1}^N \alpha_n y_n = 0, \quad 0 \leq \alpha_n \leq C.$$

Questa formulazione ha due conseguenze importanti:

- la complessità dipende dal numero di esempi N , non dal numero di feature D ;
- poiché compaiono solo prodotti scalari tra dati, è possibile sostituire $\langle x_i, x_j \rangle$ con una funzione kernel $K(x_i, x_j)$.

Questa osservazione è alla base del **kernel trick**: lavorare implicitamente in uno spazio delle feature di dimensione maggiore senza doverlo costruire esplicitamente.

Nota: questa parte è stata scritta solo per riuscire a capire meglio il kernel trick. Un approfondimento completo può essere trovato nella sezione B.4 del capitolo B.

9.5 Kernel Trick

La formulazione Dual SVM è molto potente, in quanto permette l'uso di quello che è noto in letteratura come "**kernel trick**". Questo torna utile quando i dati non sono linearmente separabili nello spazio originale.

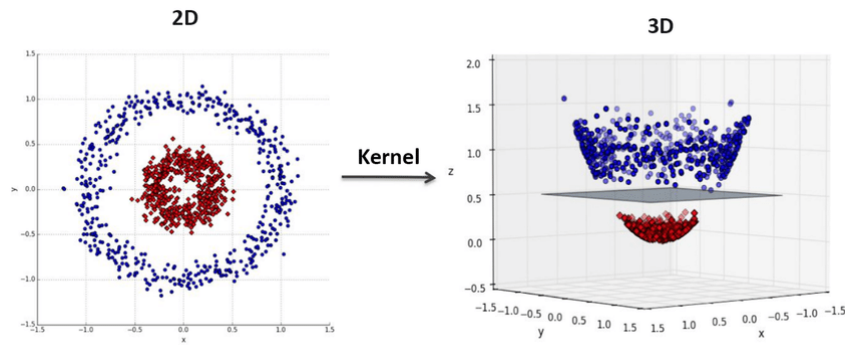


Figura 9.3: Applicazione di kernel su dati non linearmente separabili. Aumentando di una dimensione, i dati diventano separabili da un iperpiano.

9.5.1 Trasformare i dati

il nostro desiderio, è quello di lavorare nei dati su uno spazio con più dimensioni rispetto all'originale. Una trasformazione esplicita del dataset $x \rightarrow \phi(x)$ in uno spazio più grande, presenta i seguenti rischi:

- Vettori molto lunghi a causa delle troppe features ottenute
- Costi in termini di calcoli e memoria molto alti

9.5.2 Il Kernel Trick

Ritorniamo sui nostri passi. La funzione di decisione di un SVM è

$$\min_{\alpha} \quad \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N y_i y_j \alpha_i \alpha_j \langle x_i, x_j \rangle - \sum_{i=1}^N \alpha_i$$

La formulazione in questione, valuta esclusivamente il prodotto scalare delle coppie di campioni $\langle x_i, x_j \rangle$, che ricordiamo essere indice di somiglianza tra i due campioni valutati. Detto ciò, l'idea è quella di **sostituire il prodotto scalare con un kernel**, ossia una funzione $K(x, z)$. Questa, matematicamente, si comporta come un prodotto scalare in uno spazio delle features più grande.

$$K(x, z) = \langle \phi(x), \phi(z) \rangle$$

Possiamo quindi evitare il calcolo esplicito di $\phi(x)$, e lavorare sostituendo il prodotto scalare con il kernel più opportuno.

$$\min_{\alpha} \quad \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N y_i y_j \alpha_i \alpha_j K(x_i, x_j) - \sum_{i=1}^N \alpha_i$$

Ottenendo lo stesso risultato.

Esempio di applicazione con kernel polinomiale di grado 2 Immaginiamo di avere due vettori, $x = (x_1, x_2)$ e $z = (z_1, z_2)$. Supponiamo di mappare $\phi(x)$ e $\phi(z)$, ottenendo

$$\phi(x) = (x_1^2, \sqrt{2}x_1x_2, x_2^2, \sqrt{2}x_1, \sqrt{2}x_2, 1) \quad \phi(z) = (z_1^2, \sqrt{2}z_1z_2, z_2^2, \sqrt{2}z_1, \sqrt{2}z_2, 1)$$

Il prodotto scalare diventa

$$\langle \phi(x), \phi(z) \rangle = x_1^2 z_1^2 + 2x_1 x_2 z_1 z_2 + x_2^2 z_2^2 + 2x_1 z_1 + 2x_2 z_2 + 1$$

Ma lo stesso risultato è ottenibile applicando un kernel polinomiale di grado 2, ossia

$$K(x, z) = (\langle x, z \rangle + 1)^2 = (x_1 z_1 + x_2 z_2 + 1)^2 = x_1^2 z_1^2 + 2x_1 x_2 z_1 z_2 + x_2^2 z_2^2 + 2x_1 z_1 + 2x_2 z_2 + 1$$

Il risultato è il seguente: riusciamo a lavorare su spazi di features molto alti, ma senza mai costruirli.

Riferimenti

Il materiale di questo capitolo è stato tratto da:

- Materiale visto a lezione.
- Parti del capitolo 12 di [3].

Capitolo 10

Ensemble Learning

L'ensemble learning è una tecnica di machine learning che combina le previsioni di più modelli base (detti *base learners* o *weak learners*) per ottenere una previsione complessiva più accurata e robusta. L'idea alla base dell'ensemble learning è che un insieme di modelli, ciascuno con i propri punti di forza e debolezze, possa compensare gli errori reciproci, portando a una migliore performance rispetto a qualsiasi singolo modello.

10.1 Tipi di Ensemble Learning

Per formalizzare il concetto, l'ensemble learning allena L modelli base $h_1(x), h_2(x), \dots, h_L(x)$ su un dataset di addestramento e combina le loro previsioni per ottenere una previsione finale $H(x)$.

10.1.1 Averaging

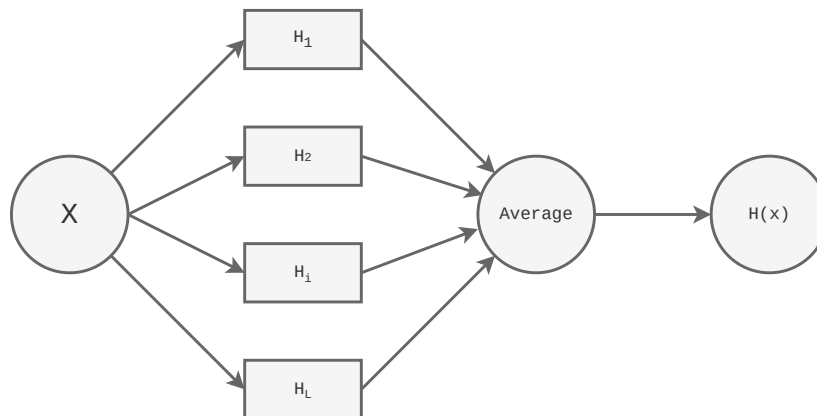


Figura 10.1: Esempio di ensemble learning tramite averaging: esistono L modelli $H_1, H_2, \dots, H_L$ che fanno previsioni su un dato input x . La previsione finale $H(x)$ è la media delle previsioni dei modelli base.

Un primo metodo di fare ensemble learning è l'**averaging**, che consiste nel calcolare la media delle previsioni dei modelli base. Questo metodo è particolarmente utile per problemi di

regressione, dove la previsione finale $H(x)$ è data da:

$$H(x) = \frac{1}{L} \sum_{i=1}^L h_i(x)$$

10.1.2 Weighted Averaging

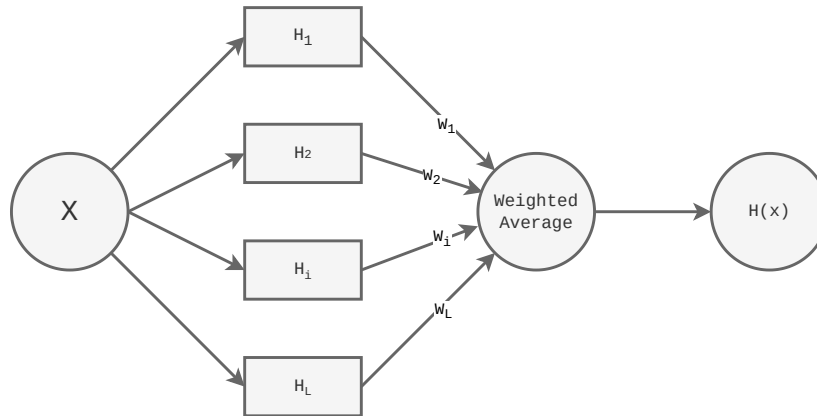


Figura 10.2: Esempio di ensemble learning tramite weighted averaging: ogni modello H_i ha un peso associato w_i che riflette la sua affidabilità. La previsione finale $H(x)$ è una media pesata delle previsioni dei modelli base.

Un'estensione dell'averaging è il **weighted averaging**, dove ogni modello base ha un peso associato w_i che riflette la sua affidabilità. L'idea è di dare più importanza ai modelli che performano meglio. La previsione finale è calcolata come:

$$H(x) = \frac{\sum_{i=1}^L w_i h_i(x)}{\sum_{i=1}^L w_i}$$

10.1.3 Gating

Un metodo più sofisticato di ensemble learning è il **gating**, in cui un modello di gating $g(x)$ decide quale modello base utilizzare per fare la previsione finale. Questo approccio è particolarmente utile quando i modelli base sono specializzati in diverse regioni dello spazio delle caratteristiche. La previsione finale è data da:

$$H(x) = h_{g(x)}(x)$$

dove $g(x)$ restituisce l'indice del modello base da utilizzare per l'input x .

10.1.4 Stacking

Un altro approccio è lo **stacking**, in cui le previsioni dei modelli base vengono utilizzate come input per un modello di livello superiore (detto *meta-learner*) C allenato su un *validation set*. Questo è utile perché permette al meta-learner di apprendere come combinare le previsioni dei

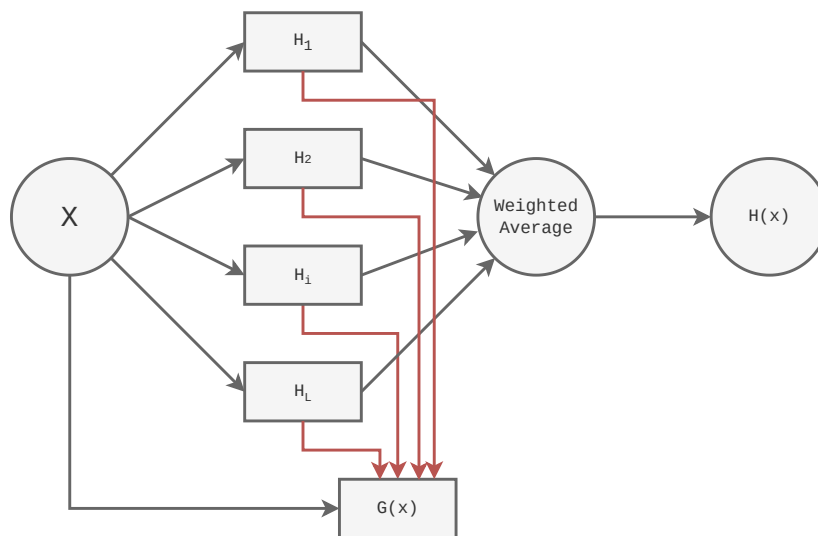


Figura 10.3: Esempio di ensemble learning tramite gating: un modello di gating $g(x)$ seleziona quale modello base utilizzare per fare la previsione finale $H(x)$.

modelli base in modo ottimale. Si può formalizzare come:

$$H(x) = C(h_1(x), h_2(x), \dots, h_L(x))$$

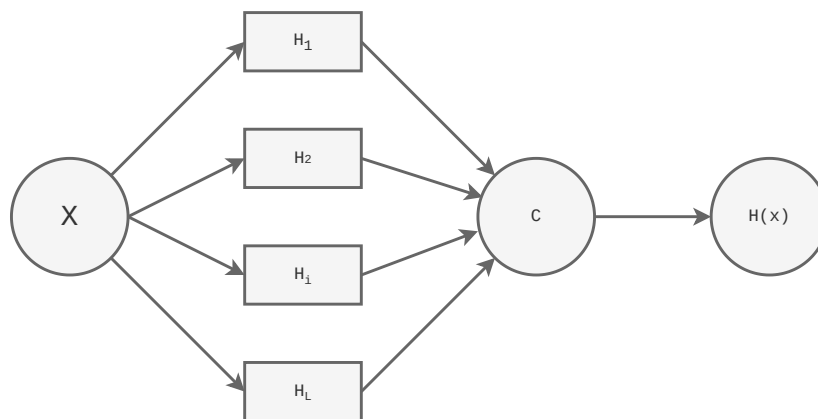


Figura 10.4: Esempio di ensemble learning tramite stacking: le previsioni dei modelli base $H_1, H_2, \dots, H_L$ vengono utilizzate come input per un modello di livello superiore (meta-learner) C che produce la previsione finale $H(x)$.

10.2 Tecniche di Manipolazione dei Dati

Per migliorare la diversità tra i modelli base, l'ensemble learning spesso utilizza tecniche di manipolazione dei dati per creare diversi sottoinsiemi di dati di addestramento. Il vantaggio di questa diversità è che modelli diversi possono catturare differenti aspetti del problema, riducendo il rischio di overfitting e migliorando la generalizzazione.

10.2.1 Bootstrap replication

Una tecnica comune è la **bootstrap replication**, che consiste nel creare diversi sottoinsiemi di dati di addestramento campionando con ripetizione dal dataset originale. Ogni modello base viene addestrato su un sottoinsieme diverso, aumentando la diversità tra i modelli.

Dato un dataset D con n campioni, si creano L sottoinsiemi $D_1, D_2, \dots, D_L$ ciascuno ottenuto campionando n volte con ripetizione da D . Ogni modello base h_i viene quindi addestrato su D_i . Poiché il campionamento è con ripetizione, circa il 36.8% dei campioni originali non verrà incluso in ciascun sottoinsieme, favorendo la diversità tra i modelli.

10.2.2 Bagging

Un'altra tecnica è il **bagging** (bootstrap aggregating), che combina la bootstrap replication con l'averaging. In questo approccio, si creano diversi sottoinsiemi di dati di addestramento tramite bootstrap replication, si addestrano modelli base su ciascun sottoinsieme e si combinano le loro previsioni tramite averaging o weighted averaging. Questo metodo è particolarmente efficace per ridurre la varianza dei modelli base, migliorando la stabilità e la performance complessiva dell'ensemble.

10.2.3 Boosting

Il **boosting** è una tecnica di ensemble learning che mira a creare un forte modello combinando molti modelli deboli in **sequenza**. A differenza del bagging, dove i modelli base sono indipendenti, nel boosting ogni modello successivo viene addestrato per correggere gli errori del modello precedente. Questo processo iterativo porta a un miglioramento continuo delle performance dell'ensemble.

10.3 AdaBoost

Adaboost (Adaptive Boosting) è uno degli algoritmi di boosting più popolari e ampiamente utilizzati. Costruisce un classificatore forte come combinazione pesata di T classificatori deboli (weak hypotheses) h_t , enfatizzando iterativamente gli esempi misclassificati tramite pesi d'istanza $w_{t,i}$.

Input e notazione. Sia il training set $(X, y) = \{(x_i, y_i)\}_{i=1}^n$ e sia T il numero di iterazioni. I pesi al passo t sono $w_t = (w_{t,1}, \dots, w_{t,n})$ e h_t è il modello allenato al passo t .

10.3.1 Algoritmo

Si può descrivere cosa fanno i vari passi dell'algoritmo in questo modo:

1. **Initialize a vector of n uniform weights**

Inizializza una distribuzione uniforme sugli esempi:

$$w_1 = \left[\frac{1}{n}, \dots, \frac{1}{n} \right].$$

Interpretazione: all'inizio tutti gli esempi contano uguale.

Algorithm 4 AdaBoost**Require:** Training set $\{(x_i, y_i)\}_{i=1}^n$ with $y_i \in \{-1, +1\}$, number of rounds T **Ensure:** Final hypothesis H

```

1: Initialize  $w_{1,i} \leftarrow \frac{1}{n}$  for  $i = 1, \dots, n$ 
2: for  $t = 1, \dots, T$  do
3:   Train weak learner  $h_t$  on  $\{(x_i, y_i)\}_{i=1}^n$  using weights  $w_t$ 
4:    $\varepsilon_t \leftarrow \sum_{i: y_i \neq h_t(x_i)} w_{t,i}$ 
5:    $\alpha_t \leftarrow \frac{1}{2} \ln \left( \frac{1-\varepsilon_t}{\varepsilon_t} \right)$ 
6:   for  $i = 1, \dots, n$  do
7:      $w_{t+1,i} \leftarrow w_{t,i} \exp(-\alpha_t y_i h_t(x_i))$ 
8:   end for
9:   Normalize:  $w_{t+1,i} \leftarrow \frac{w_{t+1,i}}{\sum_{j=1}^n w_{t+1,j}}$  for  $i = 1, \dots, n$ 
10: end for
11: return  $H(x) = \text{sign} \left( \sum_{t=1}^T \alpha_t h_t(x) \right)$ 

```

2. **for** $t = 1, \dots, T$

Ciclo di boosting: ad ogni iterazione si costruisce un nuovo weak learner che si concentra sugli esempi più “difficili” (quelli con peso alto).

3. **Train model** h_t **on** X, y **with instance weights** w_t

Allena h_t usando i pesi w_t : gli esempi con $w_{t,i}$ maggiore influenzano di più il training (equivalente a “replicare” più volte quegli esempi).

4. **Compute the weighted training error rate of** h_t :

$$\varepsilon_t = \sum_{i: y_i \neq h_t(x_i)} w_{t,i}.$$

Interpretazione: è la probabilità (secondo la distribuzione w_t) che h_t sbaglia; non è un errore “non pesato”, ma un errore misurato rispetto ai pesi correnti.

5. **Choose** $\alpha_t = \frac{1}{2} \ln \left(\frac{1-\varepsilon_t}{\varepsilon_t} \right)$

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1-\varepsilon_t}{\varepsilon_t} \right).$$

Interpretazione: α_t è il peso del weak learner nell’ensemble finale. Se ε_t è piccolo (modello buono), α_t è grande; se $\varepsilon_t \approx 0.5$, $\alpha_t \approx 0$.

6. **Update all instance weights:**

$$w_{t+1,i} = w_{t,i} \exp(-\alpha_t y_i h_t(x_i)) \quad \forall i = 1, \dots, n.$$

Cosa fa di preciso:

- se $y_i = h_t(x_i)$ (predizione corretta), allora $y_i h_t(x_i) = +1$ e $w_{t+1,i} = w_{t,i} e^{-\alpha_t}$: il peso diminuisce.
- se $y_i \neq h_t(x_i)$ (predizione errata), allora $y_i h_t(x_i) = -1$ e $w_{t+1,i} = w_{t,i} e^{+\alpha_t}$: il peso aumenta.

Quindi i punti sbagliati vengono enfatizzati e i punti corretti “svalutati”.

7. Normalize w_{t+1} to be a distribution:

$$w_{t+1,i} = \frac{w_{t+1,i}}{\sum_{j=1}^n w_{t+1,j}} \quad \forall i = 1, \dots, n.$$

Interpretazione: si riporta la somma dei pesi a 1 così che w_{t+1} resti una distribuzione di probabilità sugli esempi.

8. end for

Dopo T iterazioni si hanno T weak learners e i rispettivi pesi α_t .

9. Return the hypothesis

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right).$$

Interpretazione: è un voto pesato (somma pesata delle predizioni); la $\text{sign}(\cdot)$ restituisce la classe finale.

10.3.2 Decision Boundary

Il decision boundary di un discriminatore è la **frontiera** che separa le diverse classi nello spazio delle caratteristiche. Ovvero quel valore per il classificatore è "indeciso" tra le due classi.

In Adaboost il decision boundary evolve man mano che si aggiungono nuovi weak learners all'ensemble. All'inizio, con pochi weak learners, il decision boundary può essere semplice e lineare. Man mano che si aggiungono più weak learners, il decision boundary diventa più complesso e capace di catturare le caratteristiche non lineari dei dati.

Esempio su dataset two moons. Si può fare un esempio con AdaBoost su un dataset sintetico *two moons* per visualizzare l'evoluzione del decision boundary al crescere del numero di iterazioni. Come weak learner si può usare un *decision stump* (albero di decisione di profondità 1), che fanno una sola divisione lineare.

Nella Figura 10.5 si mostra come il decision boundary di AdaBoost cambia dopo diverse iterazioni (1, 12, 25 e 50). Si può notare che inizialmente il boundary è semplice e lineare, ma con l'aggiunta di più weak learners diventa sempre più complesso, riuscendo a catturare la forma non lineare del dataset.

Riferimenti

I riferimenti principali per questo capitolo sono:

- Materiale visto a lezione.
- Materiale sul boosting e AdaBoost dal capitolo 17 del libro *Introduction to Machine Learning* [1].

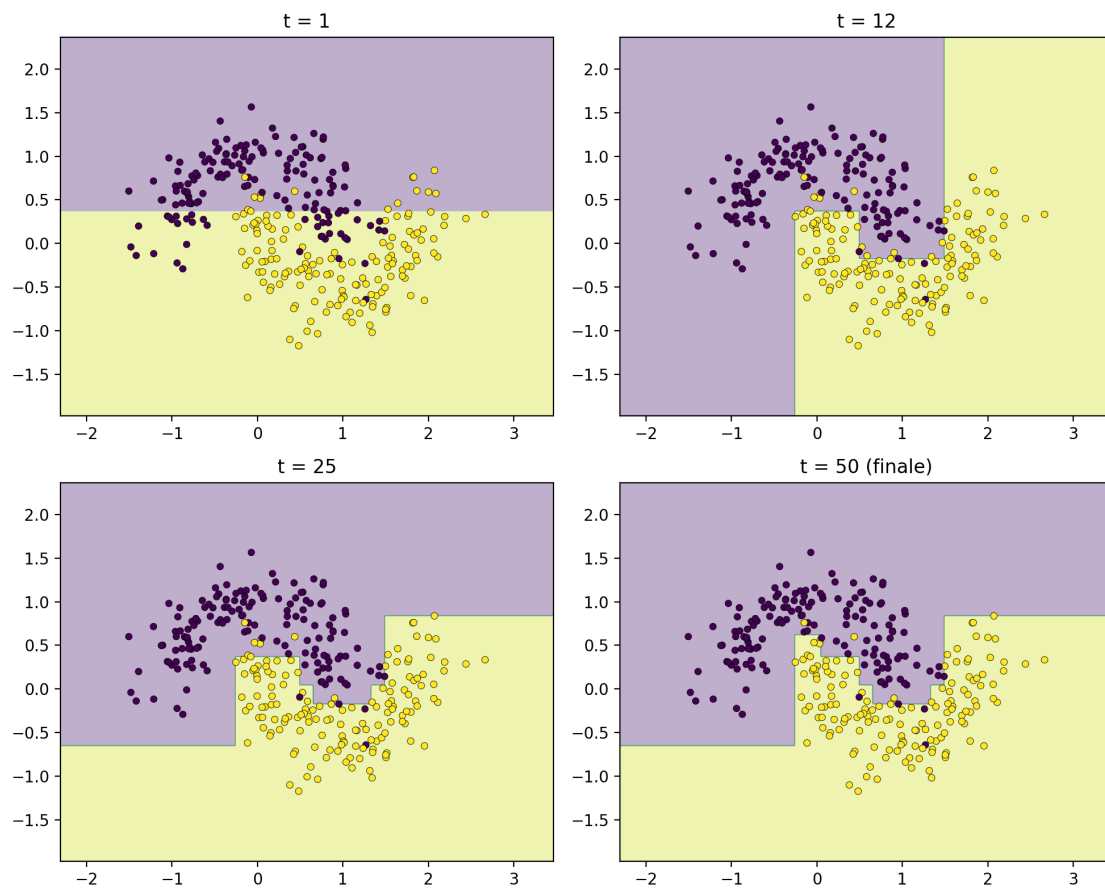


Figura 10.5: Evoluzione del decision boundary di AdaBoost al crescere del numero di iterazioni. In alto a sinistra è mostrato il boundary dopo la prima iterazione ($t = 1$), in alto a destra dopo $t = 12$, in basso a sinistra dopo $t = 25$ e in basso a destra il decision boundary finale ($t = 50$). Si osserva come la combinazione sequenziale di weak learners permetta di approssimare progressivamente una frontiera di decisione non lineare, concentrandosi sulle regioni di maggiore difficoltà.

Parte III

Esempi di Progetti

Appendice A

Classificatore di Risonanze magnetiche

Partecipanti al progetto

Questo progetto, assegnato durante il corso di Machine Learning, è stato realizzato da:

- Diego Martinez - <https://github.com/Diego54523>
- Andrea Tirenti - <https://github.com/Dr-Faxzty>
- Emanuele Galiano - <https://github.com/emanuelegaliano>

Si può trovare il codice completo del progetto al link della repository GitHub:

<https://github.com/Diego54523/unict-ml-year-2025-group-5-ReLU-tanti>

Abstract

Questo lavoro affronta il problema della classificazione di immagini di risonanza magnetica (MRI) cerebrale a supporto della diagnosi della malattia di Alzheimer, considerando quattro stadi clinici: Non Demented, Very Mild Demented, Mild Demented e Moderate Demented. Lo studio è condotto su un ampio dataset pubblico di 35.202 immagini MRI, caratterizzato da una distribuzione sbilanciata delle classi. In una prima fase, sono stati valutati approcci di transfer learning basati su reti convoluzionali pre-addestrate (ResNet-18 e ResNet-50 su ImageNet e RadImageNet), impiegate come feature extractor in combinazione con modelli di classificazione. I risultati hanno evidenziato limiti significativi nella capacità discriminativa, in particolare nella distinzione tra stadi adiacenti e precoci della patologia, attribuibili al domain shift e al texture bias tipico delle CNN addestrate su immagini naturali. Per superare tali criticità, è stata progettata una CNN personalizzata, ottimizzata per il dominio MRI monocanale e addestrata end-to-end con tecniche di data augmentation, regolarizzazione e bilanciamento delle classi. Le feature apprese sono state successivamente validate mediante classificatori MLP e SVM, ottenendo prestazioni elevate e stabili, con un'accuratezza fino al 98%. L'analisi di interpretabilità tramite Grad-CAM conferma che il modello apprende rappresentazioni semanticamente e clinicamente rilevanti. I risultati evidenziano l'importanza di una feature extraction domain-specific per applicazioni di diagnostica medica.

A.1 Problema

La malattia di Alzheimer è una patologia neurodegenerativa progressiva che rappresenta una delle principali cause di demenza nella popolazione anziana. A causa dell'invecchiamento globale della popolazione, il numero di pazienti affetti è in costante aumento, rendendo fondamentale lo sviluppo di strumenti diagnostici affidabili e precoci.

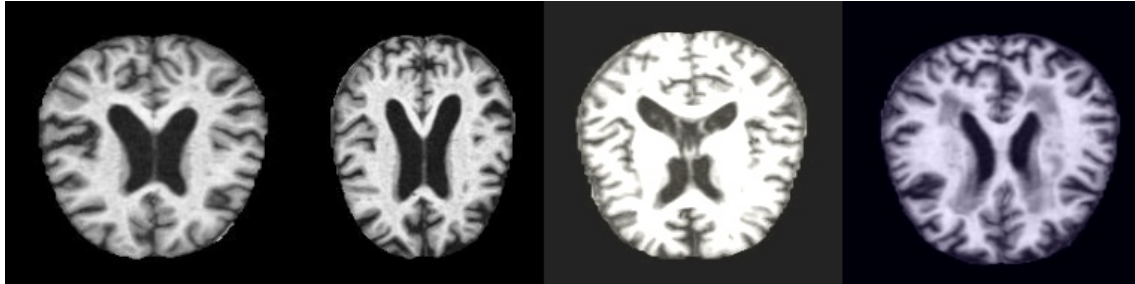


Figura A.1: Esempi di immagini RM (MRI) cerebrali per la classificazione dell'Alzheimer: da sinistra a destra *NonDemented* (assenza di demenza), *Mild Demented* (demenza lieve), *Moderate Demented* (demenza moderata), *Very Mild Demented* (demenza molto lieve).

Il problema affrontato in questo lavoro consiste nella classificazione automatica di immagini MRI cerebrali in quattro classi, rappresentative di differenti stadi della malattia di Alzheimer (ad esempio: *Non Demented*, *Very Mild Demented*, *Mild Demented*, *Moderate Demented*). L'obiettivo è sviluppare un modello in grado di distinguere accuratamente tali classi, supportando il processo diagnostico e contribuendo a una possibile identificazione precoce della patologia.

A.2 Dataset

Il dataset utilizzato è costituito da immagini di risonanze magnetiche (MRI) celebrale ed è stato ottenuto dalla piattaforma **Kaggle**, dove è reso disponibile gratuitamente. L'acquisizione delle immagini non è resa nota nella descrizione sul sito di Kaggle.

Ogni cartella contiene esclusivamente le immagini appartenenti alla classe corrispondente. Il dataset comprende un totale di 35.202 immagini. L'etichettatura dei dati è **già fornita** nel dataset originale e non è stata modificata. Le immagini sono suddivise in quattro classi cliniche, organizzate in cartelle separate, in base al livello di *severità* della demenza (con a seguito il numero di esempi per classe):

1. *Non Demented* 12.800
2. *Mild Demented* - 4.674.
3. *Moderate Demented* - 6.528.
4. *Very Mild Demented* - 11.200.

Questa distribuzione evidenzia una parziale **sbilanciatura tra le classi**.

La suddivisione dei dati in **training set** e **test set** non avviene tramite una separazione fisica delle immagini in cartelle distinte. Al contrario, l'intero dataset viene inizialmente caricato e la divisione tra dati di training e di test viene effettuata a **livello di codice**. Le modalità di split

(percentuale: 80/20) variano in base all'approccio di classificazione adottato (ad esempio utilizzo diretto delle immagini o classificazione basata su feature estratte).

A.3 Metodi

La prima parte del progetto è stata la feature extraction: abbiamo allenato diversi modelli per capire quale riusciva ad estrarre meglio le features.

A.3.1 ResNet-18

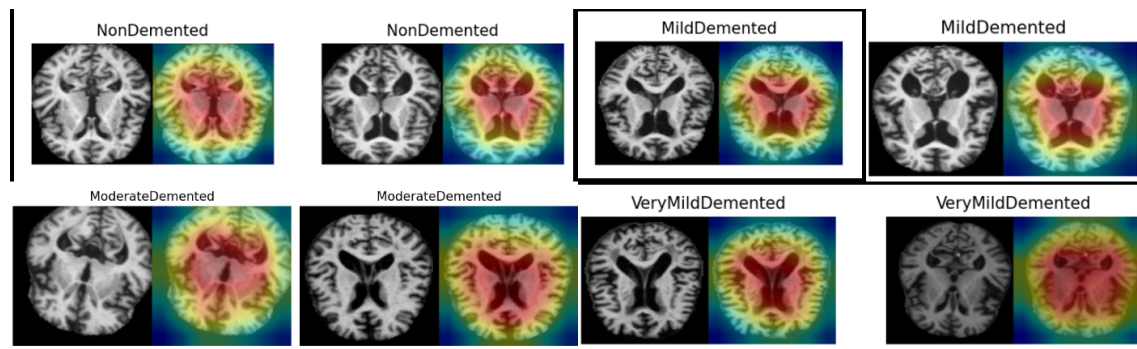


Figura A.2: Esempi di mappe di attivazione (Grad-CAM) ottenute dal modello ResNet-18 per le quattro classi considerate: *NonDemented*, *MildDemented*, *ModerateDemented* e *VeryMildDemented*. Per ciascun esempio sono mostrate l'immagine MRI originale (a sinistra) e la corrispondente heatmap sovrapposta (a destra), che evidenzia le regioni cerebrali maggiormente rilevanti per la decisione del modello. Le aree in rosso indicano le zone con maggiore contributo alla classificazione.

In questa fase sperimentale è stato utilizzato un modello *ResNet-18* pre-addestrato sul dataset ImageNet come feature extractor. Le immagini MRI sono state ridimensionate a 224×224 pixel, al fine di risultare compatibili con l'input del modello. Il dataset è stato suddiviso in training set e test set secondo una proporzione pari all'80% e 20%, rispettivamente.

Per una prima valutazione delle capacità del modello, sono state estratte le feature mediante ResNet-18 e successivamente utilizzate come input di un classificatore *Multi-Layer Perceptron* (MLP) con ultimo strato lineare. Il modello è stato addestrato per un totale di 20 epoche, usando un learning rate pari a 10^{-3} e momentum pari a $9 \cdot 10^{-1}$. I risultati ottenuti sono riportati nelle seguenti tabelle.

Classe	Precision	Recall	F1-score	Support
MildDemented	0.90	0.31	0.47	919
ModerateDemented	0.99	0.97	0.98	1317
NonDemented	0.85	0.64	0.73	2559
VeryMildDemented	0.57	0.89	0.70	2246
Accuracy			0.74	7041
Macro Avg	0.83	0.70	0.72	7041
Weighted Avg	0.79	0.74	0.73	7041

Tabella A.1: Classification report del modello.

Classe reale / Predetta	Mild	Moderate	Non	VeryMild
MildDemented	289	9	55	566
ModerateDemented	1	1283	2	31
NonDemented	21	6	1649	883
VeryMildDemented	10	4	243	1989

Tabella A.2: Confusion matrix del modello.

Il report sopracitato evidenzia limitazioni intrinseche all'utilizzo dell'architettura ResNet-18 come mero feature extractor statico ("frozen") in ambito medico. Le criticità emerse, in particolare

la bassa Recall per la classe MildDemented (0.31) e l'elevata confusione tra NonDemented e VeryMildDemented, trovano riscontro nella letteratura scientifica relativa al transfer learning e al domain shift.

Analisi del modello. Come dimostrato da Raghu et al. [8], le feature apprese su dataset di immagini naturali (ImageNet) privilegiano caratteristiche visive ad alto contrasto e texture specifiche che non si trasferiscono efficacemente alle immagini MRI, dove l'informazione diagnostica risiede in sottili variazioni strutturali e di scala di grigi. Questo fenomeno è aggravato dal cosiddetto texture bias delle CNN, descritto da Geirhos et al. [4]: la rete tende a classificare basandosi sulla texture locale piuttosto che sulla forma globale. Nel contesto dell'Alzheimer, dove la distinzione tra un cervello sano (NonDemented) e uno stadio prodromale (VeryMildDemented) dipende da minime alterazioni morfologiche (e.g., atrofia dell'ippocampo) piuttosto che tessiturali, le feature estratte risultano insufficientemente discriminanti, portando il classificatore a confondere massicciamente queste due classi, come confermato anche dagli studi specifici di Wen et al. [10].

Alla luce di tali osservazioni, si è deciso di ridurre lo svantaggio dato dallo sbilanciamento delle classi introducendo dei pesi, assegnati a ogni classe, ai fini di regolare l'importanza, che il modello dà, agli errori commessi, in base alla classe di riferimento. Sulla base di quanto riportato, è stato effettuato un secondo training del Multi-Layer Perceptron con nuovo numero di epoche di addestramento da 20 variato da 100, mantenendo invariata la struttura del classificatore.

Classe	Precision	Recall	F1-score	Support
MildDemented	0.79	0.91	0.85	919
ModerateDemented	0.98	0.99	0.99	1317
NonDemented	0.93	0.83	0.88	2559
VeryMildDemented	0.84	0.89	0.86	2246
Accuracy			0.89	7041
Macro Avg	0.89	0.91	0.89	7041
Weighted Avg	0.89	0.89	0.89	7041

Tabella A.3: Classification report del modello.

L'aumento del numero di epoche ha determinato un miglioramento significativo delle prestazioni complessive del modello, come evidenziato dall'incremento dell'accuracy totale e delle metriche di precision, recall e F1-score per tutte le classi considerate. Questi risultati evidenziano l'efficacia data dai pesi assegnati alle varie classi, basti prendere come esempio la classe minoritaria, *MildDemented*, la quale adesso presenta una recall drasticamente migliorata, deducibile anche dalla confusion matrix.

Successivamente, è stata valutata una diversa configurazione del classificatore, impiegando un MLP con funzione di attivazione *Softmax* nello strato finale. Il modello è stato addestrato per 100 epoche, utilizzando un learning rate pari a 10^{-3} e un momentum pari a $9 \cdot 10^{-1}$.

Classe	Precision	Recall	F1-score	Support
MildDemented	0.73	0.90	0.81	919
ModerateDemented	0.99	0.99	0.99	1317
NonDemented	0.94	0.76	0.84	2559
VeryMildDemented	0.78	0.87	0.82	2246
Accuracy			0.86	7041
Macro Avg	0.86	0.88	0.87	7041
Weighted Avg	0.87	0.86	0.86	7041

Tabella A.5: Classification report del modello finale con MLP.

True / Pred	Mild	Moderate	Non	Very Mild
MildDemented	835	6	32	46
ModerateDemented	6	1308	1	2
NonDemented	97	5	2129	328
VeryMildDemented	115	12	121	1998

Tabella A.4: Confusion matrix del modello.

True / Pred	Mild	Moderate	Non	Very Mild
MildDemented	831	2	20	66
ModerateDemented	3	1308	0	6
NonDemented	129	7	1942	481
VeryMildDemented	169	8	114	1955

Tabella A.6: Confusion matrix del modello.

Nonostante l'incremento delle prestazioni complessive, questa configurazione non è risultata pienamente soddisfacente. In particolare, dall'analisi della matrice di confusione (Tabella A.6) emerge una capacità limitata del modello di discriminare correttamente la classe *VeryMildDemented*, caratterizzata da un numero significativo di campioni erroneamente classificati. Tale osservazione ha motivato la ricerca di soluzioni alternative in grado di garantire prestazioni più bilanciate tra le diverse classi.

A.3.2 RadNet-50

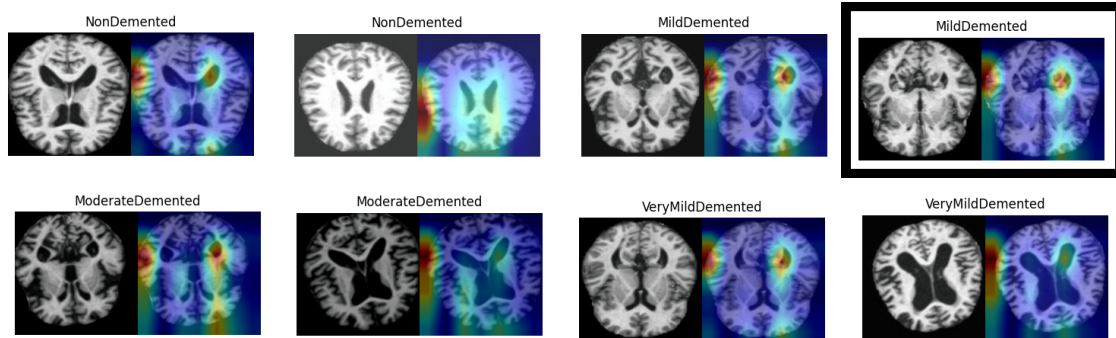


Figura A.3: Esempi di mappe di attivazione (Grad-CAM) ottenute dal modello RadNet-50 per le quattro classi considerate: *NonDemented*, *MildDemented*, *ModerateDemented* e *VeryMildDemented*. Per ciascun esempio sono mostrate l'immagine MRI originale (a sinistra) e la corrispondente heatmap sovrapposta (a destra), che evidenzia le regioni cerebrali maggiormente rilevanti per la decisione del modello. Le aree in rosso indicano le zone con maggiore contributo alla classificazione.

Per migliorare le prestazioni rispetto al feature extractor ResNet-18, abbiamo utilizzato una ResNet-50 pre-addestrata su *RadImageNet*, dall'articolo [6] (di seguito *RadNet-50*), un insieme di dati radiologici che rende il backbone più adatto a domini medicali rispetto al pre-training su ImageNet. Il modello è stato impiegato come **feature extractor** e successivamente le feature sono state date in input ad un classificatore **MLP con Softmax**.

Pre-processing. Le immagini MRI, originariamente in scala di grigi, sono state replicate su tre canali e normalizzate con le statistiche di ImageNet. L'input del modello è stato portato a 224×224 . Durante l'addestramento sono state applicate trasformazioni di data augmentation leggere e compatibili con il dominio MRI, al fine di migliorare la capacità di generalizzazione del modello:

- ridimensionamento a 256×256 e **crop** (casuale in training, centrale in validazione e test);
- trasformazioni affini moderate (rotazioni, traslazioni e variazioni di scala);
- normalizzazione con media e deviazione standard di ImageNet.

Ottimizzazione e iperparametri principali. L'addestramento è stato condotto utilizzando l'ottimizzatore **AdamW** con **weight decay** pari a 10^{-4} e una *K-Fold* a 5 fold. Per stabilizzare il fine-tuning del modello sono stati adottati **learning rate differenziati** per le diverse parti della rete:

- **head** (layer finale): $5 \cdot 10^{-5}$;
- **layer4** del backbone: 10^{-5} ;
- **layer3** del backbone: $5 \cdot 10^{-6}$.

Per ridurre l'overfitting e migliorare la stabilità dell'addestramento sono stati inoltre impiegati **dropout** (0.5) nella head, **label smoothing** (0.05), **gradient clipping** con norma massima pari a 1.0 e **mixed precision** (AMP), quando disponibile. Inoltre, per limitare la deriva del backbone durante il fine-tuning, sono stati congelati i parametri (e le statistiche delle BatchNorm) ad eccezione degli ultimi blocchi convoluzionali (layer4 e, opzionalmente, layer3).

Estrazione delle feature e del classificatore MLP. Una volta ottenuto il backbone ottimizzato, sono state estratte feature di dimensione 2048 (output della rete prima del layer fully connected) e usate come input di un **MLP con Softmax**. Il classificatore è stato addestrato per 50 epoche con **SGD** (learning rate 0.01, momentum 0.9) e **pesi di classe** nella loss per mitigare lo sbilanciamento del dataset.

Risultati (MLP su feature RadNet-50). Di seguito si riportano i risultati ottenuti dal classificatore MLP sulle feature estratte da RadNet-50.

Classe	Precision	Recall	F1-score	Support
MildDemented	0.58	0.18	0.28	935
ModerateDemented	0.68	0.97	0.80	1306
NonDemented	0.71	0.59	0.65	2560
VeryMildDemented	0.49	0.60	0.54	2240
Accuracy			0.61	7041
Macro Avg	0.61	0.58	0.56	7041
Weighted Avg	0.62	0.61	0.59	7041

Tabella A.7: Classification report del classificatore MLP addestrato su feature RadNet-50.

True / Pred	Mild	Moderate	Non	Very Mild
MildDemented	169	135	43	588
ModerateDemented	9	1262	1	34
NonDemented	35	212	1521	792
VeryMildDemented	77	240	584	1339

Tabella A.8: Confusion matrix del classificatore MLP su feature RadNet-50.

Conclusioni. L'approccio è stato in generale fallimentare, probabilmente a causa della natura specifica delle immagini MRI che richiedono un fine-tuning più approfondito del backbone. Tuttavia, l'analisi delle mappe di attivazione (Figura A.3) mostra che il modello non è in grado di focalizzarsi sulle regioni cerebrali rilevanti, suggerendo la necessità di un addestramento più mirato.

L'articolo di Raghu et al. [8], a nostro avviso, fornisce una chiave di lettura convincente per interpretare il comportamento osservato, in parte sovrapponibile a quanto già emerso con ResNet-18. Nonostante il pre-training su un dominio radiologico (RadImageNet), RadNet-50 tende verosimilmente a fare affidamento su indizi a bassa scala, come variazioni locali di intensità e pattern di texture, che risultano poco stabili e solo debolmente informativi nel contesto delle MRI cerebrali. In queste immagini, infatti, l'informazione diagnostica legata agli stadi iniziali dell'Alzheimer è spesso associata a cambiamenti strutturali sottili e globali (es. variazioni morfometriche e atrofie localizzate) piuttosto che a differenze "tessiturali" marcate.

Inoltre, la natura monocolore delle MRI (immagini in scala di grigi) riduce ulteriormente la varietà di segnali utilizzabili dalla rete rispetto a quanto avviene nelle immagini naturali RGB: l'assenza di componenti cromatiche limita la diversità delle feature a cui un backbone pre-addestrato può agganciarsi, aumentando il rischio che l'estrazione delle rappresentazioni si concentri su

correlazioni spurie o dettagli non realmente discriminanti. In questo scenario, un modello di tipo discriminativo, ottimizzato per separare classi a partire dalle feature disponibili, può quindi incontrare difficoltà nel catturare le differenze clinicamente rilevanti tra classi adiacenti (in particolare *NonDemented* vs *VeryMildDemented*), producendo rappresentazioni non sufficientemente separabili per un classificatore a valle come l'MLP. Di conseguenza, i risultati ottenuti suggeriscono la necessità di un fine-tuning più profondo e/o di una pipeline maggiormente “MRI-aware”, capace di enfatizzare segnali morfologici e pattern anatomici su scala più ampia rispetto a semplici variazioni locali di texture.

A.3.3 Custom-CNN

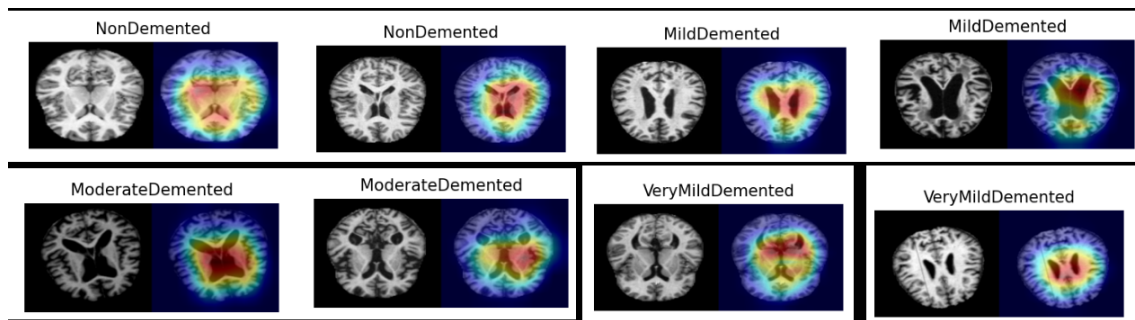


Figura A.4: Esempi di mappe di attivazione (Grad-CAM) ottenute dal modello Custom-CNN per le quattro classi considerate: *NonDemented*, *MildDemented*, *ModerateDemented* e *VeryMildDemented*. Per ciascun esempio sono mostrate l'immagine MRI originale (a sinistra) e la corrispondente heatmap sovrapposta (a destra), che evidenzia le regioni cerebrali maggiormente rilevanti per la decisione del modello. Le aree in rosso indicano le zone con maggiore contributo alla classificazione.

Per migliorare ulteriormente le performance del classificatore, abbiamo progettato e implementato una CNN personalizzata.

Architettura. La Custom-CNN implementata è una rete convoluzionale progettata ad hoc per il dataset MRI. L'architettura, ispirata a ResNet, è composta da un totale di 14 layer ed è strutturata nei seguenti componenti:

- **Layer di ingresso e preprocessing:** un layer convoluzionale iniziale che riceve input monodimensionali ($1 \times 176 \times 208$) e produce un tensore con 32 canali. A questo seguono le operazioni di batch normalization e la funzione di attivazione ReLU, per stabilizzare l'addestramento e introdurre non linearità. Un layer di max pooling riduce ulteriormente la risoluzione spaziale a $32 \times 44 \times 52$.
- **Blocchi residuali (basic blocks):** tre blocchi residuali, ciascuno formato da due layer convoluzionali sequenziali con batch normalization e attivazione ReLU. Le connessioni residuali (*shortcut*) combinano direttamente l'input del blocco con la sua uscita tramite una convoluzione 1×1 e batch normalization, facilitando l'apprendimento di identità e mitigando il degrado delle prestazioni in reti profonde (problema del vanishing gradient).
- **Pooling globale:** un layer di average pooling riduce il tensore di feature a una rappresentazione di dimensione $128 \times 1 \times 1$, condensando l'informazione spaziale.

- **Classificatore fully-connected:** un layer lineare finale mappa le feature a quattro classi target, con l'applicazione di dropout al 25% per regolarizzare la fase di training e ridurre l'overfitting.

Addestramento e ottimizzazione. La rete è stata addestrata utilizzando una suddivisione del dataset in training set e test set con proporzione 80%–20%. Prima dell'addestramento vero e proprio, è stata eseguita una fase di **standardizzazione degli input**: sono state calcolate la media ($\mu \approx 0.2954$) e la deviazione standard ($\sigma \approx 0.3207$) sul training set. Questi valori sono stati utilizzati per normalizzare i dati affinché avessero media nulla e deviazione standard unitaria, facilitando la stabilità numerica del modello.

Statistica	Valore
Media (μ)	0.2954
Deviazione Standard (σ)	0.3207

Tabella A.9: Parametri di normalizzazione calcolati sul training set

L'ottimizzazione è stata affidata all'algoritmo **Adam** (Adaptive Moment Estimation), preferito al classico Stochastic Gradient Descent (SGD) per la sua capacità di ridurre i tempi di convergenza adattando dinamicamente il learning rate. Il modello è stato addestrato per un totale di 20 epoche con un learning rate iniziale fissato a $lr = 10^{-3}$.

L'elevata capacità di generalizzazione osservata è attribuibile a diversi fattori: l'utilizzo di immagini MRI monocanale (intrinsecamente meno complesse rispetto a immagini naturali RGB), la profondità contenuta dell'architettura che limita l'overfitting, e l'impiego combinato di tecniche di regolarizzazione quali dropout e data augmentation, oltre che all'impiego della tecnica di bilanciamento di classe, ovvero il campionamento pesato (WeightedRandomSampler). Tali scelte progettuali hanno permesso al modello di apprendere feature discriminanti specifiche per il dominio clinico considerato.

Risultati. I risultati ottenuti sul test set mostrano un miglioramento significativo rispetto agli approcci basati su feature extractor applicati. In particolare, il modello Custom-CNN raggiunge un'accuratezza complessiva pari al 96% e valori elevati di precision, recall e F1-score per tutte le classi considerate. La corrispondente matrice di confusione evidenzia un numero estremamente ridotto di errori di classificazione, indicando una separazione efficace tra i diversi stadi della patologia.

L'analisi delle mappe di attivazione (Grad-CAM), riportata in Figura 1.4, conferma ulteriormente le capacità del modello: le regioni di attivazione risultano più localizzate e coerenti con le aree cerebrali rilevanti, suggerendo che la rete abbia appreso rappresentazioni semantiche significative e clinicamente plausibili.

Classe	Precision	Recall	F1-score	Support
MildDemented	0.96	0.98	0.97	919
ModerateDemented	1.00	1.00	1.00	1317
NonDemented	0.91	0.98	0.94	2559
VeryMildDemented	0.97	0.88	0.93	2246
Accuracy			0.95	7041
Macro Avg	0.96	0.96	0.96	7041
Weighted Avg	0.95	0.95	0.95	7041

Tabella A.10: Classification report del modello sul test set (Accuracy: 95.19%)

Estrazione delle Feature. La rete convoluzionale, caricata con i pesi ottimali ottenuti nella fase precedente, è stata modificata strutturalmente rimuovendo l'ultimo layer completamente connesso (il livello di classificazione originario). Tale operazione è stata implementata sostituendo il layer lineare finale con una mappatura identità ($f(x) = x$). In questa configurazione, il modello non produce più una distribuzione di probabilità sulle classi, bensì un vettore di feature latenti $\mathbf{z} \in \mathbb{R}^d$ (corrispondente all'output del penultimo layer) per ogni immagine di input normalizzata. Tali vettori costituiscono una rappresentazione compatta e ad alto contenuto semantico dei dati MRI.

Classificazione con MLP. Le feature estratte sono state utilizzate come input per addestrare un classificatore dedicato, basato su un Multilayer Perceptron (MLP). L'architettura del classificatore consiste in:

- Un **layer di input** dimensionato sulla grandezza del vettore di feature estratto dalla CNN;
- Un **hidden layer** composto da 64 unità (neuroni), volto a proiettare le feature in uno spazio decisionale ottimizzato;
- Un **output layer** finale con funzione di attivazione **Softmax**, che restituisce la probabilità di appartenenza alle quattro classi target.

Per mitigare l'effetto dello sbilanciamento delle classi presente nel dataset, l'addestramento dell'MLP ha integrato l'uso di pesi correttivi (*class weights*) all'interno della funzione di costo, calcolati in modo inversamente proporzionale alla frequenza di ciascuna classe nel training set.

Classe	Precision	Recall	F1-score	Support
MildDemented	0.96	0.98	0.97	919
ModerateDemented	1.00	1.00	1.00	1317
NonDemented	0.96	0.95	0.95	2559
VeryMildDemented	0.95	0.95	0.95	2246
Accuracy			0.96	7041
Macro Avg	0.97	0.97	0.97	7041
Weighted Avg	0.96	0.96	0.96	7041

Tabella A.11: Classification report del modello finale (Accuracy: 96.24%).

True / Pred	Mild	Moderate	Non	Very Mild
MildDemented	904	0	5	10
ModerateDemented	0	1317	0	0
NonDemented	17	1	2428	113
VeryMildDemented	23	0	96	2127

Tabella A.12: Confusion matrix del modello finale.

Classificazione con SVM. Al fine di validare ulteriormente la qualità delle rappresentazioni apprese dalla Custom-CNN e massimizzare l'esplorazione dello spazio delle feature, è stato

implementato un classificatore basato su Support Vector Machine (SVM). L'obiettivo di questo esperimento era verificare se un approccio a margine massimo potesse offrire una generalizzazione superiore o confermare la robustezza delle feature estratte, indipendentemente dal classificatore utilizzato.

Il workflow sperimentale ha previsto i seguenti step:

- **Preprocessing:** I vettori delle feature ($d = 128$), estratti dal penultimo layer della Custom-CNN, sono stati sottoposti a *Standard Scaling* per ottenere media nulla e varianza unitaria. Questo passaggio è critico per le SVM, in quanto l'algoritmo è sensibile alla scala delle distanze nello spazio delle feature.
- **Ottimizzazione:** È stata condotta una *Grid Search* con *5-fold Cross-Validation* per individuare la combinazione ottimale di iperparametri. Lo spazio di ricerca ha esplorato diversi valori per il fattore di regolarizzazione C , il coefficiente del kernel γ e la tipologia di kernel (Lineare, Polinomiale, RBF).
- **Gestione dello sbilanciamento:** Per contrastare la disparità numerica tra le classi, i pesi della funzione di costo sono stati bilanciati automaticamente in modo inversamente proporzionale alla frequenza delle classi nel training set (`class_weight='balanced'`).

La configurazione ottimale individuata è risultata essere un kernel a base radiale (**RBF**) con $C = 500$ e $\gamma = 10^{-2}$. L'alto valore di C suggerisce che il modello penalizza fortemente gli errori di classificazione sul training set, affidandosi alla buona separabilità delle feature.

Classe	Precision	Recall	F1-score	Support
MildDemented	0.98	0.99	0.98	919
ModerateDemented	1.00	1.00	1.00	1317
NonDemented	0.98	0.97	0.98	2559
VeryMildDemented	0.96	0.97	0.97	2246
Accuracy			0.98	7041
Macro Avg	0.98	0.98	0.98	7041
Weighted Avg	0.98	0.98	0.98	7041

Tabella A.13: Classification report SVM (Accuracy: 97.95%).

True / Pred	Mild	Moderate	Non	Very Mild
MildDemented	906	0	2	11
ModerateDemented	0	1317	0	0
NonDemented	6	0	2484	69
VeryMildDemented	10	0	47	2189

Tabella A.14: Confusion matrix SVM.

I risultati ottenuti sul test set (Accuracy $\approx 98\%$) sono comparabili, se non leggermente superiori, a quelli ottenuti con l'MLP. Questo raggiungimento di un "plateau" nelle prestazioni suggerisce che il collo di bottiglia non risiede più nella capacità del classificatore, bensì nella qualità intrinseca delle feature estratte dalla Custom-CNN, che si confermano essere altamente discriminanti e rappresentative per il task di diagnosi dell'Alzheimer.

Inoltre, la visualizzazione tramite PCA del confine di decisione (Figura A.5) mostra come il kernel RBF riesca a definire regioni di separazione non lineari complesse, gestendo efficacemente le aree di sovrapposizione tra le classi *NonDemented* e *VeryMildDemented*.

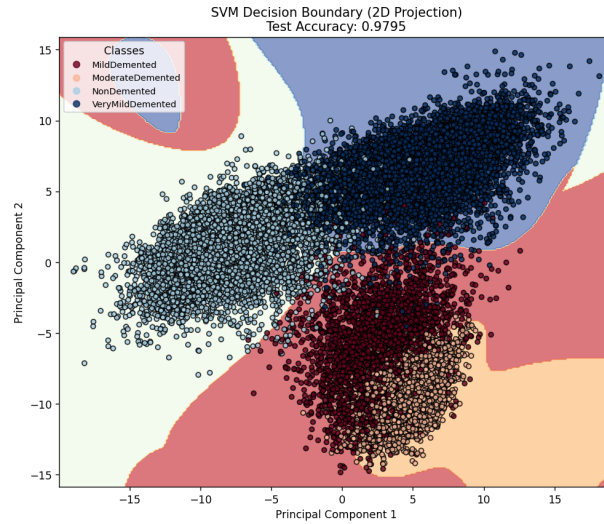


Figura A.5: Proiezione 2D tramite PCA del decision boundary appreso dalla SVM. Si noti la capacità del modello di separare i cluster delle classi nonostante la riduzione dimensionale.

A.4 Valutazione

Per una valutazione a $K = 4$ classi ($c \in \{1, 2, 3, 4\}$), sia C la matrice di confusione 4×4 , dove $C_{i,j}$ rappresenta il numero di campioni con classe i che sono stati classificati come classe j . Definiamo:

$$\begin{aligned}
 TP_k &= C_{k,k} \\
 FP_k &= \sum_{i=1, i \neq k}^4 C_{i,k} \\
 FN_k &= \sum_{i=1, i \neq k}^4 C_{k,i} \\
 TN_k &= \sum_{i=1}^4 \sum_{j=1}^4 C_{i,j} - TP_k - FP_k - FN_k
 \end{aligned}$$

Le metriche utilizzate per valutare le performance del modello sono:

- **Precision:** misura la proporzione dei veri positivi rispetto al totale dei positivi predetti dal modello.

$$\text{precision}_k = \frac{TP_k}{TP_k + FP_k}$$

- **Recall:** misura la proporzione dei veri positivi rispetto al totale dei positivi effettivi nel dataset.

$$\text{recall}_k = \frac{TP_k}{TP_k + FN_k}$$

- **F1-score:** è la media armonica tra precision e recall, fornendo una misura bilanciata delle

performance del modello.

$$\text{F1-score}_k = 2 \cdot \frac{\text{precision}_k \cdot \text{recall}_k}{\text{precision}_k + \text{recall}_k}$$

- **Support:** il numero di occorrenze di ogni classe nel dataset di test.

$$\text{support}_k = \sum_{i=1}^4 C_{k,i}$$

A.5 Demo

A.5.1 Obiettivo della demo

La demo è stata realizzata con l'obiettivo di fornire una dimostrazione interattiva del funzionamento degli algoritmi di classificazione sviluppati. Essa consente di verificare visivamente la capacità del sistema di prendere in input immagini MRI, processarle tramite modelli addestrati e restituire una predizione coerente, eventualmente accompagnata da informazioni di interpretabilità.

A.5.2 Tecnologie utilizzate

La demo è stata sviluppata utilizzando Streamlit, framework Python per la realizzazione rapida di interfacce web interattive. L'inferenza viene eseguita tramite modelli PyTorch precedentemente addestrati e salvati durante la fase di training.

A.5.3 Funzionalità principali

La demo offre le seguenti funzionalità:

- caricamento di immagini MRI tramite interfaccia grafica;
- visualizzazione dell'immagine di input;
- inferenza tramite modelli di classificazione addestrati;
- visualizzazione dell'etichetta predetta dal modello;
- visualizzazione dell'etichetta di ground truth (quando disponibile);
- generazione opzionale di una heatmap di attivazione per l'analisi delle regioni rilevanti dell'immagine.

A.5.4 Modalità di utilizzo

La demo interattiva può essere avviata dalla root del progetto tramite il seguente comando:

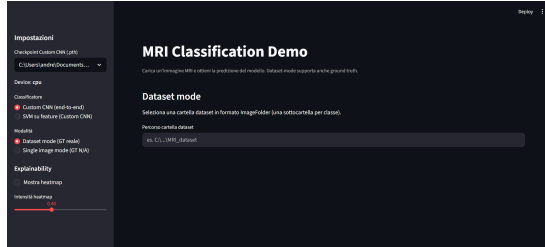
```
streamlit run src/demo/demo.py
```

La demo supporta due modalità operative principali.

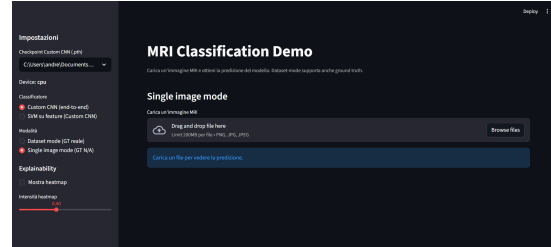
Dataset mode In questa modalità l'utente può selezionare immagini MRI provenienti da un dataset organizzato in formato `ImageFolder`.

Single image mode In questa modalità è possibile caricare manualmente una singola immagine MRI tramite file system.

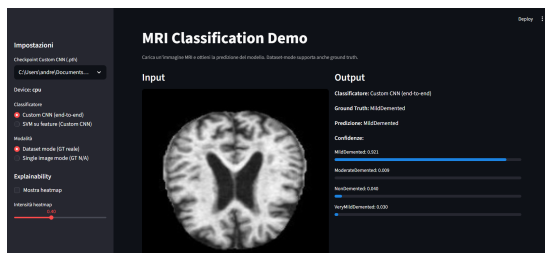
Immagini Interfaccia Di seguito, riportiamo delle immagini rappresentative della demo.



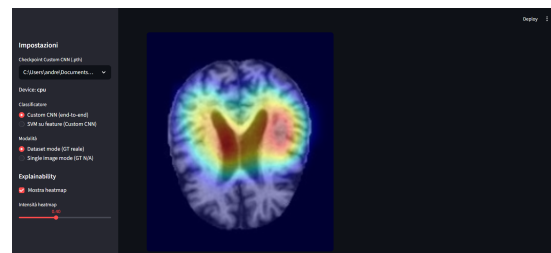
(a) Schermata iniziale



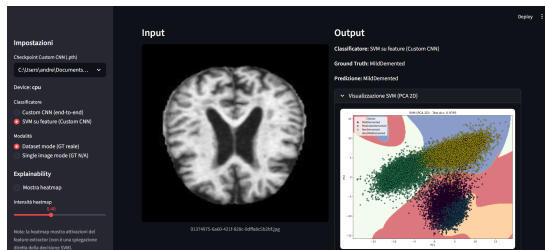
(b) Caricamento singola immagine



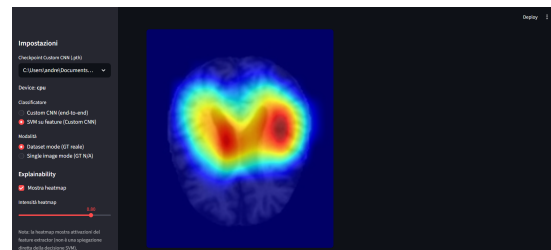
(c) Modello Custom_CNN



(d) Heatmap relativa alla Custom_CNN



(e) Modello SVM con visualizzazione del decision boundary



(f) Visualizzazione Heatmap con intensità dei colori aumentata

Figura A.6: Panoramica delle funzionalità dell'interfaccia della Demo.

A.6 Codice

Il codice è organizzato in modo modulare per separare le diverse fasi del workflow di machine learning. La directory **src/** contiene i moduli principali per il preprocessing dei dati, l'estrazione delle feature, l'addestramento dei modelli e la valutazione delle prestazioni. I notebook presenti nella cartella **notebooks/** sono utilizzati per l'analisi esplorativa dei dati e per l'esecuzione controllata degli esperimenti. I risultati degli esperimenti, incluse le feature estratte, i grafici e altri artefatti, sono salvati nella directory **results/**. La documentazione del progetto, infine, è salvata nella directory **docs/**.

Per eseguire il codice è sufficiente clonare la repository, installare le dipendenze indicate e lanciare gli script. Tutti i dettagli aggiornati su struttura del progetto, requisiti software ed esecuzione sono disponibili nella repository GitHub del progetto.

A.7 Conclusioni

In questo lavoro è stato affrontato il problema della classificazione di immagini di risonanza magnetica cerebrale per la distinzione tra diversi livelli di severità della demenza. Sono stati analizzati e confrontati differenti approcci di classificazione, basati sia sull'utilizzo diretto delle immagini sia sull'estrazione di feature, valutando le prestazioni dei modelli su dati non visti.

Le principali lezioni apprese riguardano l'utilizzo di modelli per la feature extraction. Modelli troppo complessi o addestrati su dati eccessivamente granulari possono faticare a generalizzare. Inoltre, l'importanza della qualità del preprocessing e della rappresentazione dei dati influisce in modo significativo le prestazioni dei modelli di classificazione.

Dati i risultati validi ottenuti tramite il feature extractor **Custom_CNN**(da noi implementato), riteniamo che una delle principali sfide, una feature extraction adeguata al tipo di dati, affrontate in questo progetto, sia stata risolta.

Ai fini di migliorare ulteriormente il progetto si potrebbe:

Fare un controllo del Leakage. - Una preoccupazione importante in progetti di diagnostica medica è assicurarsi che il modello non stia usando dati ridondanti tra training set e test set, sfruttando informazioni spurie. Nel dataset usato è possibile che ci siano più immagini della stessa persona o comunque immagini correlate. Il codice attuale effettua uno split casuale per immagini; questo implica che diverse scansioni dello stesso paziente potrebbero finire una nel train e una nel test (se ci dovessero essere ripetizioni). In tal caso, il modello potrebbe semplicemente memorizzare caratteristiche individuali (es. conformazione cranica, artefatti di scansione) e riconoscere un paziente nel test set perché già "visto" in training, ottenendo performance sovrastimate senza realmente generalizzare a nuovi pazienti.

Aggiungere interpretabilità. - Essendo un modello medico, la prima preoccupazione è capire "come i dati vengono classificati". In questo momento nessuno dei modelli provati riesce a spiegare come ha classificato un'immagine, essendo modelli black-box. Una strada potrebbe essere quella di utilizzare delle Grad-CAM++ (come citato in [2]) o altri metodi con un eventuale esperto di dominio.

Test su nuovi dati. - Si potrebbe pensare di testare il modello su dati mai visti, che sono fuori dal dominio di quelli utilizzati per il training. Questo è utile per affermare la correttezza del modello su dati mai visti.

Parte IV

Materiale Aggiuntivo

Appendice B

Approfondimenti teorici

B.1 Problema della separabilità non lineare nel perceptrone

Questa parte è tratta dal capitolo 2, dove si parla del perceptrone nella sezione 2.3.

Definizione (separabilità lineare). Sia $X = \{x^{(1)}, \dots, x^{(m)}\} \subset \mathbb{R}^d$ e sia $y \in \{0, 1\}^m$ un insieme di etichette. Diciamo che i due insiemi di punti $A = \{x^{(i)} : y^{(i)} = 1\}$ e $B = \{x^{(i)} : y^{(i)} = 0\}$ sono *linearmente separabili* se esistono $w \in \mathbb{R}^d$ e $b \in \mathbb{R}$ tali che

$$w^\top x > b \quad \forall x \in A, \quad w^\top x < b \quad \forall x \in B.$$

Equivalente: esiste un iperpiano $w^\top x = b$ la cui parte positiva contiene tutti i punti di A e la parte negativa tutti i punti di B .

Quante funzioni booleane sono linearmente separabili? Per m argomenti binari esistono 2^{2^m} funzioni booleane possibili; solo una parte è realizzabile con un singolo classificatore lineare. Dati noti:

$$\begin{aligned} m = 2 &\Rightarrow 14 \text{ su } 16 \text{ sono separabili linearmente,} \\ m = 3 &\Rightarrow 104 \text{ su } 256 \text{ sono separabili linearmente,} \\ m = 4 &\Rightarrow 1882 \text{ su } 65536 \text{ sono separabili linearmente.} \end{aligned}$$

Non esiste una formula chiusa semplice che dia, in funzione di m , quante funzioni sono linearmente separabili; tuttavia si osserva che, all'aumentare di m , la *frazione* di funzioni separabili decresce rapidamente.

Dicotomie di un insieme di punti. Dato $X = \{x^{(1)}, \dots, x^{(m)}\} \subset \mathbb{R}^d$, le possibili etichettature (o *dicotomie*) sono 2^m :

$$\mathcal{Y} = \{0, 1\}^m.$$

Solo una parte di queste dicotomie è realizzabile mediante una frontiera lineare. Con d fissata, il numero di dicotomie realizzabili cresce in modo polinomiale in m (ordine al più m^d), mentre il totale cresce esponenzialmente (2^m); di conseguenza,

$$\Pr\{\text{dicotomia separabile linearmente}\} \longrightarrow 0 \quad \text{quando } m \gg d.$$

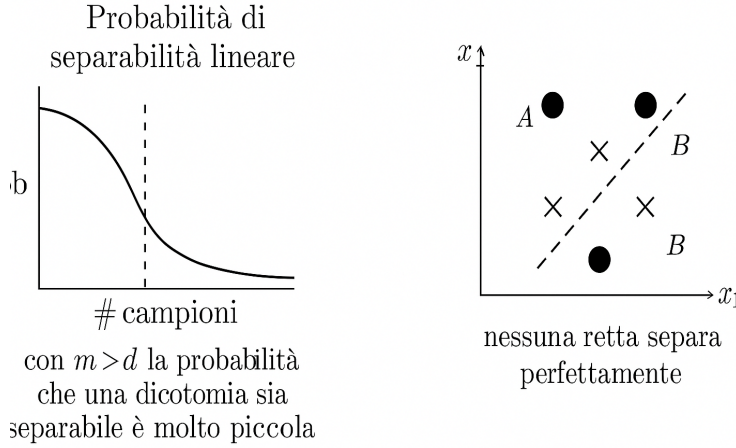


Figura B.1: Separabilità non lineare. A sinistra: andamento qualitativo della probabilità che una dicotomia sia linearmente separabile al crescere del numero di campioni m (con dimensione d fissata); a destra: esempio in $\mathbb{R}^2$ di punti non separabili con un singolo iperpiano.

Intuitivamente: se il numero di campioni cresce molto a parità di dimensione d , è sempre meno probabile che una separazione perfetta con una sola retta/iperpiano esista.

B.2 Generalizzazione funzione di costo del softmax

Questa parte è tratta dal capitolo 4 nella sezione 4.9.5.

Il modello softmax è una generalizzazione della regressione logistica multiclasse, si può infatti dimostrare che la funzione di costo del softmax riduce a quella della regressione logistica binaria per $K = 2$ classi. Ricordiamo che la funzione di costo del softmax è:

$$J(\vartheta) = -\frac{1}{m} \sum_{i=1}^m \sum_{c=0}^{K-1} \mathbf{1}\{y^{(i)} = c\} \log(h_{\vartheta}^{(c)}(x^{(i)})) + \frac{\lambda}{2} \sum_{c=0}^{K-1} \sum_{j=1}^n (\vartheta_j^{(c)})^2,$$

È facile vedere che, per $K = 2$ (senza regolarizzazione) vale:

$$\begin{aligned} J(\vartheta) &= -\frac{1}{m} \sum_{i=1}^m \sum_{c=0}^1 \mathbf{1}\{y^{(i)} = c\} \log(h_{\vartheta}^{(c)}(x^{(i)})) \\ &= -\frac{1}{m} \sum_{i=1}^m \left[\mathbf{1}\{y^{(i)} = 0\} \log h_{\vartheta}^{(0)}(x^{(i)}) + \mathbf{1}\{y^{(i)} = 1\} \log h_{\vartheta}^{(1)}(x^{(i)}) \right] \\ &= -\frac{1}{m} \sum_{i=1}^m \left[(1 - y^{(i)}) \log h_{\vartheta}^{(0)}(x^{(i)}) + y^{(i)} \log h_{\vartheta}^{(1)}(x^{(i)}) \right] \\ &= -\frac{1}{m} \sum_{i=1}^m \left[(1 - y^{(i)}) \log(1 - h_{\vartheta}^{(1)}(x^{(i)})) + y^{(i)} \log h_{\vartheta}^{(1)}(x^{(i)}) \right], \end{aligned}$$

dove si è usata la normalizzazione della softmax $h_{\vartheta}^{(0)}(x) + h_{\vartheta}^{(1)}(x) = 1$. Questa è esattamente la loss della regressione logistica binaria: dunque la regressione logistica è un caso particolare della softmax.

Si può ulteriormente dimostrare che il softmax è una generalizzazione della regressione logistica multiclasse usando una sua proprietà principale: l'*overparametrizzazione*. Per $K = 2$:

$$h_{\vartheta}(x) = \begin{bmatrix} \frac{e^{\vartheta^{(0)\top} x}}{e^{\vartheta^{(0)\top} x} + e^{\vartheta^{(1)\top} x}} \\ \frac{e^{\vartheta^{(1)\top} x}}{e^{\vartheta^{(0)\top} x} + e^{\vartheta^{(1)\top} x}} \end{bmatrix} = \begin{bmatrix} \frac{e^{(\vartheta^{(0)} - \vartheta^{(1)})^\top x}}{e^{(\vartheta^{(0)} - \vartheta^{(1)})^\top x} + e^{\mathbf{0}^\top x}} \\ \frac{e^{(\vartheta^{(1)} - \vartheta^{(0)})^\top x}}{e^{(\vartheta^{(0)} - \vartheta^{(1)})^\top x} + e^{\mathbf{0}^\top x}} \end{bmatrix} = \begin{bmatrix} \frac{e^{\Delta^\top x}}{1 + e^{\Delta^\top x}} \\ \frac{1}{1 + e^{\Delta^\top x}} \end{bmatrix}, \quad \Delta = \vartheta^{(0)} - \vartheta^{(1)}.$$

Ponendo $\mathbf{w} = \vartheta^{(1)} - \vartheta^{(0)} = -\Delta$ e $\sigma(z) = \frac{1}{1 + e^{-z}}$ si ottiene

$$h_{\vartheta}(x) = \begin{bmatrix} 1 - \sigma(\mathbf{w}^\top x) \\ \sigma(\mathbf{w}^\top x) \end{bmatrix}.$$

Quindi, per $K = 2$, il softmax riduce alla regressione logistica binaria.

B.3 Dimostrazione del teorema del margine a 1

Questa parte è tratta dal capitolo 9 nella sezione 9.2.2.

Imponiamo $\|w\| = 1$. Questo può essere scritto come $w = \frac{w'}{\|w'\|}$, dove w' è un vettore di parametri qualsiasi non normalizzato. Sostituendo nella formulazione iniziale, otteniamo

$$y \left(\left\langle \frac{w'}{\|w'\|}, x \right\rangle + b \right) \geq r$$

Noto che $r \geq 0$ in quanto una distanza euclidea, possiamo andare a dividere l'intero vincolo per r . Otteniamo:

$$y \left(\left\langle \frac{w'}{\|w'\| r}, x \right\rangle + \underbrace{\frac{b}{r}}_{b''} \right) \geq 1$$

Bene, adesso definiamo $\|w''\| = \left\| \frac{w'}{\|w'\| r} \right\| = \frac{1}{r} \frac{\|w'\|}{\|w'\|} = \frac{1}{r}$, $\Rightarrow r = \frac{1}{\|w''\|}$.

Per facilitare la dimostrazione, imponiamo di dover massimizzare r^2 , ossia $\frac{1}{\|w''\|^2}$. Tuttavia, massimizzare questo valore, significa minimizzare $\|w''\|^2$, o, per semplificare il lavorare con le derivate, equivale a minimizzare $\frac{1}{2} \|w''\|^2$.

$$\min_{w'', b''} \frac{1}{2} \|w''\|^2 \quad y(\langle w'', x \rangle + b'') \geq 1$$

Data questa totale equivalenza, abbiamo dimostro il teorema.

B.4 Dual SVM

Questa parte è tratta dal capitolo 9 nella sezione 9.4.

La formulazione di SVM descritta in precedenza, è nota come primal SVM. Ha variabili w e b , e w è di dimensione pari a $x \in \mathbb{R}^D$, con D features. Ne consegue che il problema di ottimizzazione cresce linearmente all'aumentare del numero di features. La seguente formulazione, crescerà invece col numero di esempi del dataset, non dipendendo più dal numero di features. Sarà inoltre semplice applicarne dei kernel, per separare meglio dati non linearmente separabili. Chiaremo questa formulazione **Dual SVM**, e si presenterà così:

$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N y_i y_j \alpha_i \alpha_j \langle x_i, x_j \rangle - \sum_{i=1}^N \alpha_i \\ \text{soggetto a} \quad & \sum_{i=1}^N y_i \alpha_i = 0, \\ & 0 \leq \alpha_i \leq C \quad \text{per } i = 1, \dots, N. \end{aligned}$$

Prerequisito matematico: Lagrangiani Data una funzione f da ottimizzare, e un insieme di condizioni $g_1, \dots, g_n$, un **Lagrangiano** è una funzione $\mathcal{L}(f, g_1, \dots, g_n, \alpha_1, \dots, \alpha_n)$ che incorpora le condizioni del problema di ottimizzazione:

$$\mathcal{L}(f, \alpha_1, \dots, \alpha_n) = f(x_1, \dots, x_k) - \sum_{i=1}^n \alpha_i g_i(x_1, \dots, x_k)$$

Ad esempio, se $f(w) = \frac{w^2}{2}$ e $g_1(w, x) : wx - 1 \geq 0$, allora

$$\mathcal{L}(w, \alpha) = \frac{w^2}{2} - \alpha(wx - 1), \quad \alpha \geq 0$$

Si calcolano le derivate parziali del Lagrangiano rispetto alle variabili del problema, in questo esempio w e α , e si impongono nulle. Si ottiene così un sistema di equazioni le cui soluzioni soddisfano le condizioni di ottimalità (condizioni di stazionarietà).

Il Lagrangiano del problema è

$$\mathcal{L}(w, \alpha) = \frac{1}{2}w^2 - \alpha(wx - 1), \quad \alpha \geq 0.$$

Calcoliamo le derivate parziali.

Derivando rispetto a w :

$$\frac{\partial \mathcal{L}}{\partial w} = w - \alpha x.$$

Imponendo la condizione di stazionarietà otteniamo

$$w - \alpha x = 0 \quad \Rightarrow \quad w = \alpha x.$$

Derivando rispetto a α :

$$\frac{\partial \mathcal{L}}{\partial \alpha} = -(wx - 1).$$

Imponendo la derivata nulla:

$$wx - 1 = 0.$$

B.4.1 Convex duality con Moltiplicatori di Lagrange

Riprendiamo le variabili della formulazione primale: w, b, ξ . Introduciamo i moltiplicatori di Lagrange $\alpha_n \geq 0$ associati ai vincoli

$$y_n(\langle w, x_n \rangle + b) \geq 1 - \xi_n,$$

e i moltiplicatori $\gamma_n \geq 0$ associati ai vincoli $\xi_n \geq 0$.

Il Lagrangiano è

$$\begin{aligned} \mathcal{L}(w, b, \xi, \alpha, \gamma) = & \frac{1}{2} \|w\|^2 + C \sum_{n=1}^N \xi_n \\ & - \sum_{n=1}^N \alpha_n (y_n(\langle w, x_n \rangle + b) - 1 + \xi_n) - \sum_{n=1}^N \gamma_n \xi_n. \end{aligned}$$

Deriviamo rispetto alle variabili primali.

$$\frac{\partial \mathcal{L}}{\partial w} = w - \sum_{n=1}^N \alpha_n y_n x_n,$$

$$\frac{\partial \mathcal{L}}{\partial b} = - \sum_{n=1}^N \alpha_n y_n,$$

$$\frac{\partial \mathcal{L}}{\partial \xi_n} = C - \alpha_n - \gamma_n.$$

Imponendo le condizioni di stazionarietà otteniamo

$$w = \sum_{n=1}^N \alpha_n y_n x_n,$$

$$\sum_{n=1}^N \alpha_n y_n = 0,$$

$$C - \alpha_n - \gamma_n = 0.$$

Dalla terza equazione segue che

$$0 \leq \alpha_n \leq C.$$

Sostituendo l'espressione di w nel Lagrangiano ed eliminando ξ_n e γ_n tramite le condizioni di stazionarietà, otteniamo il problema duale.

Il duale consiste nel massimizzare rispetto agli α_n :

$$\begin{aligned} \max_{\alpha} \quad & \sum_{n=1}^N \alpha_n - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle \\ \text{soggetto a} \quad & \sum_{n=1}^N \alpha_n y_n = 0, \\ & 0 \leq \alpha_n \leq C. \end{aligned}$$

Box Constraints Indichiamo con questo nome l'insieme di disuguaglianze $0 \leq \alpha_i \leq C$ per $i = 1, \dots, N$. Questi, limitano il vettore dei moltiplicatori di Lagrange $\alpha = [\alpha_1, \dots, \alpha_N]^\top \in \mathbb{R}^N$ in una "scatola" definita tra 0 e C per ogni asse.

Dal Duale al Primale Una volta trovati i parametri α , possiamo ottenere i parametri w usando il **representer theorem**.

$$w = \sum_{n=1}^N \alpha_n y_n x_n$$

e indicheremo il suo valore ottimale con w^* . Per determinare b^* , utilizziamo la condizione di margine valida per i support vector tali che $0 < \alpha_n < C$. Per questi punti vale infatti

$$y_n(\langle w^*, x_n \rangle + b^*) = 1.$$

Da cui possiamo ricavare

$$b^* = y_n - \langle w^*, x_n \rangle.$$

Appendice C

Seminario 1: *Data and AI Law*

Tenuto dal Co-founder e CEO di ICC Digital Media 1999, Giuseppe Fragola, in data 28 Novembre 2025.

C.1 L'importanza dei dati - segreti industriali

Qualsiasi rivoluzione tecnologica ha a che fare con i dati. I dati sono il comune denominatore di ogni settore. I dati non sono da vedersi solo come dati del singolo, ma anche informazioni strategiche e segreti industriali, tra cui:

- Formule
- Organizzazione
- Progetti
- Produzione
- Fornitori e prezzi
- Tecnologie in uso

A tutela dei dati relativi ai segreti industriali, nasce un ampio corpus informativo.

C.1.1 Data Is The New Oil

I dati hanno una valenza strategica enorme. Le organizzazioni criminali, tramite attacchi hacker, puntano a ottenere informazioni preziose per fare speculazione economica. Tra questi dati, ci sono **brevetti, bilanci, fornitori e contratti**. Inoltre, visto l'inevitabile crescita della quantità di dati, i crimini informatici relativi al loro ottenimento illecito, sono destinati ad aumentare nel tempo.

C.1.2 Mission Genesis - 25 Novembre 2025

Donald Trump ha recentemente firmato un ordine esecutivo per condividere i dati scientifici di tutte le agenzie governative e utilizzare l'AI (creando un *data lake*¹) per accelerare ricerca, sicurezza nazionale e la razionalizzazione la rete elettrica nazionale.

¹Un data lake è un repository centralizzato che archivia grandi quantità di dati strutturati, semi-strutturati e non strutturati nel loro formato nativo. A differenza di un data warehouse, i dati vengono archiviati senza una struttura predefinita, il che consente una maggiore flessibilità per diverse analisi, tra cui Big Data, machine learning e analisi in tempo reale.

C.1.3 Norme: Data Protection e Copyright

Lavorare con i dati, costringe qualsiasi operatore coinvolto a confrontarsi con le norme relative alla data protection e al diritto d'autore.

C.2 Data Protection

Iniziamo **distinguendo privacy** dalla **data protection**. La privacy è infatti il diritto alla riservatezza nella propria sfera privata da parte di terzi (*Right to be alone*). La data protection fa invece riferimento al **trattamento dei dati personali solo con previa autorizzazione dell'interessato**. I dati possono essere trattati solo col consenso di chi li produce.

Cos'è un dato personale? Una qualsiasi **informazione** che direttamente o indirettamente consente di **identificare una persona fisica**, o le sue abitudini, lo stile di vita, le relazioni personali, lo stato di salute o lo stato economico.

C.2.1 GDPR - La rivoluzione

L'Europa decide di tutelare i propri cittadini, con obiettivo ultimo una bolla di protezione relativa ai dati personali. Nasce il regolamento GDPR. Questo, si applica a:

- Coloro che avendo sede EU trattano dati di soggetti interessati **a prescindere dalla loro nazionalità o residenza**.
- Coloro non EU che per la vendita di beni o servizi, **trattano dati personali** di soggetti interessati **che si trovano in EU**.

Gli spunti sono i seguenti:

- **Privacy by Design**: ogni applicazione che tratta dati degli utenti, deve nascere privacy-oriented, concepito e sviluppato avendo in mente la loro tutela.
- **Privacy by Default**: tutte le opzioni per rispettare la privacy devono essere di default.
- **Consent**
- **Portabilità**
- **Data Breach Notification**, obbligata dopo qualsiasi data breach. La norma non dà degli obblighi di sicurezza, ma sei responsabile di qualsiasi conseguenza.

ma tra i quelli più rivoluzionari, abbiamo anche:

- **Minimizzazione**: ogni azienda deve acquisire il minimo indispensabile dei dati rispetto alla finalità per le quali sono trattati. Registrarsi una newsletter dovrebbe consentire solo all'acquisizione dell'e-mail, non del numero di telefono o altri dati sensibili. Questo implica anche la possibilità, da parte dell'utente, di eliminare account personali su piattaforme online.
- **Informativa e Consenso granulare**: il consenso deve essere rilasciato specificamente per ciascuna delle finalità di trattamento dichiarate, **una ad una**.
- **Temporaneità**: i dati devono avere tempo di vita limitato, non possono

Le sanzioni previste arrivano fino al 4% del **fatturato del consolidato mondiale del gruppo**.

C.2.2 Sui Cookies

Conosciamo già i cookies: questi permettono di tracciare informazioni sul browser dell'utente (cookies di sessione e persistenti). Tra questi, abbiamo cookies di prime parti e di terze parti.

- I cookies di prime parti sono generati e leggibili solo dal dominio chi li ha creati
- I cookies di terze parti sono creati da domini esterni a quello che si sta visitando. Questo permette facilmente di tracciare informazioni sulle abitudini di un utente, ottenendo dati (o almeno, del suo browser).

Direttiva del 2009 sui Cookies L'UE introduce la direttiva nota come "Cookies Directive" per regolamentare l'uso dei cookie online. L'obiettivo della norma è quello di **garantire informazione chiara** agli utenti e **regolamentare l'uso dei cookie** attraverso **obbligo di informativa e invio solo previo consenso**.

C.3 Copyright e diritto d'autore

Disambighiamo i termini.

- Il **copyright** è il diritto di copia. Io che ho creato un contenuto, ho diritto ad una *fee*, un rimborso, ogni qual volta in cui qualcuno me ne richieda una copia. Dura all'incirca 50 anni.
- Il **diritto d'autore** tutela l'autore e la paternità dell'opera, e si acquisisce con la creazione dell'opera stessa. Dura 70 anni dalla morte dell'autore.

C.3.1 Diritto d'autore

Riconosce dei **diritti morali indisponibili**. Tra questi abbiamo che sono **indisponibili**, sono riconosciuti a priori. Si ha il diritto a **paternità, integrità, inedito, pentimento**. I diritti patrimoniali sono invece disponibili, e sono diritto di **riproduzione, diffusione, modifica e traduzione**.

Nonostante la durata fissata sia 70 anni dalla morte dell'autore, esistono delle durate particolari sono quelle di opere a **contributo indistinguibile e opere collettive**.

C.3.2 Alla tutela del Software

Dopo anni di incertezza normativa, il software è stato inserito tra le **opere dell'ingegno** tutelate dalla legge sul **diritto d'autore**. la tutela riguarda esclusivamente la forma espressiva originale del software (codice), ma non l'idea, che può essere riprodotta mediante un codice diverso. Allo sviluppatore spettano:

- **Diritti morali** - paternità indisponibile
- **Diritti patrimoniali** - riproduzione, traduzione, adattamento, trasformazione, distribuzione

questi, sono però cedibili (a tempo determinato o indeterminato) mediante opportune licenze d'uso. *Consiglio: non perdere mai le licenze ai software! Tieni sempre una cartella con le licenze.*

C.3.2.1 Comprare il software o una licenza?

Ogni qual volta che "compriamo" un software, in realtà noi stiamo acquistando una licenza (temporanea o non). Comprare un software, in senso stretto, significherebbe acquisirne il sorgente.

C.3.2.2 Principio d'esaurimento

Ogni volta che un bene viene immesso nel mercato, chi li immette nel mercato non ha più controllo delle eventuali rivendite e della libera circolazione.

C.4 AI e Copyright

Tra i punti più caldi relativi all'intelligenza artificiale e il Copyright, abbiamo:

- AI e creazione di opere nuove.
- AI e creazione di opere derivate.
- AI e i dati d'addestramento.

A chi appartengono i diritti d'autore in questi casi?

C.4.0.1 Parentesi sulle AI generative

L'AI generativa è una categoria di AI che permette la generazione di testi, immagini, video e codice grazie a modelli di machine learning supervisionati, reinforcement learning e LLM addestrati su quantità di dati sempre crescenti. Nel 2022 avviene la prima generazione di contenuti creativi nuovi a partire da un algoritmo. **Le opere generate da AI possono essere tutelate da diritto d'autore?** Se presentano originalità e creatività al pari di quella umana, questa potrebbe essere tutelata.

C.4.1 Posizioni dell'US Copyright Office

- 2023 - Rigetto della richiesta di copuright su un'opera creata da AI. *Non sussiste il necessario nesso tra autore umano ed espressione creativa.*
- 2024 - conferma e chiarimento. *È esclusa ogni tutela per opere create dalla macchina in assenza di una prevalente creatività umana.*
- Nodo aperto: *un nodo abbastanza dettagliato è un apporto creativo prevalentemente umano?* Il tribunale cinese conferma i diritti riconosciuti all'umano, per gli US la questione rimane aperta.

C.4.2 In Italia

Se prevale l'apporto creativo umano, l'opera resta un'opera dell'ingegno e gode della piena tutela del diritto d'autore e delle categorie IPR tradizionali, attribuite alla persona fisica. **Trattasi di un'opera dell'ingegno solo se prevale la creatività umana.** Un'enorme area grigia giuridica.

C.4.3 *Un primo passo - In Sud Africa*

Il Sud Africa cambia la storia riconoscendo il **primo brevetto per una invenzione autonomamente creata da una AI**.

C.4.4 **AI e soggettività giuridica**

Riconoscere una soggettività giuridica a un'AI risolverebbe problemi di attribuzione del diritto e della responsabilità per danno, ma aprirebbe scenari economici e sociali imprevedibili. Il timore riguarda **AI senzienti e dotate di soggettività**, e il rischio che queste possano diventare **entità potentissime** e ricchissime.

C.4.5 **Caso OpenAI**

OpenAI attribuisce agli utenti i diritti sulle opere create, riservandosi il diritto di usare l'immagine generata per il training.

C.4.5.1 **In assenza di disciplina pattizia**

Se l'utente ha fornito un prompt articolato con dati molto specifici e qualificati, l'umano prevale. Altrimenti, il diritto è da attribuirsi agli sviluppatori dell'AI.

C.5 **AI e opere derivate**

Chi è il responsabile se un'opera realizzata da AI plagia un'opera altrui?

Si rimane in attesa di una nuova proposta normativa. Si è valutato di attribuire la responsabilità agli sviluppatori, ma è ancora una questione aperta (e attualmente sospesa).

C.6 **AIG e addestramento**

I dataset contengono dati appartenenti a qualcuno: anche qui dovrebbe subentrare il diritto d'autore. Incontriamo una serie di problemi. Tra queste, le pubblicazioni e i contenuti scritti pre-AI generativa non hanno valutato come gestire un arrivo del genere. Le principali criticità legali sono

- **Violazione del diritto d'autore** per i dati usati nel training.
- **I dataset pubblici non forniscono indicazioni sull'uso relativo all'AI.**
- **Opere derivate.** Opere troppo simili potrebbero considerarsi plagio.

Si conclude che, in molti casi, il problema del DDA non è l'AI, ma sono i dataset.

C.6.1 **Caso New York Times e OpenAI**

New York Times ha fatto causa a OpenAI e Microsoft per violazione del copyright, accusando le aziende di aver utilizzato milioni di suoi articoli per addestrare i loro modelli di intelligenza artificiale senza autorizzazione o compenso. La causa, intentata nel dicembre 2023, è un punto di riferimento globale sul futuro del diritto d'autore nell'era dell'IA generativa.

C.6.2 FIEG contro Google AI

Google AI Overview rappresenta un *Traffic Killer*, ossia una criticità per il fatturato delle testate giornalistiche. FIEG sollecita l'AGCM affinché sensibilizzi la Commissione Europea ad aprire un procedimento ai sensi del **Digital Service ACT**.

C.7 AI Act

È una norma potente che nasce dopo 8 anni di elaborazione, proprio per la complessità della materia. L'obiettivo è stato quello di creare una bolla di AI sicura che garantisca i diritti fondamentali dei cittadini Europei dall'invasione di intelligenze artificiali sviluppate in Cina e negli USA. La norma, purtroppo, nella sua enorme complessità, nasce orfana già dalla sua promulgazione: nei giorni adiacenti alla sua redazione, viene rilasciata la prima AI generativa, creando una norma stroncata, incompleta. Riesce tuttavia a tutelare i cittadini dal profiling invasivo, e a tutelare il diritto d'autore.

C.8 AI Risk Pyramid

Il legislatore è consapevole di non poter creare delle categorie di intelligenze artificiali: definisce quindi una **piramide del rischio**, categorizzando ciascuna AI (attuale e futura) in opportuni livelli di questa piramide, in funzione del rischio che questa rappresenta per l'umanità. Ciascuna di queste categorie, rappresenta degli obblighi molto particolari.



Questa piramide tuttavia è stata soggetta ad un'importante proposta di rinvio.

C.9 L'Italia

L'Italia promulga una propria legge sull'AI, con la scusa di integrare l'AI Act. Copre tutti i settori della nostra economia, introducendo nuovi reati penali, tra cui la **manipolazione del consenso politico tramite IA**, aggravinggio e **manipolazione del mercato con IA**, un **aggravante "uso dell'IA"** e la **diffusione di DeepFake**.

C.9.1 "-umano" - Aggiunta all'articolo 1

Viene aggiunta pure la parola "umano" alla fine dell'articolo 1 LDA. Le opere dovranno essere frutto dell'ingegno **umano**.

C.9.2 Reati relativi all'uso dei dati

Introdotti nuovi reati per il:

- Text Data Mining illegittimo o non autorizzato (`robots.txt`).
- Uso dei dataset con opere protette senza base legale.
- Estrazione di contenuti da siti con Paywall e TOS.
- Copia di testi, immagini, contenuti protetti "per addestrare modelli AI".
- Aggiramento di opt-out del titolare DDA.
- Riproduzione sostanziale nell'output dell'IA.

C.10 Una soluzione ai problemi

La AI ha esaurito tutti i dati disponibili nel mondo reale, utili per il suo addestramento. Adesso si punta su quelli sintetici.

Appendice D

Seminario 2: *Computer Vision and Medical Imaging*

Dal professore e ricercatore del Dipartimento di Matematica e informatica dell'Università di Cagliari, Andrea Loddo.

D.1 Intelligenza Artificiale

Come siamo arrivati a parlare di intelligenza artificiale al giorno d'oggi? L'intelligenza artificiale non è altro che un termine per trattare degli elementi che fanno capo al Machine Learning. Non è sbagliato dire che **le fondamenta dell'AI sono radicate nel Machine Learning**.

D.1.1 Machine Learning

Non programmare esplicitamente la soluzione ad un problema, ma insegnare ad una macchina delle regole partendo da un insieme di dati che rappresenta il problema.

D.1.2 Machine learning e Deep Learning

Se il Deep Learning è alla base di molte delle soluzioni moderne, perché ha ancora senso parlare di Machine Learning?

Pertinenza del modello rispetto al task. Nel machine learning, le procedure di learning sono più rapide, e operano su meno dati rispetto al DL. I modelli sono più facilmente interpretabili. In un mondo fatto da così tanti task e problemi differenti, *non è opportuno parlare di modelli vecchi, ma piuttosto, di modelli adatti.*

Il machine learning è il punto di partenza. Non dobbiamo dimenticare che il Deep Learning nasce dal Machine Learning, e non può fare a meno dei suoi fondamenti.

D.2 Caso studio 1: radiomica

Per **radiomica** si intende l'**analisi delle immagini mediche** volta a ottenere, tramite opportuni metodi matematici e i calcolatori, **informazioni di tipo quantitativo** da queste **non rilevabili**

tramite la loro semplice osservazione visiva da parte dell'operatore. Le informazioni in questione dovranno essere **significative**, in quanto volte ad un supporto allo specialista, come nel caso delle diagnosi clinica.

Perché è rilevante? La radiomica permette di evitare esami molto invasivi, come le **biopsie**, alzando comunque il livello di accuratezza delle diagnosi. Il processo radiomico può essere integrato nel campo medico, al fine di migliorarlo. Dalle immagini è anche possibile ottenere rappresentazioni numeriche di ciò che il radiologo vede, o trovare pattern nascosti ma rilevanti.

Biomarcatori. Definiamo biomarcatore una caratteristica misurabile nell'organismo che fornisce informazioni oggettive sullo stato di salute. Gli output dei nostri modelli dovranno essere fortemente influenzate dai biomarcatori rilevati.

D.2.1 Pipeline radiomica

Acquisire le immagini. Raggi-X, risonanze, ultrasuoni, etc. Queste possono venire da **sorgenti e macchinari differenti**. Questi useranno **protocolli molto differenti**. Bisogna essere anche molto attenti sul cosa si acquisisce, evitando di catturare informazioni non-necessarie o fuorvianti. Insomma, acquisire i dati è uno step molto delicato.

Processing dell'input. Tra cui **resampling** e **quantizzazione**, per ridurre la risoluzione dell'informazione e dei livelli di grigio.

Segmentazione. Da un'immagine che rappresenta un'oggetto, ridurre l'analisi ad una sola parte dell'immagine. Può essere effettuata manualmente, in maniera semi-automatica o del tutto automatica. In alternativa, si possono delineare dei bounding-box, applicando un algoritmo di **object detection**.

Estrazione delle features. Un'immagine è una matrice di numeri. Cosa ce ne facciamo di questi numeri? Bisogna dare una forma sensata, e soprattutto **significativa** a questi numeri. Esistono vari modi di estrarre features. Tra questi: histogram based, texture based, shape based, transform based e deep features extraction.

Analisi delle features e dei campioni. È importante analizzare i campioni, per diagnosticare circostanze di **oversampling** e **undersampling** sulle varie classi. Si possono anche applicare tecniche di **normalizzare** delle features o di **riduzione delle features**, per evitare la *Curse of Dimensionality*¹. In particolar modo, il problema della riduzione delle features non è triviale in alcun modo: ogni tecnica ha infatti pro e contro. Creare features basate su quelle di partenza, crea nuovi dati. Estrarre un sottoinsieme di features, rischia di non rappresentare in maniera opportuno il dominio dei dati.

Costruzione e valutazione del modello. Tramite approcci supervised o unsupervised, e valutare i vari modelli tramite i vari parametri e metriche che è possibile calcolare.

¹La *Curse of Dimensionality* un fenomeno nel machine learning dove le prestazioni e l'efficacia degli algoritmi si deteriorano all'aumentare del numero di variabili (dimensioni) in un dataset, rendendo i dati sparsi e le relazioni difficili da trovare, aumentando esponenzialmente la complessità computazionale e il rischio di overfitting

D.2.2 Work 1: feature extraction

Presentiamo alcune conclusioni sulle tecniche usate in alcuni lavori effettuati dal gruppo di ricerca nel campo della radiomica.

Si preferiscono informazioni mirate. Le informazioni puntuali, relative alle zone di interesse, sono più comunicative rispetto alle features che rappresentano l'intera immagine.

Interpretabilità nei modelli. È difficile capire i motivi dietro agli output di alcuni modelli. Nel machine learning, tramite visualizzazioni come la **GradCam**, è possibile mappare in una heatmap le zone d'interesse principali del modello. Nel machine learning, per capire la rilevanza delle features, usiamo metodi come:

- **SHAP** (SHapley Additive exPlanations): questa, permette di capire la rilevanza di ciascuna delle features, e i motivi per cui il modello ha preso determinate decisioni, andando oltre la black-box.
- **Metodo della correlazione:** se due features sono estremamente correlate, ne basta una per dare informazioni pertinenti.

Questi due metodi si occupano quindi di analizzare rispettivamente la correlazione feature-label (desiderata), e la correlazione feature-feature (non desiderata). Queste metodologie permettono di effettuare scelte di feature reduction/selection opportune.

D.2.3 Work 2: Dynamic Feature Selection (DFS)

L'idea è quella di stabilire dinamicamente le features da considerare, in funzione dello stato del modello, del contesto o dell'input ricevuto. Un modello di input, permette di selezionare il peso di ognuna delle features in maniera opportuna. L'utilizzo di questa metodologia ha permesso di ottenere risultati strabilianti sul dataset MNIST, con un'accuracy in testing del 98% con una sola feature. Le performance calano su dataset a risoluzione maggiore e a colori: nonostante ciò, rispetto allo stato dell'arte degli altri algoritmi dinamici, anche un risultato come 0.18 in evaluation è significativo.

D.2.4 Work 3: Federated Feature Selection

Questa tecnica di feature selection si basa su alcune intuizioni che nascono dal **federated learning**.

Federated Learning È una tecnica di machine learning decentralizzata che addestra modelli di IA collaborativi su dati distribuiti, mantenendo le informazioni sensibili sui dispositivi locali senza condividerle. È opportuno al campo medico, in quanto i dati di training non possono essere diffusi.

Federated Feature Selection In questo caso, si contribuisce al modello federato non più con i pesi (che rappresentano come le immagini una forma di informazione sensibile relativa ai pazienti, al contrario di quello che si potrebbe pensare), ma con la rilevanza delle features. Questo permette al modello federato di stabilire il modo con cui i client debbano effettuare la feature selection. I risultati del modello server, valgono per i modelli di tutti i client: considerati due client A, B ,

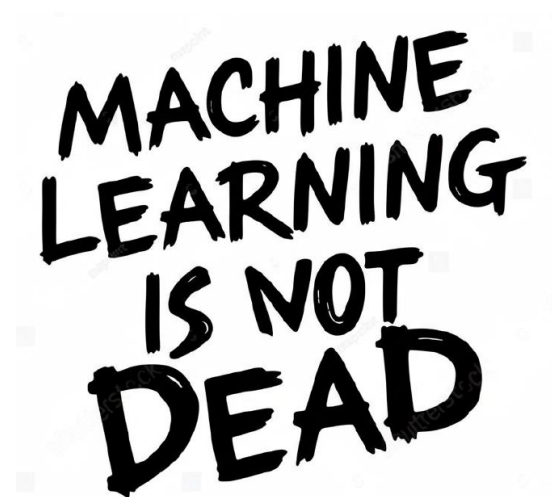
questi potrebbero usare dispositivi di acquisizione differenti e modelli molto differenti, ma a parità di tipologia di dati, le features significative sono le stesse.

D.3 Caso studio 2: immagini di cellule del sangue nei vetrini

Ho delle immagini ottenute da dei vetrini, su cui viene ossidato del sangue. Dai vetrini, tiro delle immagini al microscopio. L'immagine può comprendere tre sezioni d'interesse: globuli bianchi, globuli rossi e piastrine. Questo caso studio non è stato approfondito particolarmente a causa dei limiti di tempo, ma è stata offerta una analisi di alcune delle criticità principali, del task, e di alcune soluzioni.

D.4 Machine Learning is not dead.

Delle tecniche, che al giorno d'oggi sono considerate datate e fuorimercato, in realtà ci offrono le fondamenta per le tecniche ritenute moderne, e rimangono comunque viabili. Tutto dipende dal task che si sta affrontando, e da come i dati sono catturati e trattati.



Bibliografia

- [1] Ethem Alpaydin. *Introduction to Machine Learning*. 3rd. Cambridge, MA: The MIT Press, 2014. ISBN: 9780262028189.
- [2] Aditya Chattopadhyay et al. «Grad-CAM++: Improved Visual Explanations for Deep Convolutional Networks». In: *arXiv preprint arXiv:1710.11063* (2017). Accepted in the proceedings of IEEE Winter Conf. on Applications of Computer Vision (WACV 2018). arXiv: 1710.11063 [cs.CV]. URL: <https://arxiv.org/abs/1710.11063>.
- [3] Marc Peter Deisenroth, A. Aldo Faisal e Cheng Soon Ong. *Mathematics for Machine Learning*. Cambridge University Press, 2020. URL: <https://mml-book.com>.
- [4] Robert Geirhos et al. «ImageNet-trained CNNs are biased towards texture; increasing shape bias improves accuracy and robustness». In: *International conference on learning representations*. 2018.
- [5] Ian Goodfellow, Yoshua Bengio e Aaron Courville. *Deep Learning*. <http://www.deeplearningbook.org>. MIT Press, 2016.
- [6] Xueyan Mei et al. «RadImageNet: An Open Radiologic Deep Learning Research Dataset for Effective Transfer Learning». In: *Radiology: Artificial Intelligence* 0.1a (0), e210315. DOI: 10.1148/ryai.210315. eprint: <https://doi.org/10.1148/ryai.210315>. URL: <https://doi.org/10.1148/ryai.210315>.
- [7] Mustafa Ozuysal et al. «Fast Keypoint Recognition using Random Ferns». In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 32.3 (2010), pp. 448–461.
- [8] Maithra Raghu et al. «Transfusion: Understanding transfer learning for medical imaging». In: *Advances in neural information processing systems* 32 (2019).
- [9] Raúl Rojas. *Neural Networks: A Systematic Introduction*. Berlin, Heidelberg: Springer, 1996. ISBN: 978-3540605058.
- [10] Junhao Wen et al. «Convolutional neural networks for classification of Alzheimer’s disease: overview and reproducible evaluation». In: *Medical image analysis* 63 (2020), p. 101694.